Succinct Computational Secret Sharing for Monotone Circuits

George Lu
UT Austin
gclu@cs.utexas.edu

Shafik Nassar
UT Austin
shafik@cs.utexas.edu

Brent Waters
UT Austin and NTT Research
bwaters@cs.utexas.edu

May 14, 2025

Abstract

Secret sharing is a cornerstone of modern cryptography, underpinning the secure distribution and reconstruction of secrets among authorized participants. In these schemes, succinctness—measured by the size of the distributed shares—has long been an area of both great theoretical and practical interest, with large gaps between constructions and best known lower bounds. In this paper, we present a novel computational secret sharing scheme for monotone Boolean circuits that achieves substantially shorter share sizes than previously known constructions in the standard model. Our scheme attains a public share size of $n + \mathsf{poly}(\lambda, \log |C|)$ and a user share size of λ, where n denotes the number of users, C is the monotone circuit and λ is the security parameter, thus effectively eliminating the dependence on the circuit size. This marks a significant improvement over earlier approaches, which exhibited share sizes that grew with the number of gates in the circuit. Our construction makes use of indistinguishability obfuscation and injective one-way functions.

1 Introduction

Secret sharing [Sha79, Bla79, ISN89] is a way for an individual with a secret s to distribute n 'shares' of the secret to n parties in a way such that the secret remains hidden to any "unauthorized" subset of parties, while "authorized" subsets of parties can reconstruct the secret. The set of authorized parties is specified by some access policy, which is given as input to both the sharing and reconstruction algorithms. Secret sharing is a fundamental primitive with many applications, most notably in secure multi-party computation [BOGW88, CCD88, RBO89].

Traditionally, secret sharing guarantees information-theoretic privacy: even an unbounded adversary cannot learn anything about the secret from an unauthorized set of shares. It is known that there is a secret sharing scheme for any *monotone access policy* [BL88]. However, this classic construction gets the policy as a truth table of size 2^n, and produces a scheme where *each* share grows with the size of the truth table of the function, namely $O(2^n/\log n)$. A lot of effort has been put into minimizing the share size, and in recent years, a line of work [LVW18, LV18, ABF+19, ABNP20, AN21] managed to bring it down to $(1.5)^{n+o(n)}$. However, this remains far from the best known lower bounds [Csi96, Csi97].

More recently, a remarkable work by Applebaum et al. [ABI+23] demonstrated the power of *computational secret sharing* (CSS). As opposed to statistical security, CSS only has computational security and requires the secret to be hidden from *efficient* adversaries that hold an unauthorized set of shares. They show that CSS offers a dramatic advantage in terms of share size, by constructing a *succinct* CSS for arbitrary monotone functions with share size $\mathsf{poly}(n)$, as opposed to the exponential size of information-theoretic constructions. However, a crucial limitation is the work of [ABI+23] requires work proportional to the number of clauses in order to reconstruct the circuit. Indeed, *both* algorithms in the construction of [ABI+23] (may in general) run in time proportional to 2^n. And thus might only be efficiently computable when n is logarithmic in the security parameter. This may hold *even* if the function encoded in the truth table is efficiently computable (e.g., is derived from a monotone circuit).

In this work, we study succinct CSS for efficient representations of access policies. Specifically, we consider policies that are represented by *monotone circuits*. In this setting, we consider a CSS to be succinct if the share size

is independent of the circuit size, up to polylogarithmic factors. The classic Yao construction [Yao89, VNS+03] for monotone circuits gets shares and run time that grow with the number of *wires* in the circuit. The same work of [ABI+23] managed to improve upon that construction as well, and got the share size down to grow with the number of AND gates. To be precise, their construction achieves shares of size $\mathrm{poly}(\lambda, \log \ell)$ and public information of size $\ell_\wedge \cdot \lambda$, where ℓ is the number of gates and $\ell_\wedge$ is the number of AND gates. However, that construction does not offer an improvement for all monotone circuits, and is still, in fact, dependent on the size of the representation. So the following question remains open: *Is it possible to construct a succinct CSS scheme for monotone circuits, where the share size does not depend on the size of the circuit?*

We answer this question positively and prove the following theorem:

Theorem 1.1. *Assuming polynomially secure indistinguishability obfuscation and one-way injective functions, there exists a (standard model) succinct computational secret sharing scheme for general monotone circuits where the share size is λ and the public information size is $\mathrm{poly}(\lambda, \log \ell) + n$, where λ is the security parameter, n is the number of parties and ℓ is the size of the circuit. The sharing and reconstruction algorithms run in time $\mathrm{poly}(\lambda, \ell)$.*

We emphasize that n is the number of parties and not the circuit size, thus we get the first succinct CSS for general monotone circuits. Moreover, we only have an additive factor of n, which is not multiplied by any polylogarithmic factors or λ (as in [ABI+23]).

1.1 Technical Overview

The conceptual starting point of our construction is the classic computational secret sharing scheme of Yao [Yao82, VNS+03] for monotone Boolean circuits. Let C be a monotone circuit that has ℓ gates, including n special *input gates* that correspond to the input bits of the circuit. Each gate gets an index $i \in [\ell]$, and the indices $1, \ldots, n$ correspond to those input gates, while index ℓ corresponds to the output gate. Finally, we say that a gate i is a *child* of gate j if i computes a wire that feeds into j. Throughout this work, we assume that the indices are a topological order of the gates, which means that if i is a child of j then $i < j$.

In a very high level, each gate $i < \ell$ in the policy circuit C is assigned a random secret key K_i. The output gate ℓ is assigned the secret $K_\ell = s$. The keys must satisfy the following invariant: one can obtain K_i if and only if they posses sufficient keys of the children gates i_0, i_1, where the meaning of "sufficient" depends on the operation of the gate i:

- If i is an AND gate, then the two keys K_{i_0}, K_{i_1} are required to obtain K_i.
- If i is an OR gate, then any single one of the keys K_{i_0}, K_{i_1} is required to obtain K_i.

Each player $i \in [n]$ gets the key K_i as its private share. As a result, if any authorized set of players, represented by an n-bit string x, put their shares together, then by definition $C(x) = 1$ and they would manage to obtain the secret $K_\ell = s$.

Without going into details, the aforementioned invariant is facilitated using a symmetric key encryption scheme, but requires publishing as public information a ciphertext for *each wire* in the circuit. This results in public information that grows with the number of wires in the circuit.

Let us try to view the idea more abstractly: initially, each share i is a "proof" that whoever possesses it can satisfy input gate i. Similarly, each internal gate i has a similar proof attesting that it can be satisfied. The public information allows the initial proofs to be inductively combined to derive proofs on the subsequent gates, until we reach the output gate and retrieve the secret.

Natural iO-based construction. Through this perspective, we propose the following iO-based idea: replace the approximately ℓ ciphertexts with a single obfuscated program that generates gate proofs, if given sufficient proofs for its children gates. Since we plan to use iO, we start by suggesting an "iO friendly" proof for each gate. This proof is simply the classic symmetric-key puncturable signature scheme based on a puncturable pseudorandom function (PPRF) $F : \{0,1\}^\lambda \times [|C|] \to \{0,1\}^\lambda$ and a pseudorandom generator (PRG) $G : \{0,1\}^\lambda \to \{0,1\}^{2\lambda}$. We recall it:

- The key is a PPRF key K.

- The signature on an index i is simply $\sigma_i = F(K, i)$.
- The verification algorithm on a key K, a message i and a signature σ_i simply checks that $G(F(K, i)) = G(\sigma_i)$.

The obfuscated program needs access to the description of the circuit, but we cannot afford to embed the entire circuit in the program. Instead, we use a succinct hash with succinct local openings and embed the digest of the circuit C. For now, imagine we use a somewhere-statistically binding hash (SSB) [HW15] to compress the circuit description. We justify later that we can replace the SSB with a pure iO primitive, like a positional accumulator [KLW15]. We represent the circuit C as a vector of gate descriptions, where the description of gate i specifies the children of gate i and the operation that gate i performs.

The dealer generates a program P_{SS} that has hardcoded the secret s, a hash key, a signature key and a succinct hash of the circuit C using the SSB. The program gets as input a gate index i, a description (g_i, i_0, i_1) of gate i, an opening to that gate in the hash of C and signatures on its children gates $\sigma_{i_0}, \sigma_{i_1}$, and does the following:

1. Verifies the opening of the hash in position i to the given gate description (g_i, i_0, i_1).
2. Computes a bit v_0 (resp. v_1) that indicates whether the signature σ_{i_0} (resp. σ_{i_1}) verifies w.r.t. index i_0 (resp. i_1).
3. Computes the bit evaluated by gate i as $v = g_i(v_0, v_1)$.
4. If $v = 0$, the program outputs $\perp$. Else,
5. If i is an internal gate, it outputs the signature on gate i. If i is the output gate it outputs the secret s.

The public information consists of the obfuscated program $i\mathcal{O}(P_{\mathsf{SS}})$ and the hash key. As in [Yao82, VNS+03], share i is the signature (proof) on input wire i (which we view as an input gate). The reconstruction algorithm gets the circuit description C for free, and uses it to compute the openings to the hash of C w.r.t. the given hash key. It then uses the shares it possesses (alongside the hash openings) to inductively generate gate signatures using the obfuscated program until the program reveals the secret.

Regarding efficiency, we note that this construction is fully succinct. Each private player share is still a single signature. The public information consists only of a hash key and the obfuscation of the program P_{SS}: this program includes a succinct hash of C, and is otherwise independent of C or the number of players, therefore this share has size $\mathrm{poly}(\lambda, \log |C|)$.

Difficulty of Proving Security. We quickly recall the (selective) security game of a secret sharing scheme: an adversary $\mathcal{A}$ submits a policy $C : \{0,1\}^n \to \{0,1\}$, an unauthorized set $x \in \{0,1\}^n$ and a pair of secrets s_0, s_1. The challenger chooses one secret s_b randomly, secret shares it, and sends to $\mathcal{A}$ the shares $\{\mathsf{sh}_i\}_{x_i=1}$ corresponding to the set x. The adversary wins if it can guess the secret that was shared.

Intuitively, our construction seems secure because the adversary does not have enough shares to force the evaluation of the circuit to be 1. By the monotonicity of the circuit C, the only way to flip a gate value from 0 to 1, is to flip one of its children gates. Hence, the task boils down to proving that the adversary cannot compute signatures on gates that are supposedly 0 in the evaluation of C on x. If we have ideal obfuscation then it is straightforward to do this task.

Unfortunately, assuming "just" iO, it is not clear how to do so without having the program size depend on the circuit. We elaborate further. The proof strategy is to use a hybrid argument to gradually transition to a game where the signature is punctured on the output gate, thus P_{SS} would be equivalent to a program that never outputs s_b, therefore we can substitute it out by relying on iO security.

An input gate i (corresponding to a share that was not requested by the adversary) can be punctured using the standard PRG and iO trick: first we switch P_{SS} to a program that has $z = G(F(K, i))$ hardcoded using iO security, then we switch z to a uniform string over $\{0,1\}^{2\lambda}$ using PRG and PPRF security, and finally switch the program to one that always fails the check on index i using iO security and the fact that with all but negligible probability, the uniform z is not in the image of the PRG.

Now imagine we want to puncture an internal gate i, and assume it is an OR gate. To do so, we need to switch from a functionally equivalent program that never outputs the signature on i in the first place, and this can only be

done if both children gates to i are punctured simultaneously. This step crucially relies on the monotonicity of the circuit: we ensure i cannot be flipped by ensuring that its children cannot be flipped.

The problem is that those children gate indices should remain punctured until all of their "parents" (the gates into which the punctured children directly feed) are also punctured. Even if we assume a layered circuit (wires from gates in layer i feed only into gates in layer $i+1$), we would have to puncture the signature on an entire layer, which results in punctured PPRF keys of size $\mathsf{poly}(W)$ (where W is the width of the circuit C).

Warm up: additive dependence on the circuit size. Let $x \in \{0,1\}^n$ be the challenge set submitted by the adversary. If the adversary is admissible, then $C(x) = 0$. We extend x to $\mathbf{x} = x_1 \cdots x_\ell$ as follows: for each $i \in [\ell]$, let x_i be the value of gate i in the evaluation of C on x. We view $\mathbf{x}$ as the *honest circuit evaluation* of the challenge set x.

Now imagine the obfuscated program has access to $\mathbf{x}$. Then we can modify it to gradually refuse to give out signatures on gates such that $x_i = 0$. That is, for each $i^* \in [\ell]$ we define a program $P_{\mathsf{SS}}^{i^*}$ (which takes the same inputs as program P_{SS}) as follows:

1. Verifies the opening of the hash in position i to the given gate description (g_i, i_0, i_1).
2. Computes a bit v_0 (resp. v_1) that indicates whether the signature σ_{i_0} (resp. σ_{i_1}) verifies w.r.t. index i_0 (resp. i_1).
3. Computes the bit evaluated by gate i as $v = g_i(v_0, v_1)$.
4. If $i < i^*$ and $x_i = 0$ then overwrite $v \leftarrow 0$.
5. If $v = 0$, the program outputs $\perp$. Else,
6. If i is an internal gate, it outputs the signature on gate i. If i is the output gate it outputs the secret s.

The idea is that the string $\mathbf{x}$ encodes what gate indices need to be punctured, and the index i^* keeps track of what has been punctured so far. Together, $\mathbf{x}$ and i^* actually enforce the puncturing in the obfuscated program. This mechanism allows us to overcome the obstacle of puncturing an internal gate, without having to simultaneously puncture the signature key on its input gates.

In more detail, imagine we are already in Hybrid i^* that uses $P_{\mathsf{SS}}^{i^*}$. Now, in order to enforce the honest circuit evaluation on gate i^* (assuming that gate i^* should evaluate to 0 given the shares in the possession of the adversary, i.e., $x_{i^*} = 0$), we do the following:

- First, we make the hash key statistically binding on position i^*. This ensures that the program $P_{\mathsf{SS}}^{i^*}$ only verifies the correct gate description in Step 1.
- Next, we puncture the signature on the relevant children gates of i^*, i.e., denote the children of i^* by i_0^* and i_1^* and puncture on the set $S = \left\{ i_b^* : x_{i_b^*} = 0 \right\}$. The idea is that since $i_b^* < i^*$, then the program $P_{\mathsf{SS}}^{i^*}$ never signs the indices in S by the check in Step 4, so we can safely puncture the signing key on S.
- Now that the signing key is punctured on the set S, bit v will never be assigned the value 1 by the definition of the set S and the fact that $x_{i^*} = g_i(x_{i_0^*}, x_{i_1^*}) = 0$, therefore we can safely extend the check in Step 4 to include index i^*, meaning that we advance to using program $P_{\mathsf{SS}}^{i^*+1}$.
- Finally, we undo the changes we made to the hash and signature key: return the hash key to be in normal mode and the signature key to be unpunctured. Now, we are in Hybrid $i^* + 1$.

This processes continues inductively until we reach Hybrid $\ell + 1$. By the admissibility of the adversary, it must be that $x_\ell = C(x) = 0$. Therefore, when we switch to program $P_{\mathsf{SS}}^{\ell+1}$, we know that it will never output the secret. This allows us to switch to a program $P_{\mathsf{SS}}^{\perp}$ that does not have the secret s hardcoded at all.

Now, we address the long string in the room. Of course, we cannot hardcode all of $\mathbf{x}$ in the program, since that would blow up the size of the program. Instead, in the real construction:

- The dealer gives out a uniform random string $\mathbf{w}$ as public information.

- A hash $h_{\mathbf{w}}$ of $\mathbf{w}$ is hardcoded into the program P_{SS}. Imagine, for now, that we use a SSB hash here as well.
- Program P_{SS} now expects a valid opening of $h_{\mathbf{w}}$ on index i whenever it is asked to sign gate i.

In the security proof, the first step is to switch the way the dealer samples $\mathbf{w}$ to a pseudorandom one-time pad of $\mathbf{x}$ using a PRF, i.e., the dealer samples a PRF key K_{bit} and sets $w_i = x_i \oplus F(K_{\mathsf{bit}}, i)$. Next, the dealer hardcodes the key K_{bit} into the obfuscated program, which allows it to decode x_i given the opening of $h_{\mathbf{w}}$ on index i.

Why SSB is not necessary. Although our construction requires a somewhere-statistically binding property of the hash in order to be compatible with iO, we point out that the honest dealer (i.e., the challenger in the security game) generates the hash keys *after* the adversary has already declared the message to be hashed. This allows us to use a weaker non-adaptive notion of security, similar to that of *positional accumulators* [KLW15]. Koppula et al. [KLW15] showed that positional accumulators can be constructed from iO and OWFs.

Fully succinct CSS in the CRS model. Our warm up construction already gives a fully succinct computational secret sharing in the common (unstructured) random string model. This is due to the fact that $\mathbf{w}$ is uniformly random in the actual construction and is pseudorandom in the security analysis, which allows us to program the CRS in the security proof and argue indistinguishability. With the string $\mathbf{w}$ out of the way, this leaves us with shares of size λ and public information of size $\mathsf{poly}(\lambda, \log |C|)$. The CRS can be further "compressed" in the random oracle model. The fact that we can get iO-based fully succinct CSS in the CRS model was already pointed out in [ABI+23] as a consequence of [HW15] and other follow up works [HIJ+16, BCG+19, ASY22, BFK+19] that can generically compress shares on existing secret sharing schemes. Here, we just point out that our techniques (so far) give an alternative construction of succinct CSS in the CRS model, which is both simpler and only requires OWFs in addition to iO, as opposed to SSB and iO. However, this is not satisfying since the CRS in our warm up construction and previous constructions is long and grows with the circuit size, therefore does not yield a succinct CSS in the standard model.

Abstraction: circuit evaluation embedding. We now describe how to replace $\mathbf{w}$ with a shorter string. Recall that our technique requires embedding the evaluation of a circuit over ℓ gates. Furthermore, the embedded string has to be hidden from the adversary. While at first it may seem like we need at least ℓ bits since we are effectively encrypting an ℓ-bit string $\mathbf{x}$, we recall that $\mathbf{x}$ admits a very short description *given* the circuit C: it is merely the evaluation of every gate in C on an input x of length n. In other words, given a string $x \in \{0,1\}^n$, we need a way to compress an encryption $\mathbf{w}$ of the full evaluation $\mathbf{x}$ into a short string α in a way that allows anyone with access to α and C to expand back to $\mathbf{w}$. At the same time, we want the string α to completely hide the x. In addition, there should be a trapdoor td that allows for recovering x_i from each w_i. We call this a *circuit evaluation embedding scheme* (CEE) and define it formally in Section 3.

Simple CEE from fully homomorphic encryption. Although our goal is to construct succinct CSS purely from iO, it is useful to see first a simple CEE construction based on *fully homomorphic encryption* (FHE) and PRFs. The natural idea is to encrypt the input string using fully homomorphic *hybrid* encryption [GGI+15], and then homomorphically evaluate a ciphertext for each gate value. We recall that hybrid encryption works as follows: to encrypt a string x, we use the underlying encryption scheme to encrypt a PRF key K, then use the PRF as a "one-time pad" to hide x. We describe the construction:

- To embed the evaluation of a circuit C on a string $x \in \{0,1\}^n$, the embedding algorithm samples a PRF key K with domain $[n]$ and codomain $\{0,1\}$ and a FHE key pair $(\mathsf{pk}, \mathsf{sk})$, computes $w = x \oplus F(K,1) \cdots F(K,n)$ and encrypts K under the FHE to produce a ciphertext ct_K. It then outputs $\alpha = (\mathsf{pk}, \mathsf{ct}_K, w)$ which is simply a public key pk and "hybrid FHE" ciphertext hiding the string x. The trapdoor is the secret key sk.
- We can expand α into an *encryption* of $\mathbf{w}$ as follows: define the function $f_{C_i,w}(K) = C_i(w \oplus F(K,1) \cdots F(K,n))$, where $C_i : \{0,1\}^n \to \{0,1\}$ is the circuit computing gate i in the circuit C. Anyone with access to $\alpha = (\mathsf{pk}, \mathsf{ct}_K, w)$ can homomorphically compute $\mathsf{ct}_i = \mathsf{Eval}(\mathsf{pk}, \mathsf{ct}_K, f_{C_i,w})$.

- To extract a bit x_i from ct_i using the trapdoor sk, we simply decrypt the ciphertext $w_i = \mathsf{Dec}(\mathsf{sk}, \mathsf{ct}_i)$ and $K = \mathsf{Dec}(\mathsf{sk}, \mathsf{ct}_K)$, and then compute $x_i = F(K, i) \oplus w_i$.

The hiding property of the CEE follows by the semantic security of the hybrid FHE scheme.

Note that already, this gives us the first succinct CSS for monotone circuits based on iO and FHE: the public information has size $\mathsf{poly}(\lambda, \log |C|) + n$, where $\mathsf{poly}(\lambda, \log |C|)$ comes from the FHE public key and the encryption of the PRF key as well as the obfuscated program described in the warm up, and the n term is the string w. Each share remains of size λ as in the warm up[1].

iO-based construction of CEE. At a high level, the construction of our CEE looks similar to the construction of our secret sharing scheme. Our CEE scheme consists of a short input-encoding digest w and the obfuscation of a small program P_{CEE} which evaluates w with respect to the policy circuit C. In the same way our secret sharing program P_{SS} is tasked with iteratively verifying the honest circuit evaluation $\mathbf{x}$ bit by bit from a challenge input x, our CEE program P_{CEE} has the task of computing evaluation embedding $\mathbf{w}$ bit by bit from input-encoding digest w. On the surface, it seems like we run into the same issues as our secret sharing scheme, having simply delayed the problem. However, the key difference here is that since an adversary can potentially try to decrypt with any subset of the shares they are given, P_{SS} needs to support correctly validating $\mathbf{x}$ on potentially exponential subsets of the challenge input x. However, w is the *only* input which program P_{CEE} needs to be evaluated on. Thus, instead of producing signatures for individual wire values, we can instead compute a signature attesting to the *entire* computation of $w_1, \ldots w_i$ thus far.

Concretely, for any $i \in [\ell]$, to compute w_i, our program P_{CEE} takes as input a hash $h_{\mathbf{w}_{[i-1]}}$ of the first $i-1$ bits of $\mathbf{w}$ computed thus far, along with a signature from the previous invocation of P_{CEE} that this was computed correctly. This way, when looking at gate $C_i = (g_i, i_0, i_1)$, rather than checking that for signatures directly on bits w_{i_0}, w_{i_1}, our program can instead directly verify a signature on $h_{\mathbf{w}_{[i-1]}}$ in conjunction with local openings of this hash on indices i_0, i_1. Of course, to enable computation to continue for larger values of i, we also need P_{CEE} to output a signature of the hash of our extended prefix, $\mathbf{w}_{[i]}$. Since P_{CEE} needs to be small, it cannot be represent $\mathbf{w}_{[i]}$ directly, despite the decoder having already recovered the first $i-1$ bits in the clear. Fortunately, we can take advantage of the 'writeability' of positional accumulators, which allows our program to directly compute $h_{\mathbf{w}_{[i]}}$ from $h_{\mathbf{w}_{[i-1]}}$ and w_i.

Putting everything together, to compute bit w_i, our program P_{CEE} hard codes a hash key, signature key, hash h_C of circuit C, PRF key K_{bit}. It takes as input the local opening of h_C on gate i to a gate description (g_i, i_0, i_1), the 'prefix hash' $h_{\mathbf{w}_{[i-1]}}$ of the first $i-1$ bits of $\mathbf{w}$, and a signature σ_{i-1} on this hash, and local openings of $h_{\mathbf{w}_{[i-1]}}$ on bits i_0, i_1.

1. Verify the local opening of h_C as a gate description (g_i, i_0, i_1).
2. Verify that σ_{i-1} is a valid signature of $h_{\mathbf{w}_{[i-1]}}$.
3. Verify local openings of $h_{\mathbf{w}_{[i-1]}}$ to bits i_0, i_1, producing bit values w_{i_0}, w_{i_1} respectively.
4. Recover honest circuit evaluation $x_{i_b} = w_{i_b} \oplus F(K_{\mathsf{bit}}, i_b)$ for $b \in \{0, 1\}$ and compute $x_i = g_i(x_{i_0}, x_{i_1})$.
5. Reencode $w_i = x_i \oplus F(K_{\mathsf{bit}}, i)$, and compute $h_{\mathbf{w}_{[i]}}$ from $h_{\mathbf{w}_{[i-1]}}, w_i$.
6. Compute signature σ_i of $h_{\mathbf{w}_{[i]}}$ and output w_i, σ_i.

Now if we set input $w = x \oplus F(K_{\mathsf{bit}}, 1), \ldots F(K_{\mathsf{bit}}, n)$, this program exactly recovers the "additive dependence on circuit size" construction described previously.

Just like with secret sharing, the primary technical challenge here will be arguing computational hiding of our embedded 'message', in this case, the challenge input x. While this is simple to see when $\mathbf{w}$ is given out directly, now our program P_{CEE} includes the PRF key K_{bit} used to mask honest evaluation $\mathbf{x}$. While we cannot hope to argue that $\mathbf{w}$ appears pseudorandom, since we are giving a compact representation of it, we *can* argue that $\mathbf{w}$ is indistinguishable from a simple PRF evaluation $F(K_{\mathsf{bit}}, 1) \ldots F(K_{\mathsf{bit}}, \ell)$ independent of challenge input x. The indistinguishability of $F(K_{\mathsf{bit}}, 1) \ldots F(K_{\mathsf{bit}}, \ell)$ to our 'true' encoded input $\mathbf{w}$ proceeds via a series of hybrids. At a high level, our goal will be to ensure that on Hybrid i, the local openings to the gate (g_i, i_0, i_1) and encoding bits w_{i_0}, w_{i_1} are statistically fixed. To achieve this, we will need two properties:

[1]Note while this construction doesn't yield a uniform 'uncompressed' string unlike our prior toy example and our full iO construction, it is sufficient for our purposes as secret sharing program P_{SS} can contain the hard-coded FHE key sk in the proof

1. On input hash h_{i-1} and signature σ_{i-1}, the signature will *only* verify if $h_{i-1} = h_{\mathbf{w}_{[i-1]}}$.

2. Hash h_C is statistically binding on index i, and hash $h_{\mathbf{w}_{[i-1]}}$ is statistically binding on indices i_0, i_1.

Once we statistically fix w_{i_0}, w_{i_1}, it is easy to see that whether w_i is computed with or without circuit embedding, P_{CEE} there is only one bit value which P_{CEE} can output on index i. This allows us to leverage $i\mathcal{O}$ security to puncture K_{bit} at index i, giving us indistinguishability of the two cases. Finally, to tie all our hybrids together, we require a pebbling argument [GPSZ17] to ensure the total number of points in which we need to hard code property 1 in our obfuscated program is logarithmic in ℓ. We refer to Section 5 for the full construction and analysis of our circuit evaluation embedding.

1.2 Related Approaches

Non-interactive Batch Arguments. A *non-interactive batch argument for* NP (BARG) [CJJ21a, CJJ21b] is a proof system allowing for the verification of a batch of NP statements essentially at the price of one, meaning that the proof size grows sub-linearly in the batch size. A *monotone-policy BARG* [BBK+23, NWW24] allows the verifier to check that the set of true statements satisfies some monotone policy, described by a monotone circuit. Brakerski et al. [BBK+23] show how to get monotone policy BARGs from FHE with a CRS of size roughly $n \cdot \ell_{\mathsf{FHE}}$, where ℓ_{FHE} is the size of a single FHE ciphertext. We observe that their CRS size can be reduced to $n + \ell_{\mathsf{FHE}}$ using the standard hybrid FHE technique used in the FHE-based CEE construction we described before.

The notion of monotone-policy BARGs bears a structural resemblance to secret sharing, as both involve enforcing monotone access structures; in both settings, a monotone circuit determines whether a given set of inputs (witnesses for BARGs, shares for secret sharing) satisfies the access policy. Despite these similarities, techniques from monotone-policy BARGs do not directly translate to succinct CSS. We believe it would be interesting to establish formal connections or generic compilers between the two settings.

iO for Turing Machines. From a high level, our tools and techniques are somewhat similar to those used in the context of iO for Turing machines [BGL+15, CHJV15, KLW15]. As a first attempt, one might try to use iO for Turing machines in a "black-box" way to get succinct CSS. Indeed, if we consider access policies that can be computed by a Turing machine of size M, using iO for TMs with unbounded space [KLW15], would give us an obfuscated program, and thus secret sharing scheme, of size $\mathsf{poly}(\lambda, n, M)$. This approach has two significant drawbacks: first, since the public information is polynomial in M, it limits the class of circuits that we can handle to circuits with a short Turing Machine description, as opposed to our general result that handles any circuit. Second, even for small M, the public information of the CSS would be polynomial in n, as opposed to just having a simple additive factor of n as in our result.

Still, the problem we tackle here resembles the problem of constructing iO for TMs: in both cases, a 'large' computation is compressed into a compact, circuit-based obfuscated program that locally computes an encoding of the next tape cell (in the Turing machine setting) or circuit wire (in our setting). The output of this program is then used as input in subsequent invocations of the obfuscated program to further advance the computation. To ensure an adversary does not deviate from this intended evaluation path, the obfuscated program attaches a signature to each encoding which is later verified when that encoding is used as input. Despite the similarities, our setting differs in a couple of key ways, which are reflected in our construction:

- **Succinctness in circuit size** - iO for TMs achieves succinctness in runtime and space usage, while our goal is to compress the description of the circuit. Indeed, the obfuscated program in iO for TMs *does* grow with the description of the machine, making it unlikely that a direct application of iO for TMs would yield the desired succinctness. We overcome this problem using the fact that the circuit description is given 'for free' to the sharing and reconstruction algorithms, and utilizing the succinctness of PAs/SSB hash to embed a binding succinct description of the circuit into the obfuscated program.

- **Iterating over input space** - In the iO for TMs setting, security reductions typically iterate over the entire input space, thus causing the obfuscated program size to be polynomial in the input length, as well as incur a subexponential loss in security. On the surface, our construction of secret sharing looks like it could necessitate

a similar exhaustive approach, as our security proof needs to argue the verification program remains secure when evaluated on *any* subset corrupted shares. However, we crucially rely on the monotonicity of the access policy to prove a weaker but still sufficient property in the CSS security game: that the adversary is not able to flip the evaluation of any gate from 0 to 1. Extending this invariant to the output gate suffices to show overall security. [2]

Our construction can be viewed as being inspired by the KLW[KLW15] "machine hiding encodings" as a starting point where we make important modifications to (1) handle circuit descriptions (instead of Turing Machine descriptions), (2) simplified to only write memory once and (3) replacing the splittable signatures and their related security used in Koppula, Lewko and Waters [KLW15] with a pebbling argument [GPSZ17]. We remark that one could potentially also arrive at a circuit evaluation embedding by starting from other machine or randomized encoding schemes such as Garg and Srinivasan [GS18].

Signature-Based Witness Encryption Finally, we note our work shares some technical similarities in that the work of Avitabile et. al. [ADM+25] in building *signature-based witness encryption (SbWE) with compact ciphertexts*, both using $i\mathcal{O}$ based techniques from the $i\mathcal{O}$ for TMs literature discussed previously to realize succinct cryptography. Furthermore, since signature-based witness encryption can be viewed as a reusable generalization of secret sharing, constructions of compact SbWE can directly be translated to a (succinct) secret sharing scheme. However, their work only considers threshold policies, a domain where succinct secret sharing is easy to construct. We think it is an interesting open question to try and extend our techniques to the setting of SbWE for general policies.

2 Preliminaries

Notation. We use $[n]$ to refer to the set $\{1, 2, \ldots n\}$. For a subset $S \subseteq [n]$, we say $x \in \{0,1\}^n$ is the characteristic vector of S if the i^{th} bit of x is 1 iff $i \in S$. Given a string $x \in \{0,1\}^n$, we use S_x to denote the set for which x is the characteristic vector.

Boolean Circuits. We represent boolean circuits $C : \{0,1\}^n \to \{0,1\}$ with ℓ gates (including inputs) as an ordered list of $\ell - n$ tuples of the form $C_i = (g_i, i_0, i_1)$. The indices $i \in [n]$ are reserved for input wires, while each internal gate gets a unique index $i \in [n+1, \ell]$. For each $C_i = (g_i, i_0, i_1)$, the variable $g_i \in \{\mathsf{AND}, \mathsf{OR}, \mathsf{NOT}\}$ denotes the operation that is computed by gate i and the indices $i_0, i_1 < i$ denote the indices of the gates that compute the input wires to gate i (only i_0 for NOT gates). The gates i_0, i_1 are referred to as the children of gate i. We take the ℓ^{th} gate C_ℓ to be the output of the circuit. For convenience, we extend the representation of C to an ordered list of length ℓ, where the first n elements are simply the indices $1, \ldots, n$ denoting the input gates. We refer to boolean circuits without NOT gates as monotone boolean circuits. For any input $x \in \{0,1\}^n$ and index $i \in [\ell]$, we denote by $C(x)[i]$ the value that gate i takes in the evaluation of C on input x. We let $g(b_0, b_1)$ to denote the evaluation function evaluating a gate g on bits b_0, b_1.

2.1 Computational Secret Sharing

Definition 2.1 (Access Structure [Bei96])**.** For any set $S = \{s_1, \ldots, s_n\}$, let 2^S denote the power set of S (the set of all subsets of S). An access structure on S is a set $\mathbb{A} \subseteq 2^S \setminus \varnothing$ of non-empty subsets of S. We refer to the elements of $\mathbb{A}$ as the *authorized* sets and those not in $\mathbb{A}$ as the *unauthorized* sets. We say an access structure is *monotone* if for all sets $S_1, S_2 \in 2^S$, if $S_1 \in \mathbb{A}$ and $S_1 \subseteq S_2$, then $S_2 \in \mathbb{A}$. We say a program P implements an access structure if P takes as input a string $x \in \{0,1\}^n$ and $P(x) = 1$ iff $\{s_i \mid x_i = 1\} \in \mathbb{A}$ (and $P(x) = 0$ otherwise).

We now define the main notion of computational secret sharing. While (computational) secret sharing has been defined in many prior works, our most closely follows that of [ABI+23].

[2] One notable exception that constructs iO for TMs without iterating over the input space is the work of Jain and Jin [JJ22] which utilizes *mathematical proofs of equivalence* between TMs to argue security. An intriguing direction for future work is exploring whether the mathematical proofs framework could eliminate the additive dependence on n in our setting.

Definition 2.2 (Computational Secret Sharing)**.** Let λ be a security parameter. A computational secret sharing (CSS) scheme with supporting programs $\mathcal{P} = \{\mathcal{P}_\lambda\}$, secret space $\mathcal{S} = \{\mathcal{S}_\lambda\}$ is a tuple of efficient algorithms $\Pi_{\mathsf{CSS}} = (\mathsf{Share}, \mathsf{Recon})$:

- $\mathsf{Share}(1^\lambda, C, s) \to (\mathsf{sh}_0, \mathsf{sh}_1, \ldots, \mathsf{sh}_n)$: On input security parameter λ, a monotone Boolean circuit $C : \{0,1\}^n \to \{0,1\}$, and secret $s \in \mathcal{S}$, the share algorithm generates a set of shares $(\mathsf{sh}_0, \mathsf{sh}_1, \ldots, \mathsf{sh}_n)$ where sh_0 is the public information, and $\mathsf{sh}_1, \ldots, \mathsf{sh}_n$ are the shares distributed to the n parties.
- $\mathsf{Recon}(C, x, \mathsf{sh}_0, \{\mathsf{sh}_i\}_{i \in S_x}) \to s \cup \perp$: On input monotone Boolean circuit $C : \{0,1\}^n \to \{0,1\}$, input x, public information sh_0, and set of user shares $\{\mathsf{sh}_i\}$ on indices $\in S_x$, the reconstruction algorithm outputs the secret s (or $\perp$ for a reconstruction error).

We refer to share size $\mathsf{shsize} = \mathsf{shsize}(\lambda, |C|)$ to denote the maximum size of individual shares $\max\{\mathsf{sh}_i \mid i \in [n]\}$ in the above scheme and public information size to denote the size of sh_0. We require the following properties:

- **Correctness:** For all security parameters λ, secrets $s \in \mathcal{S}$, monotone Boolean circuits $C : \{0,1\}^n \to \{0,1\}$ and inputs x where $C(x) = 1$, we have that

$$\Pr[\mathsf{Recon}(C, x, \mathsf{sh}_0, \{\mathsf{sh}_i\}_{i \in S_x}) = s : \mathsf{Share}(1^\lambda, C, s) \to (\mathsf{sh}_0, \mathsf{sh}_1, \ldots, \mathsf{sh}_n)] = 1$$

- **Security:** For a bit $b \in \{0,1\}$ and an adversary $\mathcal{A}$, define the secret sharing security game $\mathsf{ExptCSS}_{\mathcal{A}}(\lambda, b)$ as follows:

  1. Algorithm $\mathcal{A}(1^\lambda)$ chooses a policy C, an unauthorized input x where $C(x) = 0$ and a pair of secrets s_0, s_1.
  2. The challenger samples $(\mathsf{sh}_0, \mathsf{sh}_1, \ldots, \mathsf{sh}_n) \leftarrow \mathsf{Share}(1^\lambda, C, s_b)$ and returns $\mathsf{sh}_0, \{\mathsf{sh}_i\}_{i \in S_x}$.
  3. Algorithm $\mathcal{A}$ outputs a bit $b' \in \{0,1\}$, which is the output of the experiment.

  We say Π_{CSS} is secure if, for all efficient adversaries $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\left|\Pr[\mathsf{ExptCSS}_{\mathcal{A}}(\lambda, 0) = 1] - \Pr[\mathsf{ExptCSS}_{\mathcal{A}}(\lambda, 1) = 1]\right| = \mathsf{negl}(\lambda).$$

Without loss of generality, we may assume that the first step that $\mathcal{A}$ runs is deterministic. That is, once $\mathcal{A}$ is fixed then P, x and the pair s_0, s_1 are fixed.

2.2 Building Blocks

In this section, we will review the ingredients we will use to build our secret sharing scheme.

Definition 2.3 (Indistinguishability Obfuscation [BGI+12, GGH+13])**.** Let $\mathcal{C} = \{\mathcal{C}_\lambda\}_{\lambda \in \mathbb{N}}$ be a family of polynomial-size circuits. An indistinguishability obfuscator $i\mathcal{O}$ is an efficient algorithm that takes as input the security parameter λ, a circuit $C \in \mathcal{C}_\lambda$ and outputs a circuit C'. An $i\mathcal{O}$ scheme should satisfy the following properties:

- **Functionality-preserving:** For all security parameters $\lambda \in \mathbb{N}$, all $C \in \mathcal{C}_\lambda$, and all inputs x, we have that $C'(x) = C(x)$ where $C' \leftarrow i\mathcal{O}(1^\lambda, C)$.
- **Security:** For all efficient (possibly non-uniform) adversaries $\mathcal{A} = (\mathsf{Samp}, \mathcal{A}')$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that the following holds: if for all security parameters $\lambda \in \mathbb{N}$,

$$\Pr[\forall x, C_0(x) = C_1(x) : (C_0, C_1, \mathsf{st}) \leftarrow \mathsf{Samp}(1^\lambda)] = 1 - \mathsf{negl}(\lambda),$$

  then

$$\left|\Pr[\mathcal{A}'(\mathsf{st}, i\mathcal{O}(1^\lambda, C_0)) = 1] - \Pr[\mathcal{A}'(\mathsf{st}, i\mathcal{O}(1^\lambda, C_1)) = 1]\right| = \mathsf{negl}(\lambda),$$

  where $(C_0, C_1, \mathsf{st}) \leftarrow \mathsf{Samp}(1^\lambda)$.

Definition 2.4 (Pseudorandom Generator)**.** Let λ be a security parameter, $n = n(\lambda)$ be a seed length, and $\ell = \ell(\lambda)$ be an output length. A pseudorandom generator $\mathsf{PRG}\colon \{0,1\}^n \to \{0,1\}^\ell$ is an efficiently-computable function such that for all efficient adversaries $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\left|\Pr\left[\mathcal{A}(\mathsf{PRG}(s)) = 1 : s \xleftarrow{R} \{0,1\}^n\right] - \Pr\left[\mathcal{A}(r) = 1 : r \xleftarrow{R} \{0,1\}^\ell\right]\right| = \mathsf{negl}(\lambda).$$

We extend the security game to adversaries that request a constant number of samples $c \in \mathbb{N}$, that is, for all efficient adversaries $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\left|\Pr\left[\mathcal{A}(\mathsf{PRG}(s_1), \ldots, \mathsf{PRG}(s_c)) = 1\right] - \Pr\left[\mathcal{A}(r_1, \ldots, r_c) = 1\right]\right| = \mathsf{negl}(\lambda),$$

where $s_i \xleftarrow{R} \{0,1\}^n$ and $r_i \xleftarrow{R} \{0,1\}^\ell$. The two security games are computationally equivalent by a standard hybrid argument.

Definition 2.5 ((Injective) One Way Functions and Hardcore Bits)**.** Let λ be a security parameter, $n = n(\lambda)$ be a seed length, and $\ell = \ell(\lambda)$ be an output length. We say a function $\mathsf{hcb}(x)$ is a hardcore bit [BM82, Yao82, GM82] of a one way function $F\colon \{0,1\}^n \to \{0,1\}^\ell$ if for all efficient adversaries $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\left|\Pr\left[\mathcal{A}(F(x), \mathsf{hcb}(x)) = 1\right] - \Pr\left[\mathcal{A}(F(x), \mathsf{hcb}(x) \oplus 1) = 1\right]\right| = \mathsf{negl}(\lambda),$$

where $x \xleftarrow{R} \{0,1\}^n$.

Theorem 2.6 (Generic Hardcore Bits (informal) [GL89])**.** *There exists a randomized hardcore bit* $\mathsf{hcb}(x; r)$ *for any one way function* F*.*

Definition 2.7 (Positional Accumulator [KLW15])**.** Let λ be a security parameter. A positional accumulator with block length $\ell_{\mathsf{blk}} = \ell_{\mathsf{blk}}(\lambda)$, output length $\ell_{\mathsf{hash}} = \ell_{\mathsf{hash}}(\lambda)$, and opening length $\ell_{\mathsf{open}} = \ell_{\mathsf{open}}(\lambda)$ is a tuple of efficient algorithms $\Pi_{\mathsf{PA}} = (\mathsf{Setup}, \mathsf{SetupEnforce}, \mathsf{Hash}, \mathsf{Open}, \mathsf{Verify})$ with the following properties:

- $\mathsf{Setup}(1^\lambda, 1^{\ell_{\mathsf{blk}}}, 1^{|I|}) \to \mathsf{hk}$: On input the security parameter λ, the block size ℓ_{blk}, the enforcing set size $|I|$, the setup algorithm outputs a hash key hk. We let $\Sigma = \{0,1\}^{\ell_{\mathsf{blk}}}$ denote the block alphabet. We omit parameter enforcing set size $1^{|I|}$ when $|I| = 1$, i.e. when we are enforcing on a single index.
- $\mathsf{SetupEnforce}(1^\lambda, 1^{\ell_{\mathsf{blk}}}, \mathbf{x}, I^*) \to \mathsf{hk}$: On input security parameter λ, block size ℓ_{blk}, a tuple of inputs $\mathbf{x} \in \Sigma^N$, and a enforcing index set I^*, the enforcing setup algorithm outputs a hash key hk.
- $\mathsf{Hash}(\mathsf{hk}, \mathbf{x}) \to h$: On input the hash key hk and a message $\mathbf{x} \in \Sigma^N$, the hash algorithm *deterministically* outputs a hash $h \in \{0,1\}^{\ell_{\mathsf{hash}}}$.
- $\mathsf{Append}(\mathsf{hk}, h, x) \to h'$: On input hash key hk, hash h, and an input block $x \in \Sigma$, the hash algorithm *deterministically* outputs an updated hash $h' \in \{0,1\}^{\ell_{\mathsf{hash}}}$.
- $\mathsf{Open}(\mathsf{hk}, \mathbf{x}, i) \to \pi_i$: On input the hash key hk, an input $\mathbf{x} \in \Sigma^N$, and an index $i \in [N]$, the open algorithm outputs an opening $\pi_i \in \{0,1\}^{\ell_{\mathsf{open}}}$.
- $\mathsf{Verify}(\mathsf{hk}, h, i, x_i, \pi_i) \to \{0,1\}$: On input the hash key hk, a hash value $h \in \{0,1\}^{\ell_{\mathsf{hash}}}$, an index $i \in [N]$, a value $x_i \in \Sigma$, and an opening $\pi_i \in \{0,1\}^{\ell_{\mathsf{open}}}$, the verification algorithm outputs a bit $b \in \{0,1\}$ indicating whether it accepts or rejects.

We require the following properties:

- **Correctness:** For all security parameters $\lambda \in \mathbb{N}$, all block sizes $\ell_{\mathsf{blk}} = \ell_{\mathsf{blk}}(\lambda)$, all integers $N \le 2^\lambda$, all indices and enforcing set sizes $i, |I| \in [N]$, and any $\mathbf{x} \in \Sigma^N$,

$$\Pr\left[\mathsf{Verify}(\mathsf{hk}, h, i, x_i, \pi_i) = 1 : \begin{array}{c} \mathsf{hk} \leftarrow \mathsf{Setup}(1^\lambda, 1^{\ell_{\mathsf{blk}}}, 1^{|I|}); \\ h \leftarrow \mathsf{Hash}(\mathsf{hk}, \mathbf{x}); \pi_i \leftarrow \mathsf{Open}(\mathsf{hk}, \mathbf{x}, i) \end{array}\right] = 1.$$

- **Setup Indistinguishability:** For a bit $b \in \{0,1\}$ and an adversary $\mathcal{A}$, define the index hiding game $\mathsf{ExptIH}_{\mathcal{A}}(\lambda, b)$ as follows:

  1. Algorithm $\mathcal{A}(1^\lambda)$ chooses a block length 1^ℓ_{blk}, input sequence $\mathbf{x}$ an index set $I^* \subseteq [N]$.
  2. The challenger samples $\mathsf{hk}_0 \leftarrow \mathsf{Setup}(1^\lambda, 1^{\ell_{\mathsf{blk}}}, 1^{|I^*|})$ and $\mathsf{hk}_1 \leftarrow \mathsf{SetupEnforce}(1^\lambda, 1^\ell_{\mathsf{blk}}, \mathbf{x}, I^*)$ and gives hk_b to $\mathcal{A}$.
  3. Algorithm $\mathcal{A}$ outputs a bit $b' \in \{0,1\}$, which is also the output of the experiment.

  We require that for all polynomials $\ell_{\mathsf{blk}} = \ell_{\mathsf{blk}}(\lambda)$ and all efficient adversaries $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\left|\Pr[\mathsf{ExptSI}_{\mathcal{A}}(\lambda, 0) = 1] - \Pr[\mathsf{ExptSI}_{\mathcal{A}}(\lambda, 1) = 1]\right| = \mathsf{negl}(\lambda).$$

- **Read Enforcing:** We say that a hash key hk is read enforcing for an input $\mathbf{x} = (x_1, \ldots, x_N)$ and index set $I^* \in [N]$ if there does not exist $i^* \in I^*$ and $x' \neq x_{i^*} \in \Sigma$, and π' where $\mathsf{Verify}(\mathsf{hk}, h, i^*, x', \pi') = 1$ for $h \leftarrow \mathsf{Hash}(\mathsf{hk}, \mathbf{x})$. We require that for all polynomials $\ell_{\mathsf{blk}} = \ell_{\mathsf{blk}}(\lambda)$, there exists a negligible function $\mathsf{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$, all inputs $\mathbf{x} \in \Sigma^N$, and all $I^* \subseteq [N]$,

$$\Pr[\mathsf{hk} \text{ is read enforcing for } (\mathbf{x}, I^*) : \mathsf{hk} \leftarrow \mathsf{SetupEnforce}(1^\lambda, 1^{\ell_{\mathsf{blk}}}, \mathbf{x}, I^*)] = 1 - \mathsf{negl}(\lambda).$$

- **Write Enforcing:** For all security parameters $\lambda \in \mathbb{N}$, all block sizes $\ell_{\mathsf{blk}} = \ell_{\mathsf{blk}}(\lambda)$, all inputs $\mathbf{x} = (x_1, \ldots x_N) \in \Sigma^N$ and blocks x_{N+1}, we require that

$$\Pr\left[\mathsf{Hash}(\mathsf{hk}, (x_1, \ldots x_N, x_{N+1})) = \mathsf{Append}(\mathsf{hk}, h, x_{N+1}) : \begin{array}{c} \mathsf{hk} \leftarrow \mathsf{Setup}(1^\lambda, 1^{\ell_{\mathsf{blk}}}, 1^{|I^*|}); \\ h \leftarrow \mathsf{Hash}(\mathsf{hk}, \mathbf{x}) \end{array}\right] = 1.$$

- **Succinctness:** The hash length ℓ_{hash}, and opening length ℓ_{open} are all fixed polynomials in the security parameter λ and the block size ℓ_{blk} (and notably independent of the number of blocks hashed N).

Theorem 2.8 (Positional Accumulators [KLW15])**.** *. Assuming the existence of an indistinguishability obfuscation scheme and one-way functions, there exists a positional accumulator for arbitrary polynomial input lengths $\ell = \ell(\lambda)$.*

Remark 2.9. We present a simpler definition of positional accumulators modifying that given in [GSWW22] compared to the original definition of [KLW15], and is more reminiscent of the abstraction of somewhere statistically binding (SSB) hashes [HW15, OPWW15]. We do not require the full ability of arbitrary read/write sequences and instead only need the ability to 'append' to the end of a hash input.

Definition 2.10 (Puncturable PRF [BW13, KPTZ13, BGI14])**.** A t-puncturable pseudorandom function consists of a tuple of efficient algorithms $\Pi_{\mathsf{PPRF}} = (\mathsf{KeyGen}, \mathsf{Eval}, \mathsf{Puncture})$ with the following syntax:

- $\mathsf{KeyGen}(1^\lambda, 1^{\ell_{\mathsf{in}}}, 1^{\ell_{\mathsf{out}}}) \to K$: On input a security parameter λ, an input length ℓ_{in}, and an output length ℓ_{out}, the key generation algorithm outputs a key K. We assume the key contains an implicit description of ℓ_{in} and ℓ_{out}.
- $\mathsf{Puncture}(K, S) \to K^{(S)}$: On input a key K and a set $S \subseteq \{0,1\}^{\ell_{\mathsf{in}}}$ of size t, the puncture algorithm outputs a punctured key $K^{(S)}$. We assume the punctured key contains an implicit description of ℓ_{in} and ℓ_{out}.
- $\mathsf{Eval}(K, x) \to y$: On input a key K and an input $x \in \{0,1\}^{\ell_{\mathsf{in}}}$, the evaluation algorithm outputs a value $y \in \{0,1\}^{\ell_{\mathsf{out}}}$. We sometime write $F(K, x)$ to denote $\mathsf{Eval}(K, x)$.

In addition, Π_{PPRF} should satisfy the following properties:

- **Functionality-preserving:** For all $\lambda, \ell_{\mathsf{in}}, \ell_{\mathsf{out}} \in \mathbb{N}$, every set $S \subseteq \{0,1\}^{\ell_{\mathsf{in}}}$ of size t, and every $x \in \{0,1\}^{\ell_{\mathsf{in}}} \setminus S$,

$$\Pr\left[\mathsf{Eval}(K, x) = \mathsf{Eval}\left(K^{(S)}, x\right) : \begin{array}{c} K \leftarrow \mathsf{KeyGen}(1^\lambda, 1^{\ell_{\mathsf{in}}}, 1^{\ell_{\mathsf{out}}}) \\ K^{(S)} \leftarrow \mathsf{Puncture}(K, S) \end{array}\right] = 1$$

- **Punctured pseudorandomness:** For a bit $b \in \{0,1\}$ and a security parameter λ, we define the (selective) punctured pseudorandomness game between an adversary $\mathcal{A}$ and a challenger as follows:
  - On input a parameter λ, the adversary outputs an input length $1^{\ell_{\text{in}}}$ and an output length $1^{\ell_{\text{out}}}$, and commits to a set $S \subseteq \{0,1\}^{\ell_{\text{in}}}$ of size t.
  - The challenger samples a key $K \leftarrow \mathsf{KeyGen}(1^\lambda, 1^{\ell_{\text{in}}}, 1^{\ell_{\text{out}}})$ and gives the punctured key $K^{(S)} \leftarrow \mathsf{Puncture}(K, S)$ to $\mathcal{A}$.
  - If $b = 0$, then the challenger sets $y_i^* = \mathsf{Eval}(K, x_i^*)$ for each $x_i^* \in S$. Otherwise it samples $y_i^* \xleftarrow{R} \{0,1\}^{\ell_{\text{out}}}$ for each $i \in [t]$. In any case, the challenger gives $(y_i^*)_{i\in[t]}$ to $\mathcal{A}$.
  - The adversary outputs a guess $b' \in \{0,1\}$, which is the output of the experiment.

  We say that Π_{PPRF} satisfies punctured pseudorandomness if for all polynomial time adversaries $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that

$$\mathsf{PPRFAdv}_{\mathcal{A}}(\lambda) := \left|\Pr[\mathsf{ExptPP}_{\mathcal{A}}(\lambda, 0) = 1] - \Pr[\mathsf{ExptPP}_{\mathcal{A}}(\lambda, 1) = 1]\right| \leq \mathsf{negl}(\lambda)$$

Definition 2.11. Let $\Pi_{\mathsf{PPRF}} = (\mathsf{KeyGen}, \mathsf{Eval}, \mathsf{Puncture})$ be a puncturable PRF. For a bit $b \in \{0,1\}$ and a security parameter λ, we define the (selective) pseudorandomness game between an adversary $\mathcal{A}$ and a challenger as follows:

- On input a parameter λ, the adversary outputs an input length $1^{\ell_{\text{in}}}$ and an output length $1^{\ell_{\text{out}}}$, and commits to a point $x^* \in \{0,1\}^{\ell_{\text{in}}}$.
- The challenger samples a key $K \leftarrow \mathsf{KeyGen}(1^\lambda, 1^{\ell_{\text{in}}}, 1^{\ell_{\text{out}}})$. If $b = 0$, then the challenger sets $y^* = \mathsf{Eval}(K, x^*)$. Otherwise it sets $y^* \xleftarrow{R} \{0,1\}^{\ell_{\text{out}}}$. In any case, the challenger gives y^* to $\mathcal{A}$.
- The adversary $\mathcal{A}$ is allowed to make polynomially many queries $x \in \{0,1\}^{\ell_{\text{in}}}$ and the challenger responds with $\mathsf{Eval}(K, x)$.
- The adversary outputs a guess $b' \in \{0,1\}$, which is the output of the experiment.

We say that $\Pi_{\mathsf{PRF}} := (\mathsf{KeyGen}, \mathsf{Eval})$ satisfies pseudorandomness if for all polynomial time adversaries $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that

$$|\Pr[b' = 1 : b = 0] - \Pr[b' = 1 : b = 1]| \leq \mathsf{negl}(\lambda).$$

Fact 2.12. If $\Pi_{\mathsf{PPRF}} = (\mathsf{KeyGen}, \mathsf{Eval}, \mathsf{Puncture})$ is a puncturable PRF, then $\Pi_{\mathsf{PRF}} = (\mathsf{KeyGen}, \mathsf{Eval})$ satisfies pseudorandomness.

3 Circuit Evaluation Embedding

In this section, we define the new notion of circuit evaluation embeddings which we will use to build our secret sharing scheme. We can view this as a way to secretly embed the computation of a public circuit C and its evaluation on a hidden input x into a short digest α, which can be publicly expanded into a local encoding of the gates of C evaluated on x.

Definition 3.1 (Circuit Evaluation Embedding)**.** A circuit evaluation embedding is a tuple of efficient algorithms $\Pi_{\mathsf{CEE}} = (\mathsf{Gen}, \mathsf{EmbedGen}, \mathsf{Expand}, \mathsf{Extract})$ with the following syntax:

- $\mathsf{Gen}(1^\lambda, 1^n, C) \to \alpha$: On input a security parameter λ, an input length n and a description of a circuit $C : \{0,1\}^n \to \{0,1\}$, the generation algorithm outputs a seed α.
- $\mathsf{EmbedGen}(1^\lambda, 1^n, C, x) \to (\alpha^*, \mathsf{td})$: On input a security parameter λ, an input length n, a description of a circuit $C : \{0,1\}^n \to \{0,1\}$ and a string $x \in \{0,1\}^n$, the embedding generation algorithm outputs a seed α^* and a trapdoor td.

- $\mathsf{Expand}(C, \alpha) \to \mathbf{w}$: On input the description of a circuit $C : \{0,1\}^n \to \{0,1\}$ and a seed α, the expansion algorithm *deterministically* outputs a string $\mathbf{w} \in \{0,1\}^\ell$ of length ℓ, where ℓ is the number of gates in the circuit C.
- $\mathsf{Extract}(\mathsf{td}, i, b) \to 0/1$: On input a trapdoor td, an index $i \in [\ell]$ and a bit $b \in \{0,1\}$, the extraction algorithm outputs a bit.

We require Π_{CEE} to satisfy the following properties:

- **Computational mode indistinguishability:** We define the mode indistinguishability experiment between an attacker $\mathcal{A}$ and the challenger, parameterized by a bit $b \in \{0,1\}$, as follows:
  1. The attacker $\mathcal{A}$ on input 1^λ chooses a circuit $C : \{0,1\}^n \to \{0,1\}$ with ℓ wires and a string $x \in \{0,1\}^n$, and sends (C, x) to the challenger.
  2. If $b = 0$ then the challenger samples $\alpha \leftarrow \mathsf{Gen}(1^\lambda, 1^n)$. Otherwise, the challenger samples $(\alpha, \mathsf{td}) \leftarrow \mathsf{EmbedGen}(1^\lambda, 1^n, C, x)$. In any case, it sends α to $\mathcal{A}$.
  3. Algorithm $\mathcal{A}$ outputs a bit b' which is the output of the experiment.

  We say that Π_{CEE} satisfies computational mode indistinguishability if for any efficient $\mathcal{A}$ there exists a negligible function $\mathsf{negl}(\lambda)$ such that
  $$|\Pr[b' = 1 : b = 0] - \Pr[b' = 1 : b = 1]| \leq \mathsf{negl}(\lambda).$$
- **Statistical extraction:** for any $\lambda, n, \ell \in \mathbb{N}$, any circuit $C : \{0,1\}^n \to \{0,1\}$ of size ℓ, any $x \in \{0,1\}^n$ and any (α^*, td) in the support of $\mathsf{EmbedGen}(1^\lambda, 1^n, C, x)$, it holds that $\mathsf{Extract}(\mathsf{td}, i, w_i) = C(x)[i]$, where w_i is the i^{th} bit of $\mathbf{w} = \mathsf{Expand}(C, \alpha^*)$ and $C(x)[i]$ is the value of wire i when evaluation C on x.
- **Succinctness:** Let $m := m(\lambda, n, \ell)$ be any function. We say that Π_{CEE} is m-succinct if for any $\lambda, n, \ell \in \mathbb{N}$ and any circuit $C : \{0,1\}^n \to \{0,1\}$ with ℓ gates, the seed α output by algorithm $\mathsf{Gen}(1^\lambda, 1^n, C)$ is an m-bit string.
- **Succinct trapdoors:** We say that Π_{CEE} has succinct trapdoors if there exists a polynomial $\mathsf{poly}(\lambda)$ such that for any $\lambda, n, \ell \in \mathbb{N}$, any circuit $C : \{0,1\}^n \to \{0,1\}$ and any $x \in \{0,1\}^n$, the trapdoor $\mathsf{td}_{\mathsf{CEE}}$ output by algorithm $\mathsf{EmbedGen}(1^\lambda, 1^n, C, x)$ is $\mathsf{poly}(\lambda)$ bits long.

In Section 5, we present an instantiation of Π_{CEE} based on iO and injective OWFs, which achieves succinctness of $\mathsf{poly}(\lambda, \log \ell) + n$. But to get an intuition of Definition 3.1, we describe now a simple construction with a very bad succinctness property. This construction only relies on a PRF $\Pi_{\mathsf{PRF}} = (\mathsf{KeyGen}, \mathsf{Eval})$ and achieves ℓ-succinctness.

Construction 3.2. This construction only uses a PRF $\Pi_{\mathsf{PRF}} = (\mathsf{KeyGen}, \mathsf{Eval})$ and works as follows:

- $\mathsf{Gen}(1^\lambda, 1^n, C) \to \alpha$: On input a security parameter λ, an input length n and a description of a circuit $C : \{0,1\}^n \to \{0,1\}$, the generation algorithm outputs a uniform random string $\alpha \xleftarrow{R} \{0,1\}^\ell$.
- $\mathsf{EmbedGen}(1^\lambda, 1^n, C, x) \to (\alpha^*, \mathsf{td})$: On input a security parameter λ, an input length n, a description of a circuit $C : \{0,1\}^n \to \{0,1\}$ and a string $x \in \{0,1\}^n$, the embedding generation algorithm:
  1. Samples a PRF key $K \leftarrow \Pi_{\mathsf{PRF}}.\mathsf{KeyGen}(1^\lambda, 1^{\log \ell}, 1)$ with inputs of size $\log \ell$ and outputs of size 1.
  2. Computes all the gate values of C when evaluated on x. Denote the values by $\tilde{\mathbf{x}} = x_1 \cdots x_\ell$.
  3. Computes $\alpha^* = \tilde{\mathbf{x}} \oplus \mathsf{pad}$, where $\mathsf{pad} = \Pi_{\mathsf{PRF}}.\mathsf{Eval}(K, 1) \cdots \Pi_{\mathsf{PRF}}.\mathsf{Eval}(K, \ell)$.
  4. Outputs (α, K).
- $\mathsf{Expand}(C, \alpha) \to \mathbf{w}$: On input the description of a circuit $C : \{0,1\}^n \to \{0,1\}$ and a seed α, the expansion algorithm outputs a string $\mathbf{w} = \alpha$.
- $\mathsf{Extract}(K, i, b) \to 0/1$: On input a trapdoor K, an index $i \in [\ell]$ and a bit $b \in \{0,1\}$, the extraction algorithm outputs $b \oplus \Pi_{\mathsf{PRF}}.\mathsf{Eval}(K, i)$.

Computational mode indistinguishability follows from standard PRF security, while statistical extraction and ℓ-succinctness follows directly by construction.

4 Computational Secret Sharing From Circuit Evaluation Embeddings

In this section, we construct a computational secret sharing scheme for monotone boolean circuits based on iO, OWFs and a circuit evaluation embedding. Instantiating our circuit evaluation embedding scheme with the construction in Section 5 our scheme has public information size $n + \mathsf{poly}(\lambda, \log |C|)$ and share size λ.

Construction 4.1 (Computational Secret Sharing)**.** We construct a computational secret sharing scheme over secret space $\mathcal{S}$ and policy class of polynomial size monotone boolean circuits given the following primitives:

1. A puncturable PRF $\Pi_{\mathsf{PPRF}} = (\mathsf{KeyGen}, \mathsf{Eval}, \mathsf{Puncture})$.
2. A PRG PRG from $\{0,1\}^\lambda \to \{0,1\}^{2\lambda}$.
3. A positional accumulator $\Pi_{\mathsf{PA}} = (\mathsf{Setup}, \mathsf{SetupEnforce}, \mathsf{Hash}, \mathsf{Open}, \mathsf{Verify})$.
4. An indistinguishability obfuscator for circuits $i\mathcal{O}$.
5. A circuit evaluation embedding $\Pi_{\mathsf{CEE}} = (\mathsf{Gen}, \mathsf{EmbedGen}, \mathsf{Expand}, \mathsf{Extract})$ for the class of monotone boolean circuits.

Apart from circuit evaluation embeddings, which we will discuss in the following section, these primitives can all be build from indistinguishability obfuscation and one way functions.

At a high level, our construction assigns a secret key to each gate i in the policy circuit as the output of a PRF on index i, then gives each player the secret key of the corresponding input wire. To allow players to derive the secret keys of internal gates, the share generation algorithm produces an obfuscated program that outputs the secret key of gate i, if given sufficient secret keys of the children gates. To ensure security, our construction requires publishing a string $\mathbf{w}$ that (secretly) embeds the evaluation of the policy on the challenge input (i.e., the unauthorized set). Then, we gradually change our obfuscated program to output secret keys of gates only if they are permitted by the evaluation embedded in $\mathbf{w}$. We now describe our construction formally:

- $\mathsf{Share}(1^\lambda, C, s) \to (\mathsf{sh}_0, \mathsf{sh}_1, \ldots, \mathsf{sh}_n)$: On input security parameter λ, a monotone policy circuit $C : \{0,1\}^n \to \{0,1\}$ of size ℓ, and secret $s \in \mathcal{S}$. Denote by $\ell_{\mathsf{gate}} = 1 + 2\log \ell$ the size of the representation of a single gate C_i in C. Our sharing algorithm does the following:
  1. Samples the hash keys $\mathsf{hk}_{\mathbf{w}} \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Setup}(1^\lambda, 1)$ and $\mathsf{hk}_C \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Setup}(1^\lambda, 1^{\ell_{\mathsf{gate}}})$.
  2. Samples the PPRF key $K_\lambda \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{\log \ell}, 1^\lambda)$.
  3. Samples the circuit evaluation embedding seed $\alpha \leftarrow \Pi_{\mathsf{CEE}}.\mathsf{Gen}(1^\lambda, 1^n, C)$.
  4. Expands $\mathbf{w} = \Pi_{\mathsf{CEE}}.\mathsf{Expand}(C, \alpha)$.
  5. Hashes $h_{\mathbf{w}} = \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_{\mathbf{w}}, \mathbf{w})$ and $h_C = \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_C, C)$.
  6. Obfuscates the program $P \leftarrow i\mathcal{O}(\mathsf{ComputePolicy}[\mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda, s])$ from Fig. 1.
  7. Sets $\mathsf{sh}_0 = (P, \alpha, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}})$, and $\mathsf{sh}_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i)$ for each $i \in [n]$.
  8. Outputs $(\mathsf{sh}_0, \ldots, \mathsf{sh}_n)$.
- $\mathsf{Recon}(C, x, \mathsf{sh}_0, \{\mathsf{sh}_i\}_{i \in S_x}) \to s \cup \perp$: On input a monotone policy circuit $C : \{0,1\}^n \to \{0,1\}$ of size ℓ, input $x \in \{0,1\}^n$, public information sh_0, and set of user shares $\{\mathsf{sh}_i\}_{i \in S_x}$, our reconstruction algorithm does the following:
  1. Parses $\mathsf{sh}_0 = (P, \alpha, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}})$.
  2. Expands $\mathbf{w} = \Pi_{\mathsf{CEE}}.\mathsf{Expand}(C, \alpha)$.
  3. Computes openings for all indices on both hashes, that is for all $i \in [\ell]$:
     - $\pi_i^{(C)} \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Open}(\mathsf{hk}_C, C, i)$.
     - $\pi_i^{(\mathbf{w})} \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Open}(\mathsf{hk}_{\mathbf{w}}, \mathbf{w}, i)$.

4. Compute the following values for $i \in [n]$:

$$\sigma_i = \begin{cases} \mathsf{sh}_i & i \in S_x \\ \bot & i \notin S_x \end{cases}$$

5. For $i = n+1, \ldots, \ell$, computes $\sigma_i = P(i, C_i, \pi_i^{(C)}, w_i, \pi_i^{(\mathbf{w})}, \sigma_{i_0}, \sigma_{i_1})$, where $C_i = (g_i, i_0, i_1)$ is the description of gate i in the representation of C.
6. Output σ_ℓ.

Constants: Hash keys $\mathsf{hk}_{\mathbf{w}}, \mathsf{hk}_C$, hashes $h_{\mathbf{w}}, h_C$, puncturable PRF key K_λ, and the secret s.
Inputs: Gate index i, gate description $C_i = (g_i, i_0, i_1)$, local opening $\pi_i^{(C)}$, a bit w_i, local opening $\pi_i^{(\mathbf{w})}$ and for each $b \in \{0,1\}$, a signature σ_{i_b}.

1. Verify inputs are well formed. Abort with $\bot$ if any checks do not pass.
   (a) Check $i \in [n+1, \ell]$.
   (b) Check $\mathsf{Verify}(\mathsf{hk}_C, h_C, i, C_i, \pi_i^{(C)}) = 1$.
   (c) Check $\mathsf{Verify}(\mathsf{hk}_{\mathbf{w}}, h_{\mathbf{w}}, i, w_i, \pi^{(\mathbf{w})}) = 1$.
2. For $b \in \{0,1\}$, set $v_{i_b} = 1$ if and only if $\mathsf{PRG}(\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i_b)) = \mathsf{PRG}(\sigma_{i_b})$.
3. Compute $v_i = g_i(v_{i_0}, v_{i_1})$.
4. If $i < \ell$, set $\sigma_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i)$ if $v_i = 1$ and $\sigma_i = \bot$ otherwise. Output σ_i.
5. Otherwise, if $v_i = 1$, output the secret s. If $v_i = 0$, output $\bot$.

Figure 1: Program $\mathsf{ComputePolicy}[\mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda, s]$.

4.1 Correctness and Efficiency

Theorem 4.2 (Correctness)**.** *Suppose $i\mathcal{O}$ is a functionality preserving obfuscator and Π_{PA} is a correct positional accumulator. Then Construction 4.1 is a correct CSS scheme.*

Proof. Consider any $s \in \mathcal{S}$ and monotone Boolean circuit $C : \{0,1\}^n \to \{0,1\}$. Let $x \in \{0,1\}^n$ be an authorized set, i.e., $C(x) = 1$. Let $\mathsf{sh}_0 = (P, \alpha, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}})$ be some public information and $\mathsf{sh}_1, \ldots, \mathsf{sh}_n$ be some shares output by $\mathsf{Share}(1^\lambda, C, s)$.

Now consider the values $\sigma_1, \ldots, \sigma_\ell$ as defined in $\mathsf{Recon}(C, x, \mathsf{sh}_0, \{\mathsf{sh}_i\}_{i \in S_x})$. We prove that $\sigma_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i)$ for each $i \in [\ell - 1]$ such that $C(x)[i] = 1$ by induction on i. For the input gates $i \in [n]$, the claim follows directly by Step 4. Let $i \in [n+1, \ell-1]$ and assume the invariant holds for all $j < i$. By construction of the program in Fig. 1 and the functionality preserving of $i\mathcal{O}$, if we prove that all checks in Step 1 pass and that $v_i = 1$ in Step 3 then the program will output $\sigma_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i)$.

- Check 1a passes by the fact that $i \in [n+1, \ell-1]$.
- Checks 1b and 1c pass by the correctness of Π_{PA}.
- Let be an internal gate, $g_i \in \{\mathsf{AND}, \mathsf{OR}\}$ be the operation it computes and i_0, i_1 be its children gates. By the induction hypothesis and since $i_0, i_1 < i$, we have $\sigma_{i_b} = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i_b)$, therefore $v_{i_b} \geq C(x)[i_b]$ by the way we compute v_{i_b} in Step 2. Finally, if $C(x)[i] = 1$, then $g_i(C(x)[i_0], C(x)[i_1]) = 1$ and by the monotonicity of g_i, we have $v_i = g_i(v_{i_0}, v_{i_0}) = 1$.

Therefore the claim holds for all gates $i < \ell$. By an analogous argument, we prove that the program does not output $\bot$ when evaluated on the output gate ℓ. In that case, it will output $\sigma_\ell = s$ and thus the secret will be output by Recon and correctness follows. □

Theorem 4.3 (Succinctness). *If Π_{CEE} is m-succinct and has succinct trapdoors and Π_{PA} is succinct, Construction 4.1 has public information of size $m + \mathsf{poly}(\lambda, \log |C|)$ and shares of size λ.*

Proof. Consider any $s \in \mathcal{S}$ and monotone Boolean circuit $C : \{0,1\}^n \to \{0,1\}$ of size ℓ. Let $x \in \{0,1\}^n$ be an authorized set, i.e., $C(x) = 1$. Let $\mathsf{sh}_0 = (P, \alpha, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}})$ be some public information and $\mathsf{sh}_1, \ldots, \mathsf{sh}_n$ be some shares output by $\mathsf{Share}(1^\lambda, C, s)$. By construction, each share sh_i for $i < 0$ is exactly λ bits long since it is the output of a PRF on range $\{0,1\}^\lambda$. The public information consists of the following:

- The seed α is m bits long by the m-succinctness of Π_{CEE}.
- The hash keys hk_C and $\mathsf{hk}_{\mathbf{w}}$ are bounded by $\mathsf{poly}(\lambda, \ell_{\mathsf{gate}}) = \mathsf{poly}(\lambda, \log \ell)$ by the efficiency of $\Pi_{\mathsf{PA}}.\mathsf{Setup}$.
- The program P has the following hardcoded in it:
  - The hash keys hk_C and $\mathsf{hk}_{\mathbf{w}}$ of size $\mathsf{poly}(\lambda, \log \ell)$.
  - The hashes $h_C, h_{\mathbf{w}}$ of size $\mathsf{poly}(\lambda, \log \ell)$ by the succinctness of Π_{PA}.
  - The PPRF key K_λ of size $\mathsf{poly}(\lambda, \log \ell)$ by the efficiency of $\Pi_{\mathsf{PPRF}}.\mathsf{Eval}$.
  - The secret s which we assume is of size $\mathsf{poly}(\lambda)$.
  - Indices on number in $[\ell]$ which have size $\log \ell$.

  Therefore the program has size $\mathsf{poly}(\lambda, \log \ell)$. Furthermore, as we will see in Section 4.2, the programs in the hybrids never exceed that size $\mathsf{poly}(\lambda, \log \ell)$. Notably:
  - We hardcode a trapdoor $\mathsf{td}_{\mathsf{CEE}}$ output $\Pi_{\mathsf{CEE}}.\mathsf{EmbedGen}$ in Fig. 2 and the following programs, but this has size $\mathsf{poly}(\lambda)$ by the trapdoor succinctness of Π_{CEE}.
  - We hardcode two PRG outputs in Figs. 3 and 6, but those have size 2λ.

  In total, the programs we obfuscate have size $\mathsf{poly}(\lambda, \log \ell)$, and by the efficiency of $i\mathcal{O}$, the obfuscated programs would also have size $\mathsf{poly}(\lambda, \log \ell)$.

Therefore the public information has total size of $m + \mathsf{poly}(\lambda, \log \ell)$. □

4.2 Security

To show security of our CSS scheme, rather than attempting to use our obfuscated program to enforce some global validity property on the computation of the access policy, which is restricted by the small size of the obfuscated program, we instead leverage our circuit evaluation embedding and the monotonicity of our policy circuit to enforce a *local* property on the relationship between the circuit embedding $\mathbf{w}$ and the ability of our obfuscated program to produce a signature σ on a particular index. Using this, we are able to incrementally rule out the ability for our program to produce a signature σ on any wire for which $\mathbf{w}$ (when encoding the adversary's challenge input x) encodes a 0 bit. Once this reaches the output gate, since any valid adversary's challenge input x cannot satisfy the policy circuit, we can directly remove our program's ability to generate and verify σ_ℓ, allowing us to remove secret s as well.

Theorem 4.4 (Security). *If* PRG *is a secure PRG, Π_{PPRF} is a secure and correct puncturable PRF, Π_{PA} is a setup indistinguishable and read enforcing positional accumulator, Π_{CEE} is a mode indistinguishable and extractable circuit evaluation embedding, and $i\mathcal{O}$ is a secure indistinguishability obfuscator, then Construction 4.1 is a secure CSS scheme.*

Proof. To show security, we define a sequence of experiments, beginning with $\mathsf{Exp}^{(\beta)}_{\mathsf{real}}$, which is the real secret sharing $\mathsf{ExptCSS}_{\mathcal{A}}(\lambda, \beta)$ using secret s_β and ending in $\mathsf{Exp}^{(\beta)}_{\mathsf{final}}$, an experiment where the adversary's inputs are independent of secret s_β. We will argue indistinguishability of this sequence of experiments.

- $\mathsf{Exp}^{(\beta)}_{\mathsf{real}}$: This is the real CSS security when using our scheme Construction 4.1.
  1. The adversary $\mathcal{A}(1^\lambda)$ chooses a policy $C : \{0,1\}^n \to \{0,1\}$, an unauthorized input $x \in \{0,1\}^n$ where $C(x) = 0$ and a pair of secrets s_0, s_1. Let $\ell = |C|$ be the number of gates in C.

2. The challenger samples and outputs the following:
   (a) Samples the hash keys $\mathsf{hk}_{\mathbf{w}} \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Setup}(1^\lambda, 1)$ and $\mathsf{hk}_C \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Setup}(1^\lambda, 1^{\ell_{\mathsf{gate}}})$.
   (b) Samples the PPRF key $K_\lambda \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{\log \ell}, 1^\lambda)$.
   (c) Samples the circuit evaluation embedding seed $\alpha \leftarrow \Pi_{\mathsf{CEE}}.\mathsf{Gen}(1^\lambda, 1^n, C)$.
   (d) Expands $\mathbf{w} = \Pi_{\mathsf{CEE}}.\mathsf{Expand}(C, \alpha)$.
   (e) Hashes $h_{\mathbf{w}} = \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_{\mathbf{w}}, \mathbf{w})$ and $h_C = \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_C, C)$.
   (f) Obfuscates the program $P \leftarrow i\mathcal{O}(\mathsf{ComputePolicy}[\mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda, s_\beta])$ from Fig. 1.
   (g) Sets $\mathsf{sh}_0 = (P, \alpha, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}})$, and $\mathsf{sh}_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i)$ for each $i \in [n]$.
   (h) Gives $\mathsf{sh}_0, \{\mathsf{sh}\}_{i \in S_x}$ to $\mathcal{A}$.
3. The adversary $\mathcal{A}$ outputs a guess β'.
4. The output of the experiment is 1 if $\beta' = \beta$ and 0 otherwise.

The next step is replacing the string $\mathbf{w}$ with an embedding of the true gate values of the evaluation of the circuit $C(x)$, where x is the unauthorized input submitted by the adversary. We do this by sampling α using $\Pi_{\mathsf{CEE}}.\mathsf{EmbedGen}$ and embedding x, instead of $\Pi_{\mathsf{CEE}}.\mathsf{Gen}$. Formally, we define the following experiment:

- $\mathsf{Exp}^{(\beta)}_{\mathsf{embed}}$: same as $\mathsf{Exp}^{(\beta)}_{\mathsf{real}}$, but the challenger samples $\mathbf{w}$ as follows:
  (c) The challenger samples $(\alpha, \mathsf{td}_{\mathsf{CEE}}) \leftarrow \Pi_{\mathsf{CEE}}.\mathsf{EmbedGen}(1^\lambda, 1^n, C, x)$, for the unauthorized input $x \in \{0,1\}^n$ submitted by the adversary in the first Step 1.

The output of this experiment is negligibly close to the previous one by the computational mode indistinguishability of the circuit evaluation embedding as per Definition 3.1.

We next define hybrid experiments for $i^* \in [n+1, \ell+1]$ as follows:

- $\mathsf{Exp}^{(\beta)}_{i^*}$: Same as $\mathsf{Exp}^{(\beta)}_{\mathsf{embed}}$ with the following changes:
  (f) Obfuscates the program $P \leftarrow i\mathcal{O}(\mathsf{ComputePolicy}'[i^*, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda, \mathsf{td}_{\mathsf{CEE}}, s_\beta])$ from Fig. 2.

Constants: Hash keys $\mathsf{hk}_{\mathbf{w}}, \mathsf{hk}_C$, hashes $h_{\mathbf{w}}, h_C$, puncturable PRF key K_λ and CEE trapdoor $\mathsf{td}_{\mathsf{CEE}}$, the secret s, and index i^*.
Inputs: Gate index i, gate description $C_i = (g_i, i_0, i_1)$, local opening $\pi_i^{(C)}$, a bit w_i, local opening $\pi_i^{(\mathbf{w})}$ and for each $b \in \{0,1\}$, a signature σ_{i_b}.

1. Verify inputs are well formed. Abort with $\perp$ if any checks do not pass.
   (a) Check $i \in [n+1, \ell]$.
   (b) Check $\mathsf{Verify}(\mathsf{hk}_C, h_C, i, C_i, \pi_i^{(C)}) = 1$.
   (c) Check $\mathsf{Verify}(\mathsf{hk}_{\mathbf{w}}, h_{\mathbf{w}}, i, w_i, \pi^{(\mathbf{w})}) = 1$.
2. For $b \in \{0,1\}$, set $v_{i_b} = 1$ if and only if $\mathsf{PRG}(\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i_b)) = \mathsf{PRG}(\sigma_{i_b})$.
3. Compute $v_i = g_i(v_{i_0}, v_{i_1})$.
4. If $i < i^*$ and $\Pi_{\mathsf{CEE}}.\mathsf{Extract}(\mathsf{td}_{\mathsf{CEE}}, i, w_i) = 0$ then check that $v_i = 0$. If the check fails, abort with $\perp$.
5. If $i < \ell$, set $\sigma_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i)$ if $v_i = 1$ and $\sigma_i = \perp$ otherwise. Output σ_i.
6. Otherwise, if $v_i = 1$, output the secret s. If $v_i = 0$, output $\perp$.

Figure 2: Program $\mathsf{ComputePolicy}'[i^*, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda, \mathsf{td}_{\mathsf{CEE}}, s]$.

Next, we define inner hybrids between $\mathsf{Exp}^{(\beta)}_{i^*}$ and $\mathsf{Exp}^{(\beta)}_{i^*+1}$. We give a quick explanation why each hybrid is (computationally) close to the previous one here, but give formal proofs afterwards.

- $\mathsf{sExp}^{(\beta, i^*)}_0$ is the same as Exp_{i^*}.

- $\mathsf{sExp}_1^{(\beta,i^*)}$: same as $\mathsf{sExp}_0^{(\beta,i^*)}$, but the challenger samples the hash key hk_C to be enforcing on i^*, i.e.,

  (a) Samples the hash keys $\mathsf{hk}_{\mathbf{w}} \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Setup}(1^\lambda, 1)$ and $\mathsf{hk}_C \leftarrow \Pi_{\mathsf{PA}}.\mathsf{SetupEnforce}(1^\lambda, 1^{\ell_{\mathsf{gate}}}, C, i^*)$.

  The output of this experiment is negligibly close to the previous one by the setup indistinguishability of the positional accumulator as per Definition 2.7.

- $\mathsf{sExp}_2^{(\beta,i^*)}$: same as $\mathsf{sExp}_1^{(\beta,i^*)}$, but the challenger samples the hash key $\mathsf{hk}_{\mathbf{w}}$ to be enforcing on i^* i.e.,

  (a) Samples the hash keys $\mathsf{hk}_{\mathbf{w}} \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Setup}(1^\lambda, 1^{\ell_{\mathsf{gate}}}, \mathbf{w}, i^*)$ and $\mathsf{hk}_C \leftarrow \Pi_{\mathsf{PA}}.\mathsf{SetupEnforce}(1^\lambda, 1^{\ell_{\mathsf{gate}}}, C, i^*)$.

  The output of this experiment is negligibly close to the previous one by the setup indistinguishability of the positional accumulator as per Definition 2.7.

- $\mathsf{sExp}_3^{(\beta,i^*)}$: same as before, but the challenger samples the obfuscated program P as follows:

  (f) For each $b \in \{0,1\}$, sets $z_b = \mathsf{PRG}(\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i_b^*))$. Then obfuscates the program from Fig. 3

  $$P \leftarrow i\mathcal{O}(\mathsf{ComputePolicy}^1[i^*, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda, \mathsf{td}_{\mathsf{CEE}}, x_{i_0^*}, x_{i_1^*}, z_0, z_1, s_\beta]).$$

  The two programs are functionally equivalent, so we can appeal to the iO security as per Definition 2.3.

Constants: Hash keys $\mathsf{hk}_{\mathbf{w}}, \mathsf{hk}_C$, hashes $h_{\mathbf{w}}, h_C$, puncturable PRF key K_λ and CEE trapdoor $\mathsf{td}_{\mathsf{CEE}}$, and the secret s, index i^*, bits x_0 and x_1 and strings z_0 and z_1.
Inputs: Gate index i, gate description $C_i = (g_i, i_0, i_1)$, local opening $\pi_i^{(C)}$, a bit w_i, local opening $\pi_i^{(\mathbf{w})}$ and for each $b \in \{0,1\}$, a signature σ_{i_b}.

1. Verify inputs are well formed. Abort with $\perp$ if any checks do not pass.
   (a) Check $i \in [n+1, \ell]$.
   (b) Check $\mathsf{Verify}(\mathsf{hk}_C, h_C, i, C_i, \pi_i^{(C)}) = 1$.
   (c) Check $\mathsf{Verify}(\mathsf{hk}_{\mathbf{w}}, h_{\mathbf{w}}, i, w_i, \pi^{(\mathbf{w})}) = 1$.
2. If $i = i^*$ and $x_b = 0$, set $v_{i_b^*} = 1$ if and only if $\mathsf{PRG}(\sigma_{i_b^*}) = z_b$. In all other cases, compute as before: For $b \in \{0,1\}$, set $v_{i_b} = 1$ if and only if $\mathsf{PRG}(\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i_b)) = \mathsf{PRG}(\sigma_{i_b})$.
3. Compute $v_i = g_i(v_{i_0}, v_{i_1})$.
4. If $i < i^*$ and $\Pi_{\mathsf{CEE}}.\mathsf{Extract}(\mathsf{td}_{\mathsf{CEE}}, i, w_i) = 0$ then check that $v_i = 0$. If the check fails, abort with $\perp$.
5. If $i < \ell$, set $\sigma_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i)$ if $v_i = 1$ and $\sigma_i = \perp$ otherwise. Output σ_i.
6. Otherwise, if $v_i = 1$, output the secret s. If $v_i = 0$, output $\perp$.

Figure 3: Program $\mathsf{ComputePolicy}^1[i^*, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda, \mathsf{td}_{\mathsf{CEE}}, x_0, x_1, z_0, z_1, s]$.

- $\mathsf{sExp}_4^{(\beta,i^*)}$: same as before, except the challenger uses a punctured key $K_\lambda^{(S)}$ on the set $S = \{i_b^* : x_{i_b^*} = 0\}$, that is:

  (b) Samples the PPRF key $K_\lambda \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, \log \ell, \lambda)$ and punctures $K_\lambda^{(S)} \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Puncture}(K_\lambda, S)$.

  (f) For each $b \in \{0,1\}$, sets $z_b = \mathsf{PRG}(\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i_b^*))$. Then obfuscates the program from Fig. 3

  $$P \leftarrow i\mathcal{O}(\mathsf{ComputePolicy}^1[i^*, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda^{(S)}, \mathsf{td}_{\mathsf{CEE}}, x_{i_0^*}, x_{i_1^*}, z_0, z_1, s_\beta]).$$

  The program never uses K_λ on the points in S anyway, since $i^* < i^*$ and therefore if $x_{i_b^*} = 0$ then it forces $v_{i_b^*} = 0$ in line 4. Therefore the two programs are functionally equivalent and we can appeal to iO security as per Definition 2.3.

- $\mathsf{sExp}_5^{(\beta,i^*)}$: same as before, but the challenger samples z_b as follows

  (f) For each $b \in \{0,1\}$, samples a uniform $r_b \xleftarrow{R} \{0,1\}^\lambda$ and sets

$$z_b = \begin{cases} \mathsf{PRG}(r_b) & i_b^* \in S \\ \mathsf{PRG}(\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i_b^*)) & i_b^* \notin S \end{cases}$$

  Then obfuscates the program from Fig. 3

$$P \leftarrow i\mathcal{O}(\mathsf{ComputePolicy}^1[i^*, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda^{(S)}, \mathsf{td}_{\mathsf{CEE}}, x_{i_0^*}, x_{i_1^*}, z_0, z_1, s_\beta]).$$

  The output of this experiment is negligibly close to the previous one by standard puncturable PRF security as per Definition 2.10.

- $\mathsf{sExp}_6^{(\beta,i^*)}$: same as before, but the challenger samples z_b as follows

  (f) For each $b \in \{0,1\}$, samples a uniform $r_b \xleftarrow{R} \{0,1\}^{2\lambda}$ and sets

$$z_b = \begin{cases} r_b & i_b^* \in S \\ \mathsf{PRG}(\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i_b^*)) & i_b^* \notin S \end{cases}$$

  Then obfuscates the program from Fig. 3

$$P \leftarrow i\mathcal{O}(\mathsf{ComputePolicy}^1[i^*, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda^{(S)}, \mathsf{td}_{\mathsf{CEE}}, x_{i_0^*}, x_{i_1^*}, z_0, z_1, s_\beta]).$$

  The output of this experiment is negligibly close to the previous one by standard PRG security as per Definition 2.4.

- $\mathsf{sExp}_7^{(\beta,i^*)}$: same as before, but the challenger samples the obfuscated program P as follows:

  (f) Obfuscate the program from Fig. 4:

$$P \leftarrow i\mathcal{O}(\mathsf{ComputePolicy}^2[i^*, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda^{(S)}, \mathsf{td}_{\mathsf{CEE}}, x_{i_0^*}, x_{i_1^*}, s_\beta]).$$

  Note that we no longer use the strings z_0, z_1.

  With probability $1 - 2^{-\lambda}$, a uniform z_b is outside of the image of the PRG. Conditioned on that event, the condition $\mathsf{PRG}(\sigma_{i_b^*}) = z_b$ does not hold and the two programs are functionally equivalent, so we can appeal to iO security as per Definition 2.3 to argue that the output of this experiment is negligibly close to the previous one.

Constants: Hash keys $\mathsf{hk}_{\mathbf{w}}, \mathsf{hk}_C$, hashes $h_{\mathbf{w}}, h_C$, puncturable PRF key K_λ and CEE trapdoor $\mathsf{td}_{\mathsf{CEE}}$, and the secret s, index i^* and bits x_0 and x_1.
Inputs: Gate index i, gate description $C_i = (g_i, i_0, i_1)$, local opening $\pi_i^{(C)}$, a bit w_i, local opening $\pi_i^{(\mathbf{w})}$ and for each $b \in \{0,1\}$, a signature σ_{i_b}.

1. Verify inputs are well formed. Abort with $\perp$ if any checks do not pass.
   (a) Check $i \in [n+1, \ell]$.
   (b) Check $\mathsf{Verify}(\mathsf{hk}_C, h_C, i, C_i, \pi_i^{(C)}) = 1$.
   (c) Check $\mathsf{Verify}(\mathsf{hk}_{\mathbf{w}}, h_{\mathbf{w}}, i, w_i, \pi^{(\mathbf{w})}) = 1$.
2. If $i = i^*$ and $x_{i_b^*} = 0$, set $v_{i_b^*} = 0$. In all other cases, compute as before:
   For $b \in \{0,1\}$, set $v_{i_b} = 1$ if and only if $\mathsf{PRG}(\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i_b)) = \mathsf{PRG}(\sigma_{i_b})$.
3. Compute $v_i = g_i(v_{i_0}, v_{i_1})$.
4. If $i < i^*$ and $\Pi_{\mathsf{CEE}}.\mathsf{Extract}(\mathsf{td}_{\mathsf{CEE}}, i, w_i) = 0$ then check that $v_i = 0$. If the check fails, abort with $\perp$.
5. If $i < \ell$, set $\sigma_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i)$ if $v_i = 1$ and $\sigma_i = \perp$ otherwise. Output σ_i.
6. Otherwise, if $v_i = 1$, output the secret s. If $v_i = 0$, output $\perp$.

Figure 4: Program $\mathsf{ComputePolicy}^2[i^*, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda, \mathsf{td}_{\mathsf{CEE}}, x_0, x_1, s]$.

- $\mathsf{sExp}_8^{(\beta, i^*)}$: same as before, except the challenger extends the check in line 4 to index i^*. That is, the challenger samples the obfuscated program as follows:

  (f) Obfuscate the program from Fig. 5:

  $$P \leftarrow i\mathcal{O}(\mathsf{ComputePolicy}^3[i^*, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda^{(S)}, \mathsf{td}_{\mathsf{CEE}}, x_{i_0^*}, x_{i_1^*}, s_\beta]).$$

  We argue that the two programs are functionally equivalent and use that fact to appeal to iO security as per Definition 2.3 and conclude that this hybrid is negligibly close to the previous one. See Lemma 4.15.

Constants: Hash keys $\mathsf{hk}_{\mathbf{w}}, \mathsf{hk}_C$, hashes $h_{\mathbf{w}}, h_C$, puncturable PRF key K_λ and CEE trapdoor $\mathsf{td}_{\mathsf{CEE}}$, and the secret s, index i^* and bits x_0 and x_1.
Inputs: Gate index i, gate description $C_i = (g_i, i_0, i_1)$, local opening $\pi_i^{(C)}$, a bit w_i, local opening $\pi_i^{(\mathbf{w})}$ and for each $b \in \{0,1\}$, a signature σ_{i_b}.

1. Verify inputs are well formed. Abort with $\perp$ if any checks do not pass.
   (a) Check $i \in [n+1, \ell]$.
   (b) Check $\mathsf{Verify}(\mathsf{hk}_C, h_C, i, C_i, \pi_i^{(C)}) = 1$.
   (c) Check $\mathsf{Verify}(\mathsf{hk}_{\mathbf{w}}, h_{\mathbf{w}}, i, w_i, \pi^{(\mathbf{w})}) = 1$.
2. If $i = i^*$ and $x_0 = 0$, set $v_{i_b^*} = 0$. In all other cases, compute as before:
   For $b \in \{0,1\}$, set $v_{i_b} = 1$ if and only if $\mathsf{PRG}(\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i_b)) = \mathsf{PRG}(\sigma_{i_b})$.
3. Compute $v_i = g_i(v_{i_0}, v_{i_1})$.
4. If $i < i^* + 1$ and $\Pi_{\mathsf{CEE}}.\mathsf{Extract}(\mathsf{td}_{\mathsf{CEE}}, i, w_i) = 0$ then check that $v_i = 0$. If the check fails, abort with $\perp$.
5. If $i < \ell$, set $\sigma_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i)$ if $v_i = 1$ and $\sigma_i = \perp$ otherwise. Output σ_i.
6. Otherwise, if $v_i = 1$, output the secret s. If $v_i = 0$, output $\perp$.

Figure 5: Program $\mathsf{ComputePolicy}^3[i^*, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda, \mathsf{td}_{\mathsf{CEE}}, x_0, x_1, s]$.

- $\mathsf{sExp}_9^{(\beta, i^*)}$: Same as before, but the challenger reintroduces the uniform z_b and the check it removed in $\mathsf{sExp}_7^{(\beta, i^*)}$. That is, it samples the obfuscated program as follows:

(f) For each $b \in \{0,1\}$, samples a uniform $r_b \xleftarrow{R} \{0,1\}^{2\lambda}$ and sets

$$z_b = \begin{cases} r_b & i_b^* \in S \\ \mathsf{PRG}(\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i_b^*)) & i_b^* \notin S \end{cases}$$

Then obfuscates the program from Fig. 6

$$P \leftarrow i\mathcal{O}(\mathsf{ComputePolicy}^4[i^*, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda^{(S)}, x_{i_0^*}, x_{i_1^*}, z_0, z_1, s_\beta])$$

The output of this experiment is negligibly close to the previous one, by a similar argument to that of Lemma 4.14

Constants: Hash keys $\mathsf{hk}_{\mathbf{w}}, \mathsf{hk}_C$, hashes $h_{\mathbf{w}}, h_C$, puncturable PRF key K_λ and CEE trapdoor $\mathsf{td}_{\mathsf{CEE}}$, and the secret s, index i^*, bits x_0 and x_1 and strings z_0 and z_1.

Inputs: Gate index i, gate description $C_i = (g_i, i_0, i_1)$, local opening $\pi_i^{(C)}$, a bit w_i, local opening $\pi_i^{(\mathbf{w})}$ and for each $b \in \{0,1\}$, a signature σ_{i_b}.

1. Verify inputs are well formed. Abort with $\perp$ if any checks do not pass.
   (a) Check $i \in [n+1, \ell]$.
   (b) Check $\mathsf{Verify}(\mathsf{hk}_C, h_C, i, C_i, \pi_i^{(C)}) = 1$.
   (c) Check $\mathsf{Verify}(\mathsf{hk}_{\mathbf{w}}, h_{\mathbf{w}}, i, w_i, \pi^{(\mathbf{w})}) = 1$.
2. If $i = i^*$ and $x_b = 0$, set $v_{i_b^*} = 1$ if and only $\mathsf{PRG}(\sigma_{i_b^*}) = z_b$. In all other cases, compute as before: For $b \in \{0,1\}$, set $v_{i_b} = 1$ if and only if $\mathsf{PRG}(\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i_b)) = \mathsf{PRG}(\sigma_{i_b})$.
3. Compute $v_i = g_i(v_{i_0}, v_{i_1})$.
4. If $i < i^* + 1$ and $\Pi_{\mathsf{CEE}}.\mathsf{Extract}(\mathsf{td}_{\mathsf{CEE}}, i, w_i) = 0$ then check that $v_i = 0$. If the check fails, abort with $\perp$.
5. If $i < \ell$, set $\sigma_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i)$ if $v_i = 1$ and $\sigma_i = \perp$ otherwise. Output σ_i.
6. Otherwise, if $v_i = 1$, output the secret s. If $v_i = 0$, output $\perp$.

Figure 6: Program $\mathsf{ComputePolicy}^4[i^*, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda, \mathsf{td}_{\mathsf{CEE}}, x_0, x_1, z_0, z_1, s]$.

- $\mathsf{sExp}_{10}^{(\beta,i^*)}$: same as before, but the challenger sets $z_b = \mathsf{PRG}(r_b)$ for a uniform $r_b \xleftarrow{R} \{0,1\}^\lambda$.

  (f) For each $b \in \{0,1\}$, samples a uniform $r_b \xleftarrow{R} \{0,1\}^\lambda$ and sets

  $$z_b = \begin{cases} \mathsf{PRG}(r_b) & i_b^* \in S \\ \mathsf{PRG}(\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i_b^*)) & i_b^* \notin S \end{cases}$$

  Then obfuscates the program from Fig. 6

  $$P \leftarrow i\mathcal{O}(\mathsf{ComputePolicy}^4[i^*, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda^{(S)}, x_{i_0^*}, x_{i_1^*}, z_0, z_1, s_\beta])$$

  The output of this experiment is negligibly close to the previous one by standard PRG security as per Definition 2.4.

- $\mathsf{sExp}_{11}^{(\beta,i^*)}$: same as before, but set $z_b = \mathsf{PRG}(\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i_b^*))$.

  (f) For each $b \in \{0,1\}$, sets $z_b = \mathsf{PRG}(\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i_b^*))$. Then obfuscates the program from Fig. 6

  $$P \leftarrow i\mathcal{O}(\mathsf{ComputePolicy}^4[i^*, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda^{(S)}, x_{i_0^*}, x_{i_1^*}, z_0, z_1, s_\beta])$$

  The output of this experiment is negligibly close to the previous one by standard puncturable PRF security as per Definition 2.10.

- $\mathsf{sExp}_{12}^{(\beta,i^*)}$: same as before, except we go back to using an unpunctured key K_λ.

  (b) Samples the PPRF key $K_\lambda \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, \log \ell, \lambda)$ in particular, no longer punctures K_λ..

  (f) For each $b \in \{0,1\}$, sets $z_b = \mathsf{PRG}(\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i_b^*))$. Then obfuscates the program from Fig. 6

$$P \leftarrow i\mathcal{O}(\mathsf{ComputePolicy}^4[i^*, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda, x_{i_0^*}, x_{i_1^*}, z_0, z_1, s_\beta])$$

  The two programs are functionally equivalent by the functionality preserving property of Definition 2.10, so we appeal to iO security as per Definition 2.3 to prove that the two hybrids are negligibly close.

- $\mathsf{sExp}_{13}^{(\beta,i^*)}$: same as before, but we switch to $\mathsf{ComputePolicy}'[i^*+1, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda, s_\beta]$.

  (f) Obfuscates the program from Fig. 2

$$P \leftarrow i\mathcal{O}(\mathsf{ComputePolicy}'[i^*+1, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda, s_\beta])$$

  Note that we no longer use the strings z_0, z_1.

  The two programs are functionally equivalent, so we appeal to iO security as per Definition 2.3 to prove that the two hybrids are negligibly close.

- $\mathsf{sExp}_{14}^{(\beta,i^*)}$: same as before, but the challenger samples the hash key hk_C in the normal mode (as opposed to enforcing mode). That is:

  (a) Samples the hash keys $\mathsf{hk}_{\mathbf{w}} \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Setup}(1^\lambda, 1^{\ell_{\mathsf{gate}}}, \mathbf{w}, i^*)$ and $\mathsf{hk}_C \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Setup}(1^\lambda, 1^{\ell_{\mathsf{gate}}})$.

  The output of this experiment is negligibly close to the previous one, by a similar argument to that of Lemma 4.9

- $\mathsf{sExp}_{15}^{(\beta,i^*)}$: same as before, but the challenger samples the hash key $\mathsf{hk}_{\mathbf{w}}$ in the normal mode (as opposed to enforcing mode). That is:

  (a) Samples the hash keys $\mathsf{hk}_{\mathbf{w}} \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Setup}(1^\lambda, 1)$ and $\mathsf{hk}_C \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Setup}(1^\lambda, 1^{\ell_{\mathsf{gate}}})$.

  This is identical to $\mathsf{Exp}_{i^*+1}^{(\beta)}$. The output of this experiment is negligibly close to the previous one, by a similar argument to that of Lemma 4.8

Finally, we define an experiment in which the obfuscated program does not contain the secret s_b at all.

- $\mathsf{Exp}_{\mathsf{final}}^{(\beta)}$: same as $\mathsf{sExp}_{14}^{(\beta,\ell)}$ (where ℓ is the length of the policy circuit submitted by the adversary)[3] except the challenger samples the obfuscated program as follows

  (f) Obfuscates the program from Fig. 7

$$P \leftarrow i\mathcal{O}(\mathsf{ComputePolicy}_{\mathsf{final}}[i^*, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda, \mathsf{td}_{\mathsf{CEE}}])$$

  Note that $\mathsf{hk}_{\mathbf{w}}$ is statistically enforcing on index ℓ, just as it is in Exp_ℓ^{14}.

[3] We can think of this as $\mathsf{Exp}_{\ell+1}$. However, looking ahead, we need to make a functional equivalence argument which would crucially rely on using the correct value w_ℓ in line 4, but this requires $\mathsf{hk}_{\mathbf{w}}$ to be statistically enforcing.

Constants: Hash keys $\mathsf{hk}_{\mathbf{w}}, \mathsf{hk}_C$, hashes $h_{\mathbf{w}}, h_C$, puncturable PRF key K_λ and CEE trapdoor $\mathsf{td}_{\mathsf{CEE}}$.
Inputs: Gate index i, gate description $C_i = (g_i, i_0, i_1)$, local opening $\pi_i^{(C)}$, a bit w_i, local opening $\pi_i^{(\mathbf{w})}$ and for each $b \in \{0,1\}$, a signature σ_{i_b}.

1. Verify inputs are well formed. Abort with $\perp$ if any checks do not pass.
   (a) Check $i \in [n+1, \ell]$.
   (b) Check $\mathsf{Verify}(\mathsf{hk}_C, h_C, i, C_i, \pi_i^{(C)}) = 1$.
   (c) Check $\mathsf{Verify}(\mathsf{hk}_{\mathbf{w}}, h_{\mathbf{w}}, i, w_i, \pi^{(\mathbf{w})}) = 1$.
2. For $b \in \{0,1\}$, set $v_{i_b} = 1$ if and only if $\mathsf{PRG}(\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i_b)) = \mathsf{PRG}(\sigma_{i_b})$.
3. Compute $v_i = g_i(v_{i_0}, v_{i_1})$.
4. If $\Pi_{\mathsf{CEE}}.\mathsf{Extract}(\mathsf{td}_{\mathsf{CEE}}, i, w_i) = 0$ then check that $v_i = 0$. If the check fails, abort with $\perp$.
5. If $i < \ell$, set $\sigma_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i)$ if $v_i = 1$ and $\sigma_i = \perp$ otherwise. Output σ_i.
6. If $i = \ell$, output $\perp$.

Figure 7: Program $\mathsf{ComputePolicy}_{\mathsf{final}}[\mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda, \mathsf{td}_{\mathsf{CEE}}]$.

The program from Fig. 7 is identical to that from Fig. 2 with $i^* = \ell + 1$ and a statistically enforcing hash key $\mathsf{hk}_{\mathbf{w}}$ on index ℓ. This is because the check on line 4 guarantees that the program always outputs $\perp$ on index ℓ, as we elaborate in the proof of Lemma 4.24. This allows us to switch to this final experiment using iO security. Finally, no adversary can guess the bit β with probability more than $1/2$ since the view does not depend on the secret s_β.

Lemma 4.5. *If Π_{CEE} satisfies computational mode indistinguishability, then for any efficient algorithm $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that*

$$\left|\Pr[\mathsf{Exp}^{(\beta)}_{\mathsf{real}}(\mathcal{A})] - \Pr[\mathsf{Exp}^{(\beta)}_{\mathsf{embed}}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

Proof. Denote $\varepsilon := \left|\Pr[\mathsf{Exp}^{(\beta)}_{\mathsf{real}}(\mathcal{A})] - \Pr[\mathsf{Exp}^{(\beta)}_{\mathsf{embed}}(\mathcal{A})]\right|$. We construct an attacker $\mathcal{B}$ for the mode indistinguishability game Π_{CEE}, parameterized by a bit $b \in \{0,1\}$ as follows

1. Algorithm $\mathcal{B}$ runs $\mathcal{A}$ to get a circuit $C : \{0,1\}^n \to \{0,1\}$, an unauthorized input $x \in \{0,1\}^n$ and a pair of secrets s_0, s_1.
2. Algorithm $\mathcal{B}$ chooses the circuit C and the string x and sends them to the challenger.
3. The challenger samples $\alpha_0 \leftarrow \Pi_{\mathsf{CEE}}.\mathsf{Gen}(1^\lambda, 1^n, C)$ and $(\alpha_1, \mathsf{td}_{\mathsf{CEE}}) \leftarrow \Pi_{\mathsf{CEE}}.\mathsf{EmbedGen}(1^\lambda, 1^n, C, x)$ and gives α_b to $\mathcal{B}$.
4. Algorithm $\mathcal{B}$ samples the shares as follows:
   (a) Samples the hash keys $\mathsf{hk}_{\mathbf{w}} \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Setup}(1^\lambda, 1)$ and $\mathsf{hk}_C \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Setup}(1^\lambda, 1^{\ell_{\mathsf{gate}}})$.
   (b) Samples the PPRF key $K_\lambda \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{\log \ell}, 1^\lambda)$.
   (c) Sets $\alpha = \alpha_b$.
   (d) Expands $\mathbf{w} = \Pi_{\mathsf{CEE}}.\mathsf{Expand}(C, \alpha)$.
   (e) Hashes $h_{\mathbf{w}} = \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_{\mathbf{w}}, \mathbf{w})$ and $h_C = \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_C, C)$.
   (f) Obfuscates the program $P \leftarrow i\mathcal{O}(\mathsf{ComputePolicy}[\mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda, s_\beta])$ from Fig. 1.
   (g) Sets $\mathsf{sh}_0 = (P, \alpha, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}})$, and $\mathsf{sh}_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i)$ for each $i \in [n]$.
   (h) Gives $\mathsf{sh}_0, \{\mathsf{sh}\}_{i \in S_x}$ to $\mathcal{A}$.
5. Algorithm $\mathcal{A}$ outputs a bit β', which $\mathcal{B}$ outputs as well.

If $b = 0$, then $\mathcal{B}$ simulates exactly $\mathsf{Exp}^{(\beta)}_{\mathsf{real}}(\mathcal{A})$, whereas if $b = 1$ then $\mathcal{B}$ simulates exactly $\mathsf{Exp}^{(\beta)}_{\mathsf{embed}}(\mathcal{A})$. Therefore the advantage of $\mathcal{B}$ in the mode indistinguishability game is exactly ε and therefore ε is negligible by the mode indistinguishability of Π_{CEE} and the claim follows. □

Lemma 4.6. *If $i\mathcal{O}$ is a secure indistinguishability obfuscator, then for any efficient algorithm $\mathcal{A}$, there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that*

$$\left|\Pr[\mathsf{Exp}^{(\beta)}_{n+1}(\mathcal{A})] - \Pr[\mathsf{Exp}^{(\beta)}_{\mathsf{embed}}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

Proof. Denote $\varepsilon := \left|\Pr[\mathsf{Exp}^{(\beta)}_{n+1}(\mathcal{A})] - \Pr[\mathsf{Exp}^{(\beta)}_{\mathsf{embed}}(\mathcal{A})]\right|$. We construct an attacker $\mathcal{B} = (\mathsf{Samp}, \mathcal{B}')$ for the iO security game $\mathsf{ExptSI}(b)$. Algorithm Samp works as follows:

1. Algorithm Samp runs $\mathcal{A}$ to get a circuit C, an unauthorized input $x \in \{0,1\}^n$ and a pair of secrets s_0, s_1.
2. Algorithm Samp computes the circuits and the state as follows:
   (a) Samples the hash keys $\mathsf{hk}_{\mathbf{w}} \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Setup}(1^\lambda, 1)$ and $\mathsf{hk}_C \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Setup}(1^\lambda, 1^{\ell_{\mathsf{gate}}})$.
   (b) Samples the PPRF key $K_\lambda \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{\log \ell}, 1^\lambda)$.
   (c) Samples the circuit evaluation embedding seed and trapdoor $(\alpha, \mathsf{td}_{\mathsf{CEE}}) \leftarrow \Pi_{\mathsf{CEE}}.\mathsf{EmbedGen}(1^\lambda, 1^n, C, x)$.
   (d) Expands $\mathbf{w} = \Pi_{\mathsf{CEE}}.\mathsf{Expand}(C, \alpha)$.
   (e) Hashes $h_{\mathbf{w}} = \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_{\mathbf{w}}, \mathbf{w})$ and $h_C = \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_C, C)$.
   (f) Computes the two circuits:

$$C_0 = \mathsf{ComputePolicy}[\mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda, s_\beta]$$
$$C_1 = \mathsf{ComputePolicy}'[n+1, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda, \mathsf{td}_{\mathsf{CEE}}, s_\beta]$$

   (g) Set the state st as the bit β, the string $\mathbf{w}$, the state $\mathsf{st}_{\mathcal{A}}$ of the algorithm $\mathcal{A}$, and the shares $\mathsf{sh}_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i)$ for each $i \in [n]$.
3. Output (C_0, C_1, st).

Algorithm $\mathcal{B}'$ works as follows:

1. On input an obfuscated program P and the state $\mathsf{st} = (\beta, \mathbf{w}, \mathsf{st}_{\mathcal{A}}, \mathsf{sh}_1, \ldots, \mathsf{sh}_n)$, algorithm $\mathcal{B}'$ sets $\mathsf{sh}_0 = (P, \alpha, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}})$ and gives $\mathsf{sh}_0, \{\mathsf{sh}\}_{i \in S_x}$ to $\mathcal{A}$.
2. Algorithm $\mathcal{A}$ outputs a guess β', which $\mathcal{B}$ outputs as well.

First, we observe that the two circuits C_0 and C_1 are functionally equivalent: the added check in Step 4 only affects $i < n+1$. But the range check in Step 1a guarantees that $i \geq n+1$. Second, we note that if P is an obfuscation of the circuit C_0, then $\mathcal{B}$ simulates $\mathsf{Exp}^{(\beta)}_{\mathsf{embed}}(\mathcal{A})$, whereas if P is an obfuscation of the circuit C_1, then $\mathcal{B}$ simulates $\mathsf{Exp}^{(\beta)}_{n+1}(\mathcal{A})$. Therefore the advantage of $\mathcal{B}$ in the iO security game is exactly ε and therefore ε is negligible by the security of $i\mathcal{O}$ and the claim follows. □

Lemma 4.7. *For any algorithm $\mathcal{B}$*

$$\Pr[\mathsf{sExp}^{(\beta, i^*)}_0(\mathcal{A})] = \Pr[\mathsf{Exp}^{(\beta)}_{i^*}(\mathcal{A})]$$

Proof. By definition, these hybrids are identical. □

Lemma 4.8. *If Π_{PA} satisfies setup indistinguishability, then for any efficient algorithm $\mathcal{A}$, there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that*

$$\left|\Pr[\mathsf{sExp}^{(\beta, i^*)}_1(\mathcal{A})] - \Pr[\mathsf{sExp}^{(\beta, i^*)}_0(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda)$$

Proof. Denote $\varepsilon := \left|\Pr[\mathsf{sExp}_1^{(\beta,i^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_0^{(\beta,i^*)}(\mathcal{A})]\right|$. We construct an attacker $\mathcal{B}$ for the setup indistinguishability game $\mathsf{ExptSI}(b)$ as follows

1. Algorithm $\mathcal{B}$ runs $\mathcal{A}$ to get a circuit C, an unauthorized input $x \in \{0,1\}^n$ and a pair of secrets s_0, s_1.
2. Algorithm $\mathcal{B}$ chooses the block length $1^{\ell_{\mathsf{gate}}}$, the input sequence C (the vector representation of the policy circuit) and the index i^*.
3. The challenger samples $\mathsf{hk}_0 \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Setup}(1^\lambda, 1^{\ell_{\mathsf{gate}}})$ and $\mathsf{hk}_1 \leftarrow \Pi_{\mathsf{PA}}.\mathsf{SetupEnforce}(1^\lambda, 1^{\ell_{\mathsf{gate}}}, C, i^*)$ and gives hk_b to $\mathcal{B}$.
4. Algorithm $\mathcal{B}$ samples the shares as follows:
   (a) Samples the positional accumulator key $\mathsf{hk}_{\mathbf{w}} \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Setup}(1^\lambda, 1)$ and sets $\mathsf{hk}_C = \mathsf{hk}_b$.
   (b) Samples the PPRF key $K_\lambda \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{\log \ell}, 1^\lambda)$.
   (c) Samples the circuit evaluation embedding seed and trapdoor $(\alpha, \mathsf{td}_{\mathsf{CEE}}) \leftarrow \Pi_{\mathsf{CEE}}.\mathsf{EmbedGen}(1^\lambda, 1^n, C, x)$.
   (d) Expands $\mathbf{w} = \Pi_{\mathsf{CEE}}.\mathsf{Expand}(C, \alpha)$.
   (e) Hashes $h_{\mathbf{w}} = \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_{\mathbf{w}}, \mathbf{w})$ and $h_C = \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_C, C)$.
   (f) Obfuscates the program $P \leftarrow i\mathcal{O}(\mathsf{ComputePolicy}'[i^*, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda, \mathsf{td}_{\mathsf{CEE}}, s_\beta])$ from Fig. 2.
   (g) Sets $\mathsf{sh}_0 = (P, \alpha, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}})$, and $\mathsf{sh}_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i)$ for each $i \in [n]$.
   (h) Gives $\mathsf{sh}_0, \{\mathsf{sh}\}_{i \in S_x}$ to $\mathcal{A}$.
5. Algorithm $\mathcal{A}$ outputs a bit β', which $\mathcal{B}$ outputs as well.

If $b = 0$, then $\mathcal{B}$ simulates exactly $\mathsf{sExp}_0^{(\beta,i^*)}(\mathcal{A})$, whereas if $b = 1$ then $\mathcal{B}$ simulates exactly $\mathsf{sExp}_1^{(\beta,i^*)}(\mathcal{A})$. Therefore the advantage of $\mathcal{B}$ in the setup indistinguishability game is exactly ε and therefore ε is negligible by the setup indistinguishability of Π_{PA} and the claim follows. □

Lemma 4.9. *If Π_{PA} satisfies setup indistinguishability, then for any efficient algorithm $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that*

$$\left|\Pr[\mathsf{sExp}_2^{(\beta,i^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_1^{(\beta,i^*)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

This claim follows by an analogous argument to Lemma 4.8.

Lemma 4.10. *If $i\mathcal{O}$ is a secure indistinguishability obfuscator, then for any efficient algorithm $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that*

$$\left|\Pr[\mathsf{sExp}_3^{(\beta,i^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_2^{(\beta,i^*)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

Proof. Denote $\varepsilon := \left|\Pr[\mathsf{sExp}_3^{(\beta,i^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_2^{(\beta,i^*)}(\mathcal{A})]\right|$. We construct an attacker $\mathcal{B} = (\mathsf{Samp}, \mathcal{B}')$ for the iO security game $\mathsf{ExptSI}(b)$. Algorithm Samp works as follows:

1. Algorithm Samp runs $\mathcal{A}$ to get a circuit C, an unauthorized input $x \in \{0,1\}^n$ and a pair of secrets s_0, s_1.
2. Algorithm Samp computes the circuits and the state as follows:
   (a) Samples the hash keys $\mathsf{hk}_{\mathbf{w}} \leftarrow \Pi_{\mathsf{PA}}.\mathsf{SetupEnforce}(1^\lambda, 1, \mathbf{w}, i^*)$ and $\mathsf{hk}_C \leftarrow \Pi_{\mathsf{PA}}.\mathsf{SetupEnforce}(1^\lambda, 1^{\ell_{\mathsf{gate}}}, C, i^*)$.
   (b) Samples the PPRF key $K_\lambda \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{\log \ell}, 1^\lambda)$.
   (c) Samples the circuit evaluation embedding seed and trapdoor $(\alpha, \mathsf{td}_{\mathsf{CEE}}) \leftarrow \Pi_{\mathsf{CEE}}.\mathsf{EmbedGen}(1^\lambda, 1^n, C, x)$.
   (d) Expands $\mathbf{w} = \Pi_{\mathsf{CEE}}.\mathsf{Expand}(C, \alpha)$.
   (e) Hashes $h_{\mathbf{w}} = \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_{\mathbf{w}}, \mathbf{w})$ and $h_C = \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_C, C)$.

(f) Let i_0^*, i_1^* be the children gate i^*, and let $x_{i_b^*} = C(x)[i_b^*]$ be the value of the gate i_b^* in the evaluation of C on x.

(g) For each $b \in \{0,1\}$, sets $z_b = \mathsf{PRG}(\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i_b^*))$.

(h) Computes the two circuits:

$$C_0 = \mathsf{ComputePolicy}'[i^*, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda, \mathsf{td}_{\mathsf{CEE}}, s_\beta]$$
$$C_1 = \mathsf{ComputePolicy}^1[i^*, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda, \mathsf{td}_{\mathsf{CEE}}, x_{i_0^*}, x_{i_1^*}, z_0, z_1, s_\beta]$$

(i) Set the state st as the bit β, the string $\mathbf{w}$, the state $\mathsf{st}_{\mathcal{A}}$ of the algorithm $\mathcal{A}$, and the shares $\mathsf{sh}_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i)$ for each $i \in [n]$.

3. Output (C_0, C_1, st).

Algorithm $\mathcal{B}'$ works as follows:

1. On input an obfuscated program P and the state $\mathsf{st} = (\beta, \mathbf{w}, \mathsf{st}_{\mathcal{A}}, \mathsf{sh}_1, \ldots, \mathsf{sh}_n)$, algorithm $\mathcal{B}'$ sets $\mathsf{sh}_0 = (P, \alpha, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}})$ and gives $\mathsf{sh}_0, \{\mathsf{sh}\}_{i \in S_x}$ to $\mathcal{A}$.
2. Algorithm $\mathcal{A}$ outputs a guess β', which $\mathcal{B}$ outputs as well.

First, we observe that the two circuits C_0 and C_1 are functionally equivalent. Second, we note that if P is an obfuscation of the circuit C_0, then $\mathcal{B}$ simulates $\mathsf{sExp}_2^{(\beta,i^*)}(\mathcal{A})$, whereas if P is an obfuscation of the circuit C_1, then $\mathcal{B}$ simulates $\mathsf{sExp}_3^{(\beta,i^*)}(\mathcal{A})$. Therefore the advantage of $\mathcal{B}$ in the iO security game is exactly ε and therefore ε is negligible by the security of $i\mathcal{O}$ and the claim follows. □

Lemma 4.11. *If $i\mathcal{O}$ is a secure indistinguishability obfuscator and Π_{PPRF} satisfies functionality preserving, then for any efficient algorithm $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that*

$$\left|\Pr[\mathsf{sExp}_4^{(\beta,i^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_3^{(\beta,i^*)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

Proof. Denote $\varepsilon := \left|\Pr[\mathsf{sExp}_4^{(\beta,i^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_3^{(\beta,i^*)}(\mathcal{A})]\right|$. We construct an attacker $\mathcal{B} = (\mathsf{Samp}, \mathcal{B}')$ for the iO security game $\mathsf{ExptSI}(b)$. Algorithm Samp works as follows:

1. Algorithm Samp runs $\mathcal{A}$ to get a circuit C, an unauthorized input $x \in \{0,1\}^n$ and a pair of secrets s_0, s_1.
2. Algorithm Samp computes the circuits and the state as follows:
   (a) Samples the hash keys $\mathsf{hk}_{\mathbf{w}} \leftarrow \Pi_{\mathsf{PA}}.\mathsf{SetupEnforce}(1^\lambda, 1, \mathbf{w}, i^*)$ and $\mathsf{hk}_C \leftarrow \Pi_{\mathsf{PA}}.\mathsf{SetupEnforce}(1^\lambda, 1^{\ell_{\mathsf{gate}}}, C, i^*)$.
   (b) Samples the PPRF key $K_\lambda \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{\log \ell}, 1^\lambda)$.
   (c) Computes a punctured key $K_\lambda^{(S)} \leftarrow \mathsf{Puncture}(K, S)$, where $S = \{i_b^* : x_{i_b^*} = 0\}$.
   (d) Samples the circuit evaluation embedding seed and trapdoor $(\alpha, \mathsf{td}_{\mathsf{CEE}}) \leftarrow \Pi_{\mathsf{CEE}}.\mathsf{EmbedGen}(1^\lambda, 1^n, C, x)$.
   (e) Expands $\mathbf{w} = \Pi_{\mathsf{CEE}}.\mathsf{Expand}(C, \alpha)$.
   (f) Hashes $h_{\mathbf{w}} = \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_{\mathbf{w}}, \mathbf{w})$ and $h_C = \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_C, C)$.
   (g) Let i_0^*, i_1^* be the children gates of the gate i^*, and let $x_{i_b^*}$ be the value of the gate i_b^* in the evaluation of $C(x)$.
   (h) For each $b \in \{0,1\}$, sets $z_b = \mathsf{PRG}(\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i_b^*))$.
   (i) Computes the two circuits:

$$C_0 = \mathsf{ComputePolicy}^1[i^*, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda, \mathsf{td}_{\mathsf{CEE}}, x_{i_0^*}, x_{i_1^*}, z_0, z_1, s_\beta]$$
$$C_1 = \mathsf{ComputePolicy}^1[i^*, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda^{(S)}, \mathsf{td}_{\mathsf{CEE}}, x_{i_0^*}, x_{i_1^*}, z_0, z_1, s_\beta]$$

(j) Set the state st as the bit β, the string $\mathbf{w}$, the state $\mathsf{st}_{\mathcal{A}}$ of the algorithm $\mathcal{A}$, and the shares $\mathsf{sh}_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i)$ for each $i \in [n]$.

3. Output (C_0, C_1, st).

Algorithm $\mathcal{B}'$ works as follows:

1. On input an obfuscated program P and the state $\mathsf{st} = (\beta, \mathbf{w}, \mathsf{st}_{\mathcal{A}}, \mathsf{sh}_1, \ldots, \mathsf{sh}_n)$, algorithm $\mathcal{B}'$ sets $\mathsf{sh}_0 = (P, \alpha, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}})$ and gives $\mathsf{sh}_0, \{\mathsf{sh}\}_{i \in S_x}$ to $\mathcal{A}$.
2. Algorithm $\mathcal{A}$ outputs a guess β', which $\mathcal{B}$ outputs as well.

First, by the functionality preserving and the fact that $\mathsf{ComputePolicy}^1$ never uses K_λ on points in S, the two circuits C_0, C_1 are functionally equivalent. Second, we note that if P is an obfuscation of the circuit C_0, then $\mathcal{B}$ simulates $\mathsf{sExp}_3^{(\beta,i^*)}(\mathcal{A})$, whereas if P is an obfuscation of the circuit C_1, then $\mathcal{B}$ simulates $\mathsf{sExp}_4^{(\beta,i^*)}(\mathcal{A})$. Therefore the advantage of $\mathcal{B}$ in the iO security game is exactly ε and therefore ε is negligible by the security of $i\mathcal{O}$ and the claim follows. □

Lemma 4.12. *If Π_{PPRF} satisfies punctured pseudorandomness, then for any efficient algorithm $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that*

$$\left|\Pr[\mathsf{sExp}_5^{(\beta,i^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_4^{(\beta,i^*)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

Proof. Denote $\varepsilon := \left|\Pr[\mathsf{sExp}_5^{(\beta,i^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_4^{(\beta,i^*)}(\mathcal{A})]\right|$. We construct an attacker $\mathcal{B}$ for the punctured pseudorandomness game $\mathsf{ExptPP}(b)$ as follows

1. Algorithm $\mathcal{B}$ runs $\mathcal{A}$ to get a circuit C, an unauthorized input $x \in \{0,1\}^n$ and a pair of secrets s_0, s_1. Let $S = \{i_b^* : x_{i_b^*} = 0\}$ and $\ell = |C|$ where $x_{i_b^*} = C(x)[i_b^*]$ is the value that gate i_b^* takes in the evaluation of C on x.
2. Algorithm $\mathcal{B}$ chooses the input length $1^{\log \ell}$ and the output length 1^λ and the set S.
3. The challenger samples the PPRF key $K \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{\log \ell}, 1^\lambda)$ and gives the punctured key $K^{(S)} \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Puncture}(K_\lambda, S)$.
4. If $b = 0$, the challenger sets $y_j^* = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K, j)$ for all $j \in S$. If $b = 1$, the challenger samples $y_j^* \xleftarrow{R} \{0,1\}^\lambda$. In any case, the challenger gives $(y_j^*)_{j \in S}$ to $\mathcal{B}$.
5. Algorithm $\mathcal{B}$ samples the shares as follows:
   (a) Samples the hash keys $\mathsf{hk}_{\mathbf{w}} \leftarrow \Pi_{\mathsf{PA}}.\mathsf{SetupEnforce}(1^\lambda, 1, \mathbf{w}, i^*)$ and $\mathsf{hk}_C \leftarrow \Pi_{\mathsf{PA}}.\mathsf{SetupEnforce}(1^\lambda, 1^{\ell_{\mathsf{gate}}}, C, i^*)$.
   (b) Sets $K_\lambda^{(S)} = K^{(S)}$.
   (c) Samples the circuit evaluation embedding seed and trapdoor $(\alpha, \mathsf{td}_{\mathsf{CEE}}) \leftarrow \Pi_{\mathsf{CEE}}.\mathsf{EmbedGen}(1^\lambda, 1^n, C, x)$.
   (d) Expands $\mathbf{w} = \Pi_{\mathsf{CEE}}.\mathsf{Expand}(C, \alpha)$.
   (e) Hashes $h_{\mathbf{w}} = \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_{\mathbf{w}}, \mathbf{w})$ and $h_C = \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_C, C)$.
   (f) Sets $z_{b'} = \mathsf{PRG}(y_{i_{b'}^*}^*)$ for each $i_{b'}^* \in S$. For $i_{b'}^* \notin S$, computes $z_{b'} = \mathsf{PRG}(\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda^{(S)}, i_{b'}^*))$.
   (g) Obfuscates the program $P \leftarrow i\mathcal{O}(\mathsf{ComputePolicy}^1[i^*, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda^{(S)}, \mathsf{td}_{\mathsf{CEE}}, x_{i_0^*}, x_{i_1^*}, z_0, z_1, s_\beta])$ from Fig. 3.
   (h) Sets $\mathsf{sh}_0 = (P, \alpha, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}})$, and $\mathsf{sh}_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i)$ for each $i \in [n]$.
   (i) Gives $\mathsf{sh}_0, \{\mathsf{sh}\}_{i \in S_x}$ to $\mathcal{A}$.
6. Algorithm $\mathcal{A}$ outputs a bit β', which $\mathcal{B}$ outputs as well.

If $b = 0$, then $\mathcal{B}$ simulates exactly $\mathsf{sExp}_4^{(\beta,i^*)}(\mathcal{A})$, whereas if $b = 1$ then $\mathcal{B}$ simulates exactly $\mathsf{sExp}_5^{(\beta,i^*)}(\mathcal{A})$. Therefore the advantage of $\mathcal{B}$ in the punctured pseudorandomness game is exactly ε and therefore ε is negligible by the punctured pseudorandomness security of Π_{PPRF} and the claim follows. □

Lemma 4.13. *If* PRG *is secure, then for any efficient algorithm $\mathcal{A}$, there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that*

$$\left|\Pr[\mathsf{sExp}_6^{(\beta,i^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_5^{(\beta,i^*)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

Proof. Denote $\varepsilon := \left|\Pr[\mathsf{sExp}_6^{(\beta,i^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_5^{(\beta,i^*)}(\mathcal{A})]\right|$. We construct an attacker $\mathcal{B}$ for the security game of the PRG with 2 samples as follows. Let $b \in \{0,1\}$ denote whether the PRG challenger sends uniformly random samples or random PRG outputs.

1. Algorithm $\mathcal{B}$ runs $\mathcal{A}$ to get a circuit C, an unauthorized input $x \in \{0,1\}^n$ and a pair of secrets s_0, s_1. Let $S = \{i_b^* : x_{i_b^*} = 0\}$ and $\ell = |C|$ where $x_{i_b^*} = C(x)[i_b^*]$ is the value that gate i_b^* takes in the evaluation of C on x.
2. If $b = 0$, the challenger sets $r_{b'} = \mathsf{PRG}(s'_{b'})$ for all $b' \in \{0,1\}$. If $b = 1$, the challenger samples $r_{b'} \xleftarrow{R} \{0,1\}^{2\lambda}$. In any case, the challenger gives $(r_{b'})_{b' \in \{0,1\}}$ to $\mathcal{B}$.
3. Algorithm $\mathcal{B}$ samples the shares as follows:
   (a) Samples the hash keys $\mathsf{hk}_{\mathbf{w}} \leftarrow \Pi_{\mathsf{PA}}.\mathsf{SetupEnforce}(1^\lambda, 1, \mathbf{w}, i^*)$ and $\mathsf{hk}_C \leftarrow \Pi_{\mathsf{PA}}.\mathsf{SetupEnforce}(1^\lambda, 1^{\ell_{\mathsf{gate}}}, C, i^*)$.
   (b) Samples the PPRF key $K_\lambda \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{\log \ell}, 1^\lambda)$.
   (c) Punctures $K_\lambda^{(S)} \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Puncture}(K_\lambda, S)$.
   (d) Samples the circuit evaluation embedding seed and trapdoor $(\alpha, \mathsf{td}_{\mathsf{CEE}}) \leftarrow \Pi_{\mathsf{CEE}}.\mathsf{EmbedGen}(1^\lambda, 1^n, C, x)$.
   (e) Expands $\mathbf{w} = \Pi_{\mathsf{CEE}}.\mathsf{Expand}(C, \alpha)$.
   (f) Hashes $h_{\mathbf{w}} = \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_{\mathbf{w}}, \mathbf{w})$ and $h_C = \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_C, C)$.
   (g) Sets $z_{b'} = r_{b'}$ for each $i_{b'}^* \in S$. For $i_{b'}^* \notin S$, computes $z_{b'} = \mathsf{PRG}(\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda^{(S)}, i_{b'}^*))$.
   (h) Obfuscates the program $P \leftarrow i\mathcal{O}(\mathsf{ComputePolicy}^1[i^*, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda^{(S)}, \mathsf{td}_{\mathsf{CEE}}, x_{i_0^*}, x_{i_1^*}, z_0, z_1, s_\beta])$ from Fig. 3.
   (i) Sets $\mathsf{sh}_0 = (P, \alpha, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}})$, and $\mathsf{sh}_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i)$ for each $i \in [n]$.
   (j) Gives $\mathsf{sh}_0, \{\mathsf{sh}\}_{i \in S_x}$ to $\mathcal{A}$.
4. Algorithm $\mathcal{A}$ outputs a bit β', which $\mathcal{B}$ outputs as well.

If $b = 0$, then $\mathcal{B}$ simulates exactly $\mathsf{sExp}_5^{(\beta,i^*)}(\mathcal{A})$, whereas if $b = 1$ then $\mathcal{B}$ simulates exactly $\mathsf{sExp}_6^{(\beta,i^*)}(\mathcal{A})$. Therefore the advantage of $\mathcal{B}$ in the PRG security game is exactly ε and therefore ε is negligible by the security of PRG and the claim follows. □

Lemma 4.14. *If $i\mathcal{O}$ is a secure indistinguishability obfuscator, then for any efficient algorithm $\mathcal{A}$, there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that*

$$\left|\Pr[\mathsf{sExp}_7^{(\beta,i^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_6^{(\beta,i^*)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

Proof. Denote $\varepsilon := \left|\Pr[\mathsf{sExp}_7^{(\beta,i^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_6^{(\beta,i^*)}(\mathcal{A})]\right|$. We construct an attacker $\mathcal{B} = (\mathsf{Samp}, \mathcal{B}')$ for the iO security game $\mathsf{ExptSI}(b)$. Algorithm Samp works as follows:

1. Algorithm Samp runs $\mathcal{A}$ to get a circuit C, an unauthorized input $x \in \{0,1\}^n$ and a pair of secrets s_0, s_1.
2. Algorithm Samp computes the circuits and the state as follows:
   (a) Samples the hash keys $\mathsf{hk}_{\mathbf{w}} \leftarrow \Pi_{\mathsf{PA}}.\mathsf{SetupEnforce}(1^\lambda, 1, \mathbf{w}, i^*)$ and $\mathsf{hk}_C \leftarrow \Pi_{\mathsf{PA}}.\mathsf{SetupEnforce}(1^\lambda, 1^{\ell_{\mathsf{gate}}}, C, i^*)$.
   (b) Samples the PPRF key $K_\lambda \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{\log \ell}, 1^\lambda)$.
   (c) Punctures $K_\lambda^{(S)} \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Puncture}(K_\lambda, S)$.
   (d) Samples the circuit evaluation embedding seed and trapdoor $(\alpha, \mathsf{td}_{\mathsf{CEE}}) \leftarrow \Pi_{\mathsf{CEE}}.\mathsf{EmbedGen}(1^\lambda, 1^n, C, x)$.

(e) Expands $\mathbf{w} = \Pi_{\mathsf{CEE}}.\mathsf{Expand}(C, \alpha)$.

(f) Hashes $h_{\mathbf{w}} = \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_{\mathbf{w}}, \mathbf{w})$ and $h_C = \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_C, C)$.

(g) Let i_0^*, i_1^* be the children of the gate i^*, and let $x_{i_b^*} = C(x)[i_b^*]$ be the value of the gate i_b^* in the evaluation of C on x.

(h) For each $b \in \{0,1\}$, sets $z_b \xleftarrow{R} \{0,1\}^{2\lambda}$.

(i) Computes the two circuits:

$$C_0 = \mathsf{ComputePolicy}^1[i^*, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda^{(S)}, \mathsf{td}_{\mathsf{CEE}}, x_{i_0^*}, x_{i_1^*}, z_0, z_1, s_\beta]$$
$$C_1 = \mathsf{ComputePolicy}^2[i^*, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda^{(S)}, \mathsf{td}_{\mathsf{CEE}}, x_{i_0^*}, x_{i_1^*}, s_\beta]$$

(j) Set the state st as the bit β, the string $\mathbf{w}$, the state $\mathsf{st}_{\mathcal{A}}$ of the algorithm $\mathcal{A}$, and the shares $\mathsf{sh}_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i)$ for each $i \in [n]$.

3. Output (C_0, C_1, st).

Algorithm $\mathcal{B}'$ works as follows:

1. On input an obfuscated program P and the state $\mathsf{st} = (\beta, \mathbf{w}, \mathsf{st}_{\mathcal{A}}, \mathsf{sh}_1, \ldots, \mathsf{sh}_n)$, algorithm $\mathcal{B}'$ sets $\mathsf{sh}_0 = (P, \alpha, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}})$ and gives $\mathsf{sh}_0, \{\mathsf{sh}\}_{i \in S_x}$ to $\mathcal{A}$.
2. Algorithm $\mathcal{A}$ outputs a guess β', which $\mathcal{B}$ outputs as well.

Let E be the event that z_0 and z_1 are outside of the image of PRG. This event happens with overwhelming probability $(1-2^\lambda)^2 \geq 1 - 2^{\lambda-1}$. Conditioned on event E, the two circuits C_0 and C_1 are functionally equivalent, since z_b being outside of the PRG image in C_0 is identical to always setting $v_{i_b^*} = 0$ when $x_{i_b^*} = 0$. From here, the proof follows that of Lemma 4.11. □

Lemma 4.15. *If $i\mathcal{O}$ is a secure indistinguishability obfuscator, then for any efficient algorithm $\mathcal{A}$, there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that*

$$\left|\Pr[\mathsf{sExp}_8^{(\beta,i^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_7^{(\beta,i^*)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

Proof. Denote $\varepsilon := \left|\Pr[\mathsf{sExp}_8^{(\beta,i^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_7^{(\beta,i^*)}(\mathcal{A})]\right|$. We construct an attacker $\mathcal{B} = (\mathsf{Samp}, \mathcal{B}')$ for the iO security game $\mathsf{ExptSI}(b)$. Algorithm Samp works as follows:

1. Algorithm Samp runs $\mathcal{A}$ to get a circuit C, an unauthorized input $x \in \{0,1\}^n$ and a pair of secrets s_0, s_1.
2. Algorithm Samp computes the circuits and the state as follows:

(a) Samples the hash keys $\mathsf{hk}_{\mathbf{w}} \leftarrow \Pi_{\mathsf{PA}}.\mathsf{SetupEnforce}(1^\lambda, 1, \mathbf{w}, i^*)$ and $\mathsf{hk}_C \leftarrow \Pi_{\mathsf{PA}}.\mathsf{SetupEnforce}(1^\lambda, 1^{\ell_{\mathsf{gate}}}, C, i^*)$.

(b) Samples the PPRF key $K_\lambda \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{\log \ell}, 1^\lambda)$.

(c) Punctures $K_\lambda^{(S)} \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Puncture}(K_\lambda, S)$.

(d) Samples the circuit evaluation embedding seed and trapdoor $(\alpha, \mathsf{td}_{\mathsf{CEE}}) \leftarrow \Pi_{\mathsf{CEE}}.\mathsf{EmbedGen}(1^\lambda, 1^n, C, x)$.

(e) Expands $\mathbf{w} = \Pi_{\mathsf{CEE}}.\mathsf{Expand}(C, \alpha)$.

(f) Hashes $h_{\mathbf{w}} = \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_{\mathbf{w}}, \mathbf{w})$ and $h_C = \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_C, C)$.

(g) Let i_0^*, i_1^* be the children of the gate i^*, and let $x_{i_b^*} = C(x)[i_b^*]$ be the value of the i_b^* in the evaluation of C on x.

(h) Computes the two circuits:

$$C_0 = \mathsf{ComputePolicy}^2[i^*, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda^{(S)}, \mathsf{td}_{\mathsf{CEE}}, x_{i_0^*}, x_{i_1^*}, s_\beta]$$
$$C_1 = \mathsf{ComputePolicy}^3[i^*, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda^{(S)}, \mathsf{td}_{\mathsf{CEE}}, x_{i_0^*}, x_{i_1^*}, s_\beta]$$

(i) Set the state st as the bit β, the string $\mathbf{w}$, the state $\mathsf{st}_{\mathcal{A}}$ of the algorithm $\mathcal{A}$, and the shares $\mathsf{sh}_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i)$ for each $i \in [n]$.

3. Output (C_0, C_1, st).

Algorithm $\mathcal{B}'$ works as follows:

1. On input an obfuscated program P and the state $\mathsf{st} = (\beta, \mathbf{w}, \mathsf{st}_{\mathcal{A}}, \mathsf{sh}_1, \ldots, \mathsf{sh}_n)$, algorithm $\mathcal{B}'$ sets $\mathsf{sh}_0 = (P, \alpha, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}})$ and gives $\mathsf{sh}_0, \{\mathsf{sh}\}_{i \in S_x}$ to $\mathcal{A}$.

2. Algorithm $\mathcal{A}$ outputs a guess β', which $\mathcal{B}$ outputs as well.

We argue that the two circuits C_0 and C_1 are functionally equivalent. Note that the only difference is that C_1 additionally checks the condition

$$\Pi_{\mathsf{CEE}}.\mathsf{Extract}(\mathsf{td}_{\mathsf{CEE}}, i^*, w_{i^*}) = 0 \text{ then check that } v_{i^*} = 0.$$

Recall that hash key $\mathsf{hk}_{\mathbf{w}}$ is statistically enforcing on i^*, and the honest value w_{i^*} on this position satisfies $x_{i^*} = \Pi_{\mathsf{CEE}}.\mathsf{Extract}(\mathsf{td}_{\mathsf{CEE}}, i^*, w_{i^*})$ by the statistical extraction property of Definition 3.1. Therefore with overwhelming probability $1 - \varepsilon_{\mathsf{PA}}$, there does not exist an opening on this position to any other values. Assume this event happens. Then by construction, $\Pi_{\mathsf{CEE}}.\mathsf{Extract}(\mathsf{td}_{\mathsf{CEE}}, i^*, w_{i^*}) = 0$ implies that $x_{i^*} = 0$. By definition, this implies that $g(x_{i_0^*}, x_{i_1^*}) = x_{i^*} = 0$. By line 2 (of either program), we know that $x_{i_b^*} \geq v_{i_b^*}$, therefore

$$v_{i^*} = g(v_{i_0^*}, v_{i_1^*}) \leq g(x_{i_0^*}, x_{i_1^*}) = 0$$

where the inequality follows by the monotonicity of the gate g. We conclude that the check added in C_1 holds anyway in C_0, so the two programs are functionally equivalent. From here, the proof follows that of Lemma 4.11. □

Lemma 4.16. *If $i\mathcal{O}$ is a secure indistinguishability obfuscator, then for any efficient algorithm $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that*

$$\left|\Pr[\mathsf{sExp}_9^{(\beta,i^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_8^{(\beta,i^*)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

This claim follows is analogous to Lemma 4.14 and the follows by a similar argument.

Lemma 4.17. *If* PRG *is secure, then for any efficient algorithm $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that*

$$\left|\Pr[\mathsf{sExp}_{10}^{(\beta,i^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_9^{(\beta,i^*)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

This claim follows is analogous to Lemma 4.13 and the follows by a similar argument.

Lemma 4.18. *If Π_{PPRF} satisfies punctured pseudorandomness, then for any efficient algorithm $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that*

$$\left|\Pr[\mathsf{sExp}_{11}^{(\beta,i^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_{10}^{(\beta,i^*)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

This claim follows is analogous to Lemma 4.12 and the follows by a similar argument.

Lemma 4.19. *If $i\mathcal{O}$ is a secure indistinguishability obfuscator and Π_{PPRF} satisfies functionality preserving, then for any efficient algorithm $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that*

$$\left|\Pr[\mathsf{sExp}_{12}^{(\beta,i^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_{11}^{(\beta,i^*)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

This claim follows is analogous to Lemma 4.11 and the follows by a similar argument.

Lemma 4.20. *If $i\mathcal{O}$ is a secure indistinguishability obfuscator, then for any efficient algorithm $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that*

$$\left|\Pr[\mathsf{sExp}_{13}^{(\beta,i^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_{12}^{(\beta,i^*)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

This claim follows is analogous to Lemma 4.10 and the follows by a similar argument.

Lemma 4.21. *If Π_{PA} satisfies setup indistinguishability, then for any efficient algorithm $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that*

$$\left|\Pr[\mathsf{sExp}_{14}^{(\beta,i^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_{13}^{(\beta,i^*)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

This claim follows is analogous to Lemma 4.9 and the follows by a similar argument.

Lemma 4.22. *If Π_{PA} satisfies setup indistinguishability, then for any efficient algorithm $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that*

$$\left|\Pr[\mathsf{sExp}_{15}^{(\beta,i^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_{14}^{(\beta,i^*)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

This claim follows is analogous to Lemma 4.8 and the follows by a similar argument.

Lemma 4.23. *For any algorithm $\mathcal{A}$*

$$\Pr[\mathsf{Exp}_{i^*+1}^{(\beta)}(\mathcal{A})] = \Pr[\mathsf{sExp}_{15}^{(\beta,i^*)}(\mathcal{A})].$$

Proof. Observe by $\mathsf{sExp}_{15}^{(\beta,i^*)}$, we have reverted all puncturing and hash key sampling. Thus, this only differs from $\mathsf{Exp}_{i^*}^{(\beta)}$ in using an obfuscation of $\mathsf{ComputePolicy}'[i^*+1,\ldots]$, which is exactly $\mathsf{Exp}_{i^*+1}^{(\beta)}$. □

Lemma 4.24. *Fix an efficient algorithm $\mathcal{A}$, and let ℓ be the length the policy C it chooses. If Π_{PA} satisfies setup indistinguishability, then there exists a negligible function $\mathsf{negl}(\lambda)$ such that*

$$\left|\Pr[\mathsf{Exp}_{\ell}^{14}(\mathcal{A})] - \Pr[\mathsf{Exp}_{\mathsf{final}}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

Proof. Denote $\varepsilon := \left|\Pr[\mathsf{Exp}_{\ell+1}(\mathcal{A})] - \Pr[\mathsf{Exp}_{\mathsf{final}}(\mathcal{A})]\right|$. We construct an attacker $\mathcal{B} = (\mathsf{Samp}, \mathcal{B}')$ for the iO security game $\mathsf{ExptSI}(b)$. Algorithm Samp works as follows:

1. Algorithm Samp runs $\mathcal{A}$ to get a circuit C, an unauthorized input $x \in \{0,1\}^n$ and a pair of secrets s_0, s_1.
2. Algorithm Samp computes the circuits and the state as follows:
   (a) Samples the hash keys $\mathsf{hk}_{\mathbf{w}} \leftarrow \Pi_{\mathsf{PA}}.\mathsf{SetupEnforce}(1^\lambda, 1, \mathbf{w}, \ell)$ and $\mathsf{hk}_C \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Setup}(1^\lambda, 1^{\ell_{\mathsf{gate}}})$.
   (b) Samples the PPRF key $K_\lambda \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{\log \ell}, 1^\lambda)$.
   (c) Samples the circuit evaluation embedding seed and trapdoor $(\alpha, \mathsf{td}_{\mathsf{CEE}}) \leftarrow \Pi_{\mathsf{CEE}}.\mathsf{EmbedGen}(1^\lambda, 1^n, C, x)$.
   (d) Expands $\mathbf{w} = \Pi_{\mathsf{CEE}}.\mathsf{Expand}(C, \alpha)$.
   (e) Hashes $h_{\mathbf{w}} = \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_{\mathbf{w}}, \mathbf{w})$ and $h_C = \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_C, C)$.
   (f) Computes the two circuits:

$$C_0 = \mathsf{ComputePolicy}'[\ell+1, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda^{(S)}, \mathsf{td}_{\mathsf{CEE}}, s_\beta]$$
$$C_1 = \mathsf{ComputePolicy}_{\mathsf{final}}[\mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda^{(S)}, \mathsf{td}_{\mathsf{CEE}}]$$

   (g) Set the state st as the bit β, the string $\mathbf{w}$, the state $\mathsf{st}_{\mathcal{A}}$ of the algorithm $\mathcal{A}$, and the shares $\mathsf{sh}_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i)$ for each $i \in [n]$.
3. Output (C_0, C_1, st).

Algorithm $\mathcal{B}'$ works as follows:

1. On input an obfuscated program P and the state $\mathsf{st} = (\beta, \mathbf{w}, \mathsf{st}_{\mathcal{A}}, \mathsf{sh}_1, \ldots, \mathsf{sh}_n)$, algorithm $\mathcal{B}'$ sets $\mathsf{sh}_0 = (P, \alpha, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}})$ and gives $\mathsf{sh}_0, \{\mathsf{sh}\}_{i \in S_x}$ to $\mathcal{A}$.
2. Algorithm $\mathcal{A}$ outputs a guess β', which $\mathcal{B}$ outputs as well.

We argue that the two circuits C_0 and C_1 are functionally equivalent. Note that C_0 does not skip the check on line 4 for any index since it is checked for all $i < i^* = \ell + 1$ and line 1a enforces $i \leq \ell$. Therefore, the only difference between the two programs is that C_1 always outputs $\perp$ on index $i = \ell$ whereas C_0 may output s_β only if $v_\ell = 1$. We argue that v_ℓ can equal 1 (and without C_0 aborting) only with negligible probability. Recall that $x_\ell := C(x) = 0$ for any permissible $\mathcal{A}$. Since $\mathsf{hk}_{\mathbf{w}}$ is enforcing on ℓ, then with all but negligible probability, there does not exist an opening for $h_{\mathbf{w}}$ on position i to any value other than w_ℓ that satisfies $\Pi_{\mathsf{CEE}}.\mathsf{Extract}(\mathsf{td}_{\mathsf{CEE}}, \ell, w_\ell) = x_\ell = 0$. Denote that event by E and assume it happens. Then the check

$$\Pi_{\mathsf{CEE}}.\mathsf{Extract}(\mathsf{td}_{\mathsf{CEE}}, \ell, w_\ell) = 0 \Rightarrow v_\ell = 0$$

on line 4 guarantees that $v_\ell = 0$. Therefore C_0 always outputs $\perp$ on index $i = \ell$ (conditioned on the event E). Therefore with all but negligible probability over $\mathsf{hk}_{\mathbf{w}}$, the two circuits are functionally equivalent. From here, the proof follows that of Lemma 4.11. □

Next, we prove that it is statistically impossible to win in the final experiment.

Lemma 4.25. *Fix any (potentially unbounded) algorithm $\mathcal{A}$, and let ℓ be the length the policy C it chooses. Then* $\Pr[\mathsf{Exp}_{\mathsf{final}}(\mathcal{A}) = 1] = \frac{1}{2}$.

Proof. The view of $\mathcal{A}$ does not depend on the bit β chosen by the challenger. Namely, for all $\beta \in \{0, 1\}$, the obfuscated program $P \leftarrow i\mathcal{O}(\mathsf{ComputePolicy}_{\mathsf{final}}[i^*, \mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, h_{\mathbf{w}}, K_\lambda, \mathsf{td}_{\mathsf{CEE}}])$ is identically distributed. Therefore, the output bit of the adversary β' is independent of β and thus $\Pr[\beta' = \beta] = 1/2$. □

Fix an efficient adversary $\mathcal{A}$ for the security game of Definition 2.2, and let ℓ be the length the policy C it chooses. By Lemmas 4.5 to 4.24, we conclude that for any efficient adversary $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that

$$\left|\Pr[\mathsf{Exp}_{\mathsf{real}}(\lambda) = 1] - \Pr[\mathsf{Exp}_{\mathsf{final}}(\lambda) = 1]\right| < \mathsf{negl}(\lambda).$$

By Lemma 4.25, it holds that $\Pr[\mathsf{Exp}_{\mathsf{final}}(\mathcal{A}) = 1] = \frac{1}{2}$. In total, we conclude that $\Pr[\mathsf{Exp}_{\mathsf{real}}(\mathcal{A}) = 1] = \frac{1}{2}$ and the theorem follows. □

5 Constructing Circuit Evaluation Embeddings

In this section, we construct a circuit evaluation embedding Definition 3.1 for all monotone Boolean circuits with seed size $n + \mathsf{poly}(\lambda, \log |C|)$.

Construction 5.1. Our construction makes use of the following ingredients:

- An indistinguishability obfuscator for circuits $i\mathcal{O}$.
- A positional accumulator $\Pi_{\mathsf{PA}} = (\mathsf{Setup}, \mathsf{SetupEnforce}, \mathsf{Hash}, \mathsf{Open}, \mathsf{Verify})$.
- A puncturable PRF $\Pi_{\mathsf{PPRF}} = (\mathsf{KeyGen}, \mathsf{Eval}, \mathsf{Puncture})$.
- An injective one way function F and associated hard core predicate $\mathsf{hcb}(\cdot)$. We assume any randomness for hcb is implicitly hard-coded where hcb is used.

Apart from injective one way functions, these primitives all follow from a combination of indistinguishability obfuscation and one way functions. Our use of injective one way functions can be realized via subexponential one way functions and $i\mathcal{O}$ [BPW15] or non-uniformly secure one way functions [WW24].

We compress our representation of the string $\mathbf{w}$ by generating it via an $i\mathcal{O}$ program. While initially, our $i\mathcal{O}$ program simply produces the output of a PRF, we incrementally switch our program to homomorphically compute the evaluation of the policy circuit on the challenge input. We use a combination of positional accumulators and another puncturable PRF to ensure that the obfuscated program is only able follow the 'honest' evaluation path, despite being much smaller than the policy circuit.

- $\mathsf{Gen}(1^\lambda, 1^n, C) \to \alpha$: On input security parameter λ, input length n, and circuit C, the Gen algorithm does as follows:

  1. Sample the following hash and PRF keys:
     (a) $\mathsf{hk}_{\mathbf{w}} \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Setup}(1^\lambda, 1, 1^2)$ is a positional accumulator key on one bit blocks which can bind on 2 positions.
     (b) $\mathsf{hk}_C \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Setup}(1^\lambda, 1^{1+2\log \ell})$ is a positional accumulator key on blocks of the form $C_i = g_i \times [\ell] \times [\ell]$.
        – Set $h_C \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_C, C)$.
     (c) $K_{\mathsf{bit}} \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{\log \ell}, 1)$ is a puncturable PRF key with domain $[\ell]$ and one bit outputs.
     (d) $K_\lambda \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{\log \ell}, 1^{2\lambda})$ is a puncturable PRF key with domain $[\ell]$ and outputs of the form $\{0,1\}^\lambda \times \{0,1\}^\lambda$.
  2. For $i \in [n]$, set $w_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i)$. Let $\mathsf{sd} = w_1 w_2 \dots w_n$.
     (a) Set $h_{\mathsf{sd}} \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_{\mathbf{w}}, \mathsf{sd})$.
     (b) Compute $(y_n, r'_n) \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, n)$.
     (c) Compute $K'_n \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{|h|}, 1^\lambda; r'_n)$.
     (d) Set $\sigma_n = y_n \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K'_n, h_{\mathsf{sd}})$.
  3. Output $\alpha = (\mathsf{hk}_{\mathbf{w}}, \mathsf{hk}_C, \mathsf{sd}, \sigma_n, i\mathcal{O}(1^\lambda, \mathsf{RefGen}[\mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, K_\lambda, K_{\mathsf{bit}}, h_{\mathsf{sd}}]))$.

Constants: Hash keys $\mathsf{hk}_{\mathbf{w}}, \mathsf{hk}_C$, hash h_C, puncturable PRF keys $K_\lambda, K_{\mathsf{bit}}$, hash h_{sd}
Inputs: Hash $h_{\mathbf{w}_{[i-1]}}$, string σ_{i-1}, wire index i, gate description $C_i = (g_i, i_0, i_1)$, local opening $\pi_i^{(C)}$, wire value w_{i_b} and $\pi_{i_b}^{(\mathbf{w})}$ for $b \in \{0,1\}$.

1. Verify inputs are well formed. Abort with $\perp$ if any checks do not pass.
   (a) Check $i \in [n+1, \ell]$.
   (b) Check $\Pi_{\mathsf{PA}}.\mathsf{Verify}(\mathsf{hk}_C, h_C, i, C_i, \pi_i^{(C)}) = 1$.
   (c) For $b \in \{0,1\}$, check $\Pi_{\mathsf{PA}}.\mathsf{Verify}(\mathsf{hk}_{\mathbf{w}}, h_{\mathbf{w}_{[i-1]}}, i_b, \pi_{i_b}^{(\mathbf{w})}) = w_{i_b}$.
   (d) Check σ_{i-1} is well formed:
      – Compute $(y_{i-1}, r'_{i-1}) \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i-1)$ (parse as 2 strings $\in \{0,1\}^\lambda$).
      – Compute $K'_{i-1} \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{|h|}, 1^\lambda; r'_{i-1})$
      – Check that $y_{i-1} = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K'_{i-1}, h_{\mathbf{w}_{[i-1]}}) \oplus \sigma_{i-1}$.
2. Set $w_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i)$.
3. Compute $h_{\mathbf{w}_{[i]}} = \Pi_{\mathsf{PA}}.\mathsf{Append}(\mathsf{hk}_{\mathbf{w}}, h_{\mathbf{w}_{[i-1]}}, w_i)$
   – If $i - 1 = n$, check $h_{\mathbf{w}_{[i-1]}} = h_{\mathsf{sd}}$.
4. Compute σ_i as:
   – Compute $(y_i, r'_i) \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i-1)$.
   – Compute $K'_i \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{|h|}, 1^\lambda; r'_i)$
   – Set $\sigma_i = y_{i-1} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K'_{i-1}, h_{\mathbf{w}_{[i-1]}})$.
5. Output (w_i, σ_i)

Figure 8: Program $\mathsf{RefGen}[\mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, K_\lambda, K_{\mathsf{bit}}, h_{\mathsf{sd}}]$.

- $\mathsf{EmbedGen}(1^\lambda, 1^n, C, x) \to (\alpha^*, \mathsf{td})$: On input security parameter λ, input length n, circuit C of size ℓ and input $x = x_1 \dots x_n$, the EmbedGen algorithm does as follows:

  1. Sample the following hash and PRF keys:

(a) $\mathsf{hk}_{\mathbf{w}} \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Setup}(1^\lambda, 1, 1^2)$ is a positional accumulator key on one bit blocks which can bind on 2 positions.

(b) $\mathsf{hk}_C \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Setup}(1^\lambda, 1^{1+2\log \ell})$ is a positional accumulator key on blocks of the form $C_i = g_i \times [\ell] \times [\ell]$.
- Set $h_C \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_C, C)$.

(c) $K_{\mathsf{bit}} \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{\log \ell}, 1)$ is a puncturable PRF key with domain $[\ell]$ and one bit outputs.

(d) $K_\lambda \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{\log \ell}, 1^{2\lambda})$ is a puncturable PRF key with domain $[\ell]$ and outputs of the form $\{0,1\}^\lambda \times \{0,1\}^\lambda$.

2. For $i \in [n]$, set $w_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i) \oplus x_i$. Let $\mathsf{sd} = w_1 w_2 \ldots w_n$.
   (a) Set $h_{\mathsf{sd}} \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_{\mathbf{w}}, \mathsf{sd})$.
   (b) Compute $(y_n, r'_n) \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, n)$.
   (c) Compute $K'_n \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{|h|}, 1^\lambda; r'_n)$.
   (d) Set $\sigma_n = y_n \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K'_n, h_{\mathsf{sd}})$.
3. Output $\alpha^* = (\mathsf{hk}_{\mathbf{w}}, \mathsf{hk}_C, \mathsf{sd}, \sigma_n, i\mathcal{O}(1^\lambda, \mathsf{RefGen}_{\mathsf{embed}}[\mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, K_\lambda, K_{\mathsf{bit}}, h_{\mathsf{sd}}]))$ and $\mathsf{td} = K_{\mathsf{bit}}$.

Constants: Hash keys $\mathsf{hk}_{\mathbf{w}}, \mathsf{hk}_C$, hash h_C, puncturable PRF keys $K_\lambda, K_{\mathsf{bit}}$, hash h_{sd}.
Inputs: Hash $h_{\mathbf{w}_{[i-1]}}$, string σ_{i-1}, wire index i, gate description $C_i = (g_i, i_0, i_1)$, local opening $\pi_i^{(C)}$, wire value w_{i_b} and $\pi_{i_b}^{(\mathbf{w})}$ for $b \in \{0,1\}$.

1. Verify inputs are well formed. Abort with $\perp$ if any checks do not pass.
   (a) Check $i \in [n+1, \ell]$.
   (b) Check $\Pi_{\mathsf{PA}}.\mathsf{Verify}(\mathsf{hk}_C, h_C, i, C_i, \pi_i^{(C)}) = 1$.
   (c) For $b \in \{0,1\}$, check $\Pi_{\mathsf{PA}}.\mathsf{Verify}(\mathsf{hk}_{\mathbf{w}}, h_{\mathbf{w}_{[i-1]}}, i_b, \pi_{i_b}^{(\mathbf{w})}) = w_{i_b}$.
   (d) Check σ_{i-1} is well formed:
   - Compute $(y_{i-1}, r'_{i-1}) \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i-1)$.
   - Compute $K'_{i-1} \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{|h|}, 1^\lambda; r'_{i-1})$
   - Check that $y_{i-1} = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K'_{i-1}, h_{\mathbf{w}_{[i-1]}}) \oplus \sigma_{i-1}$.
2. Set $w_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i) \oplus g_i(w_{i_0} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i_0), w_{i_1} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i_1))$.
3. Compute $h_{\mathbf{w}_{[i]}} = \Pi_{\mathsf{PA}}.\mathsf{Append}(\mathsf{hk}_{\mathbf{w}}, h_{\mathbf{w}_{[i-1]}}, w_i)$
   - If $i - 1 = n$, check $h_{\mathbf{w}_{[i-1]}} = h_{\mathsf{sd}}$.
4. Compute σ_i as:
   - Compute $(y_i, r'_i) \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i-1)$.
   - Compute $K'_i \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{|h|}, 1^\lambda; r'_i)$
   - Set $\sigma_i = y_{i-1} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K'_{i-1}, h_{\mathbf{w}_{[i-1]}})$.
5. Output (w_i, σ_i)

Figure 9: Program $\mathsf{RefGen}_{\mathsf{embed}}[\mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, K_\lambda, K_{\mathsf{bit}}, h_{\mathsf{sd}}]$.

- $\mathsf{Expand}(C, \alpha) \to \mathbf{w}$: On input circuit C, and seed α, the Expand algorithm does as follows

1. Parse seed $\alpha = (\mathsf{hk}_{\mathbf{w}}, \mathsf{hk}_C, \mathsf{sd}, \sigma_n, \tilde{P})$.
2. Parse circuit C as a sequence of gate descriptions C_i for $i \in [n+1, \ell]$ [4] .
3. Let $\mathbf{w}_{[i]}$ denote the first i elements of $\mathbf{w}$, where for we set the first n elements to that of sd as $w_1, \ldots w_n = w_1, \ldots w_n$.

[4]We index $i \in [n]$ as input gates

4. Compute hash $h_{\mathbf{w}_{[n]}} = \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_{\mathbf{w}}, \mathsf{sd})$.
5. For $i = [n+1, \ell]$, iteratively compute w_i as:
   (a) Parse $C_i = (g_i, i_0, i_1)$.
   (b) Compute hash $h_{\mathbf{w}_{[i-1]}} = \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_{\mathbf{w}}, \mathbf{w}_{[i-1]})$
   (c) Compute local opening $\pi_i^{(C)} \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Open}(\mathsf{hk}_C, C, i)$.
   (d) For $b \in \{0,1\}$, compute local openings $\pi_{i_b}^{(\mathsf{sd})} \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Open}(\mathsf{hk}_C, \mathbf{w}_{[i-1]}, i_b)$.
   (e) Evaluate $(w_i, \sigma_i) \leftarrow \tilde{P}(h_{\mathbf{w}_{[i-1]}}, \sigma_{i-1}, i, C_i, \pi_i^{(C)}, \{(i_b, w_{i_b}, \pi_{i_b}^{(\mathsf{sd})})\}_{b\in\{0,1\}})$.
6. Output string $\mathbf{w}$.

- $\mathsf{Extract}(\mathsf{td}, i, b) \to \{0,1\}$: On input trapdoor td, index i and bit b, the Extract algorithm does as follows:
  1. Parse $\mathsf{td} = K_{\mathsf{bit}}$.
  2. Output $\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i) \oplus b$.

5.1 Correctness and Succinctness

Theorem 5.2 (Statistical Extraction). *Suppose $i\mathcal{O}$ is a functionality preserving indistinguishability obfuscator Π_{PA} is a correct positional accumulator, then Construction 5.1 satisfies statistical extraction.*

Proof. Consider any circuit $C : \{0,1\}^n \to \{0,1\}$ and input $x \in \{0,1\}^n$. Let $\alpha^* = (\mathsf{hk}_{\mathsf{sd}}, \mathsf{hk}_C, \mathsf{sd}, \sigma_n, \tilde{P}), \mathsf{td} = K_{\mathsf{bit}}$ be some output of $\mathsf{EmbedGen}(1^\lambda, 1^n, C, x)$. Define the string $\mathbf{w}$ as in program $\mathsf{RefGen}_{\mathsf{embed}}$ (Fig. 9), where

$$w_i = \begin{cases} F_{K_{\mathsf{bit}}}(i) \oplus x_i & \text{for } i \in [n] \\ F_{K_{\mathsf{bit}}}(i) \oplus g_i(w_{i_0} \oplus F_{K_{\mathsf{bit}}}(i_0), w_{i_1} \oplus F_{K_{\mathsf{bit}}}(i_1)) & \text{for } i \in [n+1, \ell] \end{cases}$$

where $F_{K_{\mathsf{bit}}}(j) \coloneqq \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, j)$ and $C_i = (g_i, i_0, i_1)$ for $i \in [n+1, \ell]$.

We will first observe that this is in fact the string output by $\mathsf{Expand}(C, \alpha^*)$. First, we observe that for $i \in [n]$, this is true by definition, as $w_i = w_i$, which is set to $\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i) \oplus x_i$ by construction. Next, we can see for $i \in [n+1, \ell]$, this is indeed what is output in the final step of $\mathsf{RefGen}_{\mathsf{embed}}$, so this is true so long as $\tilde{P}$ does not abort in prior steps.

- Check 1a passes by construction, as only $i \in [n+1, \ell]$ is passed into program $\tilde{P}$.
- Checks 1b and 1c follows from correctness of Π_{PA}.
- We can inductively argue check 1d passes, as σ_{i-1} is simply output by the previous invocation to $\tilde{P}$, and is computed in the same way as it is checked.
- Check 3 passes as both EmbedGen and Expand compute hash $h_{\mathsf{sd}} = h_{\mathbf{w}_{[n]}} = \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_{\mathsf{sd}}, \mathsf{sd})$.

Taking these together with the functionality preservation of $i\mathcal{O}$, we get that $\mathbf{w}$ is indeed output as above. Statistical extractability now follows from a simple inductive argument. For gates $i \in [n]$, since $w_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i) \oplus x_i$, we can see since Extract simply xor's $\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i)$, it returns x_i. For $i \in [n+1, \ell]$, using our inductive hypothesis on w_{i_0}, w_{i_1}, we can see that $g_i(w_{i_0} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i_0), w_{i_1} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i_1))$ is exactly the wire value of $C(x)[i]$. This is also equal to $w_i \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i)$ return by Extract. □

Theorem 5.3 (Succinctness). *If Π_{PA} is succinct then Construction 5.1 is a $n + \mathsf{poly}(\lambda, \log|C|)$-succinct circuit evaluation embedding.*

Proof. Recall seed α consists of components $\mathsf{hk}_{\mathsf{sd}}, \mathsf{hk}_C, \mathsf{sd}, \sigma_n, \tilde{P}$, where program $\tilde{P}$ is an obfuscation of program RefGen (Fig. 8).

- Components $\mathsf{hk}_{\mathsf{sd}}, \mathsf{hk}_C, \sigma_n$ are all of size $\mathsf{poly}(\lambda, \log|C|)$ simply by efficiency of their setup algorithms and by definition respectively.

- String sd is an n bit string by construction.
- Finally, we can observe that of the constants and inputs of RefGen, all hash keys, hashes, and local openings are $\mathsf{poly}(\lambda, \log|C|)$ by succinctness of Π_{PA}, punctured PRF keys $K_\lambda, K_{\mathsf{bit}}$ are $\mathsf{poly}(\lambda, \log|C|)$ by efficiency of $\Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}$, and inputs $\sigma_{i-1}, C_i, i, (i_b, w_{i_b})$ are $\mathsf{poly}(\lambda, \log|C|)$ by construction. Thus, program RefGen is of size $\mathsf{poly}(\lambda, \log|C|)$, as is its obfuscation. We observe in Section 5.2 (in particular Lemma 5.12) that the size of programs in the security proof also never exceed $\mathsf{poly}(\lambda, \log|C|)$, bounding the amount of padding we need to give our obfuscation to argue security.

Taking these together, our total seed size can be bound by $n + \mathsf{poly}(\lambda, \log|C|)$. □

5.2 Security

Theorem 5.4 (Mode Indistinguishability)**.** *Suppose Π_{PPRF} is a secure and perfectly correct puncturable PRF, $i\mathcal{O}$ is a secure indistinguishability obfuscator Π_{PA} is a setup enforcing indistinguishable and read and write enforcing positional accumulator, then Construction 5.1 satisfies mode indistinguishability.*

Proof. To show mode indistinguishability, we define a sequence of experiments, beginning with $\mathsf{Exp}_{\mathsf{real}}$, which is the real mode indistinguishability game when $b = 0$ - i.e. using Gen, and ending in $\mathsf{Exp}_{\mathsf{embed}}$, the mode indistinguishability game when $b = 1$ or using EmbedGen. We will argue indistinguishability of this sequence of experiments.

At a high level, our proof proceeds by leveraging the puncturability of PRF K_{bit} to incrementally switch the bits of $\mathbf{w}$ from a simple PRF evaluation to an embedding of the evaluation of our circuit. Since the PRF evaluation in Gen on a bit i is independent of the particular value of input wire bits $\{w_{i_b}\}$ while EmbedGen is not, the challenge now is to move to an experiment where input wire bits can only statistically verify to one value. This effectively makes our program 'input independent', allowing us to leverage the classic $i\mathcal{O}$ in conjunction with puncturable PRF to hard code the output bit w_i. To accomplish this statistical input binding, we need the following to be true:

- The hard coded hash h_C is binding on index i, statistically binding to the tuple (g_i, i_0, i_1).
- The input hash $h_{\mathbf{w}_{[i-1]}}$ is statistically binding on indices i_0, i_1. Moreover, these bits are open to wire values w_{i_0}, w_{i_1} respectively.

For both requirements, the binding indices are easy to guarantee via simple mode indistinguishability of our hash. However, because hash $h_{\mathbf{w}_{[i-1]}}$ is taken as *input*, we also need to rule out the ability for an adversary to generate a dishonest input hash h which opens i_0, i_1 to different bits. We achieve this by (implicitly) constructing a simplified version of splittable signatures [KLW15], allowing us to replace our verification process for σ_{i-1} with one that only works on a fixed hash value h_{i-1}. Unfortunately, unlike mode indistinguishability in our hash function, the verification of σ_{i-1} is naturally only indistinguishable when our program is *not* able to generate signatures for any other hash $h'_{i-1} \neq h_{i-1}$. This in turn means that to argue such indistinguishability, our signature needs to be 'split' on index $i-1$ as well, hard coding hash h_{i-2}. Argued naively, this blows up the obfuscated program in our experiments to be of size $O(\ell)$, meaning we would require $O(\ell)$ padding in our original obfuscations of RefGen, $\mathsf{RefGen}_{\mathsf{embed}}$ to attempt to argue security. Fortunately, we can employ a combinatorial 'pebbling argument' [GPSZ17] where we can bound the number of hard coded hash locations at $\mathsf{poly}(\log \ell)$, bounding the total obfuscation padding required similarly.

- $\mathsf{Exp}_{\mathsf{real}}$: In the initial experiment, the challenger runs Gen, where the string sd and program RefGen (Fig. 8) are derived independently of the challenge input x.
  0. The adversary $\mathcal{A}(1^\lambda)$ sends a circuit C and input x to the challenger.
  1. The challenger samples the following keys:
     (a) $\mathsf{hk}_{\mathbf{w}} \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Setup}(1^\lambda, 1, 1^2)$ is a positional accumulator key on one bit blocks which can bind on 2 positions.
     (b) $\mathsf{hk}_C \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Setup}(1^\lambda, 1^{1+2\log \ell})$ is a positional accumulator key on blocks of the form $C_i = g_i \times [\ell] \times [\ell]$.
     – Set $h_C \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_C, C)$.

(c) $K_{\mathsf{bit}} \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{\log \ell}, 1)$ is a puncturable PRF key with domain $[\ell]$ and one bit outputs.

(d) $K_\lambda \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{\log \ell}, 1^{2\lambda})$ is a puncturable PRF key with domain $[\ell]$ and outputs of the form $\{0,1\}^\lambda \times \{0,1\}^\lambda$.

2. For $i \in [n]$, set $w_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i)$. Let $\mathsf{sd} = w_1 w_2 \ldots w_n$.
   (a) Set $h_{\mathsf{sd}} \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_{\mathbf{w}}, w)$.
   (b) Compute $(y_n, r'_n) \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, n)$.
   (c) Compute $K'_n \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{|h|}, 1^\lambda; r'_n)$.
   (d) Set $\sigma_n = y_n \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K'_n, h_{\mathsf{sd}})$.
3. The challenger returns $\alpha = (\mathsf{hk}_{\mathbf{w}}, \mathsf{hk}_C, \mathsf{sd}, \sigma_n, i\mathcal{O}(1^\lambda, \mathsf{RefGen}[\mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, K_\lambda, K_{\mathsf{bit}}, h_{\mathsf{sd}}]))$.
4. The adversary $\mathcal{A}$ returns a bit b' which is the output of the experiment.

- Exp_{i^*} for $i^* \in [n]$: Same as $\mathsf{Exp}_{\mathsf{real}}$ but instead for $i \in [i^*]$, set $w_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i) \oplus x_i$.

2. For $i \in [n]$, set w_i as:
   (a) If $i \le i^*$, set $w_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i) \oplus x_i$.
   (b) Otherwise, set $w_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i)$.

- Exp_S for $S \subseteq [n, \ell]$: Same as Exp_n but add explicit hard-coded checks for the input hash value for indices in S. In addition, we extend our program (Fig. 10) to 'secretly' evaluate w_i on the first $\max(S)$ gates (when $S = \varnothing$ we can treat $\max(S) = n$).

2. For $i \in [n]$ set $w_i = w_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i) \oplus x_i$.
   (a) For $i \in [n+1, \max(S)]$, compute extended string as follows:

$$w_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i) \oplus g_i(w_{i_0} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i_0), w_{i_1} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i_1)).$$

   (b) For $i \in [\max(S)+1, \ell]$, compute $w_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i)$.
   (c) For $i \in [\ell]$, let $\mathbf{w}_{[i]}$ denote the prefix $w_1, \ldots w_i$ of $\mathbf{w}$. Let $h_i \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_{\mathbf{w}}, \mathbf{w}_{[i]})$
3. The challenger returns $\alpha = (\mathsf{hk}_{\mathbf{w}}, \mathsf{hk}_C, C, x, \mathsf{sd}, \sigma_n, i\mathcal{O}(1^\lambda, \mathsf{RefGen}'[\mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, K_\lambda, K_{\mathsf{bit}}, \{h_j\}_{j \in S}]))$.

Constants: Hash keys $\mathsf{hk}_{\mathbf{w}}, \mathsf{hk}_C$, hash h_C, puncturable PRF keys $K_\lambda, K_{\mathsf{bit}}$, hashes h_j for $j \in S$.
Inputs: Hash $h_{\mathbf{w}_{[i-1]}}$, string σ_{i-1}, wire index i, gate description $C_i = (g_i, i_0, i_1)$, local opening $\pi_i^{(C)}$, wire value w_{i_b} and $\pi_{i_b}^{(\mathbf{w})}$ for $b \in \{0,1\}$.

1. Verify inputs are well formed. Abort with $\perp$ if any checks do not pass.
   (a) Check $i \in [n+1, \ell]$.
   (b) Check $\Pi_{\mathsf{PA}}.\mathsf{Verify}(\mathsf{hk}_C, h_C, i, C_i, \pi_i^{(C)}) = 1$.
   (c) For $b \in \{0,1\}$, check $\Pi_{\mathsf{PA}}.\mathsf{Verify}(\mathsf{hk}_{\mathbf{w}}, h_{\mathbf{w}_{[i-1]}}, i_b, \pi_{i_b}^{(\mathbf{w})}) = w_{i_b}$.
   (d) Check σ_{i-1} is well formed:
      - Compute $(y_{i-1}, r'_{i-1}) \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i-1)$.
      - Compute $K'_{i-1} \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{|h|}, 1^\lambda; r'_{i-1})$
      - Check that $y_{i-1} = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K'_{i-1}, h_{\mathbf{w}_{[i-1]}}) \oplus \sigma_{i-1}$.
2. If $i < \max(S)$ set $w_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i) \oplus g_i(w_{i_0} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i_0), w_{i_1} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i_1))$. Otherwise, set $w_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i)$.
3. Compute $h_{\mathbf{w}_{[i]}} = \Pi_{\mathsf{PA}}.\mathsf{Append}(\mathsf{hk}_{\mathbf{w}}, h_{\mathbf{w}_{[i-1]}}, w_i)$
   - If $i-1 \in S$, check $h_{\mathbf{w}_{[i-1]}} = h_{i-1}$. Abort with $\perp$ otherwise.
4. Compute σ_i as:
   - Compute $(y_i, r'_i) \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, i-1)$.
   - Compute $K'_i \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{|h|}, 1^\lambda; r'_i)$
   - Set $\sigma_i = y_{i-1} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K'_{i-1}, h_{\mathbf{w}_{[i-1]}})$.
5. Output (w_i, σ_i)

Figure 10: Program $\mathsf{RefGen}'[\mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, K_\lambda, K_{\mathsf{bit}}, \{h_j\}]$.

- $\mathsf{Exp}_{\mathsf{embed}}$: In this experiment, the challenger runs EmbedGen, where sd encodes the challenge input x and program $\mathsf{RefGen}_{\mathsf{embed}}$ (Fig. 9) evaluates circuit C.

0. The adversary $\mathcal{A}(1^\lambda)$ sends a circuit C and input x to the challenger.
1. The challenger samples the following keys:
   (a) $\mathsf{hk}_{\mathbf{w}} \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Setup}(1^\lambda, 1, 1^2)$ is a positional accumulator key on one bit blocks which can bind on 2 positions.
   (b) $\mathsf{hk}_C \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Setup}(1^\lambda, 1^{1+2\log \ell})$ is a positional accumulator key on blocks of the form $C_i = g_i \times [\ell] \times [\ell]$.
      - Set $h_C \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_C, C)$.
   (c) $K_{\mathsf{bit}} \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{\log \ell}, 1)$ is a puncturable PRF key with domain $[\ell]$ and one bit outputs.
   (d) $K_\lambda \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{\log \ell}, 1^{2\lambda})$ is a puncturable PRF key with domain $[\ell]$ and outputs of the form $\{0,1\}^\lambda \times \{0,1\}^\lambda$.
2. For $i \in [n]$, set $w_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i) \oplus x_i$. Let $\mathsf{sd} = w_1 w_2 \ldots w_n$.
   (a) Set $h_{\mathsf{sd}} \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_{\mathbf{w}}, w)$.
   (b) Compute $(y_n, r'_n) \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, n)$.
   (c) Compute $K'_n \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{|h|}, 1^\lambda; r'_n)$.
   (d) Set $\sigma_n = y_n \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K'_n, h_{\mathsf{sd}})$.
3. The challenger returns $\alpha = (\mathsf{hk}_{\mathbf{w}}, \mathsf{hk}_C, \mathsf{sd}, \sigma_n, i\mathcal{O}(1^\lambda, \mathsf{RefGen}_{\mathsf{embed}}[\mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, K_\lambda, K_{\mathsf{bit}}, h_{\mathsf{sd}}]))$.
4. The adversary $\mathcal{A}$ returns a bit b' which is the output of the experiment.

Lemma 5.5. *For any algorithm $\mathcal{A}$:*

$$\Pr[\mathsf{Exp}_0^{(C,x)}(\mathcal{A})] = \Pr[\mathsf{Exp}_{\mathsf{real}}^{(C,x)}(\mathcal{A})].$$

Proof. Since $i \in [n]$ is always > 0, these experiments are identical. □

Lemma 5.6. *For any efficient algorithm $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that for all indices $i^* \in [n]$:*

$$\left|\Pr[\mathsf{Exp}_{i^*-1}(\mathcal{A})] - \Pr[\mathsf{Exp}_{i^*}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

We will show this indistinguishability via a series of subexperiments in Section 5.3.

Lemma 5.7. *Suppose $i\mathcal{O}$ is a secure indistinguishability obfuscator. For any efficient algorithm $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that for $S = \{n\}$:*

$$\left|\Pr[\mathsf{Exp}_n(\mathcal{A})] - \Pr[\mathsf{Exp}_S(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda)$$

Proof.

Claim 5.8. *Program* RefGen *(Fig. 8) in* Exp_n *and* RefGen′ *(Fig. 10) in* $\mathsf{Exp}_{\{n\}}$ *are functionally equivalent.*

Proof. Since RefGen′ only has hard codes hash $h_n = h_{\mathsf{sd}}$ when $S = \{n\}$, and $i \in [n+1, \ell]$, the checks these programs make are identical, and thus are functionally equivalent. □

Once we have established functional equivalence of these two programs, the proof of our lemma follows from a reduction $\mathcal{B}$ to the security of our $i\mathcal{O}$ scheme.

0. Reduction $\mathcal{B}$ runs $\mathcal{A}$, playing the role of the challenger per Exp_n, and receives circuit C and input x.
1. 2. $\mathcal{B}$ samples hash/PRF keys and sd as per Exp_n.
3. $\mathcal{B}$ submits programs $\mathsf{RefGen}[\mathsf{hk}_C, \mathsf{hk}_{\mathsf{sd}}, \mathsf{hk}_C, K_\lambda, K_{\mathsf{bit}}, h_{\mathsf{sd}}]$ and $\mathsf{RefGen}'[\mathsf{hk}_C, \mathsf{hk}_{\mathsf{sd}}, \mathsf{hk}_C, K_\lambda, K_{\mathsf{bit}}, \{h_{\mathsf{sd}}\}]$ to the $i\mathcal{O}$ challenger, and receives obfuscation $\tilde{P}$.
4. $\mathcal{B}$ returns seed $\alpha = (\mathsf{hk}_{\mathsf{sd}}, \mathsf{hk}_C, \mathsf{sd}, \sigma_n, \tilde{P})$ to $\mathcal{A}$.
5. The adversary returns a bit b', which $\mathcal{B}$ outputs as well.

Observe that since $\max S = n < n+1$, these experiments are identical except for the use programs RefGen or RefGen′. Thus, the advantage of $\mathcal{B}$ in distinguishing $\tilde{P}$ is exactly the distinguishing advantage of $\mathcal{A}$ between Exp_n and $\mathsf{Exp}_{\{n\}}$. □

Lemma 5.9. *For any efficient algorithm $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that for all sets $S \subseteq [n, \ell]$, $j^* \in S$:*

$$\left|\Pr[\mathsf{Exp}_S(\mathcal{A})] - \Pr[\mathsf{Exp}_{S\cup\{j^*+1\}}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda)$$

We will show this indistinguishability via a series of subexperiments in Section 5.4.

Lemma 5.10. *Suppose $i\mathcal{O}$ is a secure indistinguishability obfuscator. For any efficient algorithm $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that:*

$$\left|\Pr[\mathsf{Exp}_{\{n,\ell\}}(\mathcal{A})] - \Pr[\mathsf{Exp}_{\mathsf{embed}}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda)$$

Proof.

Claim 5.11. *Program* RefGen′ *(Fig. 10) in* $\mathsf{Exp}_{\{n,\ell\}}$ *and* $\mathsf{RefGen}_{\mathsf{embed}}$ *(Fig. 9)in* $\mathsf{Exp}_{\mathsf{embed}}$ *are functionally equivalent.*

Proof. Since $\max\{n, \ell\} = \ell$, by experiment $\mathsf{Exp}_{\{n,\ell\}}$, program RefGen' evaluates w_i with the embedded circuit for all $i \leq [\ell]$. Since these are the only i for which RefGen' doesn't abort in Step 1a, this program is functionally equivalent to $\mathsf{RefGen}_{\mathsf{embed}}$ used in $\mathsf{Exp}_{\mathsf{embed}}$. □

Once Claim 5.11 is established, this follows from a reduction to $i\mathcal{O}$ security analogous to Lemma 5.7.

□

Lemma 5.12. *There exists a polynomial sequence of sets $S_1, \ldots S_N$ where $S_1 = \{n\}$, $S_N = \{n, \ell\}$ and for all $i \in [2, N]$, $\exists j \in S_{i-1}$ such that $S_i = S_{i-1} \cup \{j+1\}$ or $S_i = S_{i-1} \backslash \{j+1\}$. Furthermore, the maximum size of any individual set $\max_{i \in [N]} |S_i|$ is $O(\log(\ell - n))$.*

Proof. This follows as an application the pebbling lemmas of [GPSZ17]. □

Using Lemma 5.12, we observe that we can have a sequence of experiments where we can bound the size of any program obfuscated in our proof with $\mathsf{poly}(\lambda, \log(\ell - n)) = \mathsf{poly}(\lambda, \log |C|)$. This bounds the amount of padding we need to obfuscate our program with in the real scheme with similarly. Taking together Lemmas 5.5 to 5.7, 5.9 and 5.10, we get the proof of our theorem. □

5.3 Input Subexperiments

In this subsection, we prove Lemma 5.6 deferred from the previous section.

Proof. To show this, we will define a sequence of subexperiments:

- $\mathsf{sExp}_0^{(i^*)}$: This experiment proceeds as Exp_{i^*}, but we puncture the key K_{bit} on i^*.

  1. The challenger samples the following keys:
     (c) $K_{\mathsf{bit}} \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{\log \ell}, 1)$. Then sample $K_{\mathsf{bit}}^{(i^*)} \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Puncture}(K_{\mathsf{bit}}, i^*)$.
  3. The challenger returns $s = (\mathsf{hk}_{\mathbf{w}}, \mathsf{hk}_C, \mathsf{sd}, \sigma_n, i\mathcal{O}(1^\lambda, \mathsf{RefGen}[\mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, K_\lambda, K_{\mathsf{bit}}^{(i^*)}]))$

- $\mathsf{sExp}_1^{(i^*)}$: In this experiment, we sample w_{i^*} uniformly at random.

  2. For $i \in [n]$, set w_i as:
     (c) Sample $w_{i^*} \xleftarrow{R} \{0,1\}$.

- $\mathsf{sExp}_2^{(i^*)}$: In this experiment, we sample $w_{i^*} = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i^*) \oplus x_i$.

  2. For $i \in [n]$, set w_i as:
     (a) If $i \leq i^*$, set $w_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i) \oplus x_i$.
     (b) Otherwise, set $w_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i)$.

Claim 5.13. *Suppose Π_{PPRF} is a perfectly correct puncturable PRF and $i\mathcal{O}$ is a secure indistinguishability obfuscator. Then for any efficient algorithm $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that for all indices $i^* \in [n]$:*

$$\left|\Pr[\mathsf{Exp}_{i^*-1}(\mathcal{A})] - \Pr[\mathsf{sExp}_0^{(i^*)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

Proof.

Claim 5.14. *Program* RefGen *(Fig. 8) in Exp_{i^*-1} and $\mathsf{sExp}^{(i^*)}$ are functionally equivalent.*

Proof. Observe these program only differ in the use of a punctured version of key K_{bit} in latter experiment. Since program RefGen aborts on any $i < n+1$, $K_{\mathsf{bit}}^{(i^*)}$ is never evaluated on such i^* and by perfect correctness of PPRF, these programs are functionally equivalent. □

Once Claim 5.14 is established, this follows from a reduction to $i\mathcal{O}$ security analogous to Lemma 5.7.

□

Claim 5.15. *Suppose Π_{PPRF} is pseudorandom. Then for any efficient algorithm $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that for all indices $i^* \in [n]$:*

$$\left|\Pr[\mathsf{sExp}_0^{(i^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_1^{(i^*)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

Proof. Suppose that there exists an efficient adversary $\mathcal{A}$ that can distinguish these two experiments with non-negligible probability ε. We use $\mathcal{A}$ to construct an adversary $\mathcal{B}$ that breaks security of PPRF with the same advantage:

0. Reduction $\mathcal{B}$ runs $\mathcal{A}$, playing the role of the challenger as per $\mathsf{sExp}_0^{(i^*)}$. $\mathcal{B}$ samples everything honestly except when specified.

1. Reduction $\mathcal{B}$ samples the following keys:

   (c) $\mathcal{B}$ submits input length $1^{\log \ell}$, output length 1, and puncture point i^* to the challenger, receiving punctured key $K_{\mathsf{bit}}^{(i^*)}$ and challenge bit y_{i^*} to $\mathcal{B}$.

2. For $i \in [n]$, $\mathcal{B}$ sets w_i as:

   (c) Set w_{i^*} to challenge bit y_{i^*}.

Observe that when challenge bit y_{i^*} is the PRF output, this is exactly $\mathsf{sExp}_0^{(i^*)}$, and when y_{i^*} is uniformly random, this simulates $\mathsf{sExp}_1^{(i^*)}$, so the distinguishing advantage of $\mathcal{B}$ is equal to that of $\mathcal{A}$.

□

Claim 5.16. *Suppose Π_{PPRF} is pseudorandom. Then for any efficient algorithm $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that for all indices $i^* \in [n]$:*

$$\left|\Pr[\mathsf{sExp}_1^{(i^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_2^{(i^*)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

This claim follows by an analogous argument to Claim 5.15.

Claim 5.17. *Suppose Π_{PPRF} is a perfectly correct puncturable PRF and $i\mathcal{O}$ is a secure indistinguishability obfuscator. Then for any efficient algorithm $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that for all indices $i^* \in [n]$:*

$$\left|\Pr[\mathsf{sExp}_2^{(i^*)}(\mathcal{A})] - \Pr[\mathsf{Exp}_{i^*}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

This claim follows by an analogous argument to Claim 5.13.

Taking together Claims 5.13 and 5.15 to 5.17, we have the proof of Lemma 5.6.

□

5.4 Gate Subexperiments

In this section, we prove Lemma 5.9 deferred from the previous section.

Proof. To show this, we will define a sequence of subexperiments:

- $\mathsf{sExp}_0^{(S,j^*)}$: This experiment proceeds as Exp_S, but we switch hash key hk_C to be binding on j.

  1. The challenger samples the following keys:

     (b) Sample $\mathsf{hk}_C \leftarrow \mathsf{SetupEnforce}(1^\lambda, 1^{1+2\log \ell}, C, \{j^*\})$.

     – Set $h_C \leftarrow \Pi_{\mathsf{PA}}.\mathsf{Hash}(\mathsf{hk}_C, C)$.

- $\mathsf{sExp}_1^{(S,j^*)}$: In this experiment, we switch $\mathsf{hk}_\mathbf{w}$ to be binding on j_0, j_1 (where gate $G_{j^*} \in C$ is of the form (g_{j^*}, j_0, j_1)).

  1. The challenger samples the following keys:
     (a) Sample $\mathsf{hk}_\mathbf{w} \leftarrow \mathsf{SetupEnforce}(1^\lambda, 1, w_{[j^*]}, \{j_0, j_1\})$.[5]

- $\mathsf{sExp}_2^{(S,j^*)}$: This experiment proceeds as $\mathsf{sExp}_1^{(S,j^*)}$, but we generate a punctured key K_λ on j^*+1 and hard code the corresponding outputs into program RefGen_2' (Fig. 11).

  1. The challenger samples the following keys:
     (d) $K_\lambda \leftarrow \mathsf{KeyGen}(1^\lambda, 1^{\log \ell}, 1^{2\lambda})$ is a puncturable PRF key with domain $[\ell]$ and 2λ bit outputs.
        i. Compute punctured key $K_\lambda^{(j^*+1)} = \Pi_{\mathsf{PPRF}}.\mathsf{Puncture}(K_\lambda)$.
        ii. Evaluate $y'_{j^*+1}, r'_{j^*+1} \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, j^*+1)$. Set $z'_{j^*+1} \leftarrow F(y'_{j^*+1})$ and $\beta_{j^*+1} = \mathsf{hcb}(y'_{j^*+1})$
        iii. Compute derived key $K'_{j^*+1} \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{|h|}, 1^\lambda; r'_{j^*+1})$.
        iv. Puncture key $K'^{(h)}_{j^*+1} \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Puncture}(K'_{j^*+1}, h_{j^*+1})$.
        v. Set $\sigma'_{j^*+1} = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K'_{j^*+1}, h_{j^*+1}) \oplus y'_{j^*+1}$.

  3. The challenger returns $s = (\mathsf{hk}_\mathbf{w}, \mathsf{hk}_C, \mathsf{sd}, \sigma_n, iO(1^\lambda, \mathsf{RefGen}_2'[\mathsf{hk}_C, \mathsf{hk}_\mathbf{w}, h_C, K_\lambda^{(j^*+1)}, K'^{(h)}_{j^*+1}, K_{\mathsf{bit}}, \{h_i\}, \sigma'_{j^*+1}, z'_{j^*+1}, \beta_{j^*+1}]))$.

[5] Input $w_{[j]}$ is in fact computed later in Step 2 of the experiment. However, it is not dependent on the hash keys. For simplicity, we will keep $\mathsf{hk}_\mathbf{w}$ here.

Constants: Hash keys $\mathsf{hk}_{\mathbf{w}}, \mathsf{hk}_C$, hash h_C, puncturable PRF keys $K_\lambda^{(j^*+1)}, K_{\mathsf{bit}}, K'^{(h)}_{j^*+1}$, hashes h_j for $j \in S$, bitstrings $\sigma'_{j^*+1}, z'_{j^*+1}$ and bit β_{j^*+1}.

Inputs: Hash $h_{w_{[i-1]}}$, string σ_{i-1}, wire index i, gate description $C_i = (g_i, i_0, i_1)$, local opening $\pi_i^{(C)}$, wire value w_{i_b} and $\pi_{i_b}^{(w)}$ for $b \in \{0,1\}$.

1. Verify inputs are well formed. Abort with $\perp$ if any checks do not pass.
   (a) Check $i \in [n+1, \ell]$.
   (b) Check $\Pi_{\mathsf{PA}}.\mathsf{Verify}(\mathsf{hk}_C, h_C, i, C_i, \pi_i^{(C)}) = 1$.
   (c) For $b \in \{0,1\}$, check $\Pi_{\mathsf{PA}}.\mathsf{Verify}(\mathsf{hk}_{\mathbf{w}}, h_{w_{[i-1]}}, i_b, \pi_{i_b}^{(w)}) = w_{i_b}$.
   (d) Check σ_{i-1} is well formed:
   - If $i - 1 = j^* + 1$, check that:
     * If $h_{w_{[i-1]}} = h_{j^*+1}$, check that $\sigma_{i-1} = \sigma'_{j^*+1}$.
     * Otherwise, check that $z'_{j^*+1} = F(\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K'^{(h)}_{j^*+1}, h_{w_{[i-1]}}) \oplus \sigma_{i-1})$ and $\beta_{j^*+1} = \mathsf{hcb}(\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K'^{(h)}_{j^*+1}, h_{w_{[i-1]}}) \oplus \sigma_{i-1})$.
   - Otherwise, compute $(y_{i-1}, r'_{i-1}) \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda^{(j^*+1)}, i-1)$.
   - Compute $K'_{i-1} \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{|h|}, 1^\lambda; r'_{i-1})$.
   - Check that $y_{i-1} = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K'_{i-1}, h_{w_{[i-1]}}) \oplus \sigma_{i-1}$.
2. If $i < \max(S)$ set $w_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i) \oplus g_i(w_{i_0} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i_0), w_{i_1} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i_1))$. Otherwise, set $w_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i)$.
3. Compute $h_{w_{[i]}} = \Pi_{\mathsf{PA}}.\mathsf{Append}(\mathsf{hk}_{\mathbf{w}}, h_{w'[i-1]}, w_i)$
   - If $i - 1 \in S$, check $h_{w_{[i-1]}} = h_{i-1}$. Abort with $\perp$ otherwise.
4. Compute σ_i as:
   - If $i = j^* + 1$, return σ'_{j^*+1}.
   - Otherwise, compute $(y_i, r'_i) \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda^{(j^*+1)}, i-1)$.
   - Compute $K'_i \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{|h|}, 1^\lambda; r'_i)$
   - Set $\sigma_i = y_{i-1} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K'_{i-1}, h_{w'[i-1]})$.
5. Output (w_i, σ_i)

Figure 11: Program $\mathsf{RefGen}'_2[\mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, K_\lambda^{(j^*+1)}, K'^{(h)}_{j^*+1}, K_{\mathsf{bit}}, \{h_j\}, \sigma'_{j^*+1}, z'_{j^*+1}, \beta_{j^*+1}]$.

- $\mathsf{sExp}_3^{(S,j^*)}$: In this experiment, we switch hard coded string y'_{j^*+1} to a uniformly random string.

1. The challenger samples the following keys:
   (d) $K_\lambda \leftarrow \mathsf{KeyGen}(1^\lambda, 1^{\log \ell}, 1^{2\lambda})$ is a puncturable PRF key with domain $[\ell]$ and 2λ bit outputs.
   ii. Sample $y'_{j^*+1}, r'_{j^*+1} \xleftarrow{R} \{0,1\}^\lambda$. Set $z'_{j^*+1} = F(y'_{j^*+1})$ and $\beta_{j^*+1} = \mathsf{hcb}(y'_{j^*+1})$.

- $\mathsf{sExp}_4^{(S,j^*)}$: In this experiment, we switch z'_{j^*+1} to a uniformly random string $\in \{0,1\}^{2\lambda}$.

1. The challenger samples the following keys:
   (d) $K_\lambda \leftarrow \mathsf{KeyGen}(1^\lambda, 1^{\log \ell}, 1^{2\lambda})$ is a puncturable PRF key with domain $[\ell]$ and 2λ bit outputs.
   ii. Sample $r'_{j^*+1} \xleftarrow{R} \{0,1\}^\lambda$. Set $z'_{j^*+1} = F(y'_{j^*+1})$ and $\beta_{j^*+1} = \mathsf{hcb}(y'_{j^*+1}) \oplus 1$.

- $\mathsf{sExp}_5^{(S,j^*)}$: In this experiment, we add the explicit check on index j^*+1 to our program (Fig. 12).

Constants: Hash keys $\mathsf{hk}_{\mathbf{w}}, \mathsf{hk}_C$, hash h_C, puncturable PRF keys $K_\lambda^{(j^*+1)}, K_{\mathsf{bit}}, K'^{(h)}_{j^*+1}$, hashes h_j for $j \in S \cup \{j^*+1\}$, bitstrings $\sigma'_{j^*+1}, z'_{j^*+1}$ and bit β_{j^*+1}.

Inputs: Hash $h_{w_{[i-1]}}$, string σ_{i-1}, wire index i, gate description $C_i = (g_i, i_0, i_1)$, local opening $\pi_i^{(C)}$, wire value w_{i_b} and $\pi_{i_b}^{(w)}$ for $b \in \{0,1\}$.

1. Verify inputs are well formed. Abort with $\perp$ if any checks do not pass.
   (a) Check $i \in [n+1, \ell]$.
   (b) Check $\Pi_{\mathsf{PA}}.\mathsf{Verify}(\mathsf{hk}_C, h_C, i, C_i, \pi_i^{(C)}) = 1$.
   (c) For $b \in \{0,1\}$, check $\Pi_{\mathsf{PA}}.\mathsf{Verify}(\mathsf{hk}_{\mathbf{w}}, h_{w_{[i-1]}}, i_b, \pi_{i_b}^{(w)}) = w_{i_b}$.
   (d) Check σ_{i-1} is well formed:
      - If $i-1 = j^*+1$, check that:
        * If $h_{w_{[i-1]}} = h_{j^*+1}$, check that $\sigma_{i-1} = \sigma'_{j^*+1}$.
        * Otherwise, check that $z'_{j^*+1} = F(\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K'^{(h)}_{j^*+1}, h_{w_{[i-1]}}) \oplus \sigma_{i-1})$ and $\beta_{j^*+1} = \mathsf{hcb}(\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K'^{(h)}_{j^*+1}, h_{w_{[i-1]}}) \oplus \sigma_{i-1})$.
      - Otherwise, compute $(y_{i-1}, r'_{i-1}) \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda^{(j^*+1)}, i-1)$.
      - Compute $K'_{i-1} \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{|h|}, 1^\lambda; r'_{i-1})$
      - Check that $y_{i-1} = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K'_{i-1}, h_{w_{[i-1]}}) \oplus \sigma_{i-1}$.
2. If $i < \max(S)$ set $w_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i) \oplus g_i(w_{i_0} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i_0), w_{i_1} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i_1))$. Otherwise, set $w_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i)$.
3. Compute $h_{w_{[i]}} = \Pi_{\mathsf{PA}}.\mathsf{Append}(\mathsf{hk}_{\mathbf{w}}, h_{w'[i-1]}, w_i)$
   - If $i-1 \in S \cup \{j^*+1\}$, check $h_{w_{[i-1]}} = h_{i-1}$. Abort with $\perp$ otherwise.
4. Compute σ_i as:
   - If $i = j^*+1$, return σ'_{j^*+1}.
   - Otherwise, compute $(y_i, r'_i) \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda^{(j^*+1)}, i-1)$.
   - Compute $K'_i \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{|h|}, 1^\lambda; r'_i)$
   - Set $\sigma_i = y_{i-1} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K'_{i-1}, h_{w'[i-1]})$.
5. Output (w_i, σ_i)

Figure 12: Program $\mathsf{RefGen}'_5[\mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, K_\lambda^{(j^*+1)}, K'^{(h)}_{j^*+1}, K_{\mathsf{bit}}, \{h_j\}, \sigma'_{j^*+1}, z'_{j^*+1}, \beta_{j^*+1}]$.

- $\mathsf{sExp}_6^{(S,j^*)}$: In this experiment, we extend our evaluation encoding in our program (Fig. 13) to $S \cup \{j^*+1\}$.

2. For $i \in [n]$ set $w_i = w_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i) \oplus x_i$.
   (a) For $i \in [n+1, \max(S \cup \{j^*+1\})]$, compute extended string $w_i = g_i(w_{i_0} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i_0), w_{i_1} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i_1))$.
   (b) For $i \in [\max(S \cup \{j^*+1\}) + 1, \ell]$, compute $w_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i)$.

Constants: Hash keys $\mathsf{hk}_{\mathbf{w}}, \mathsf{hk}_C$, hash h_C, puncturable PRF keys $K_\lambda^{(j^*+1)}, K_{\mathsf{bit}}, K'^{(h)}_{j^*+1}$, hashes h_j for $j \in S \cup \{j^*+1\}$, bitstrings $\sigma'_{j^*+1}, z'_{j^*+1}$ and bit β_{j^*+1}.

Inputs: Hash $h_{w_{[i-1]}}$, string σ_{i-1}, wire index i, gate description $C_i = (g_i, i_0, i_1)$, local opening $\pi_i^{(C)}$, wire value w_{i_b} and $\pi_{i_b}^{(w)}$ for $b \in \{0,1\}$.

1. Verify inputs are well formed. Abort with $\perp$ if any checks do not pass.
   (a) Check $i \in [n+1, \ell]$.
   (b) Check $\Pi_{\mathsf{PA}}.\mathsf{Verify}(\mathsf{hk}_C, h_C, i, C_i, \pi_i^{(C)}) = 1$.
   (c) For $b \in \{0,1\}$, check $\Pi_{\mathsf{PA}}.\mathsf{Verify}(\mathsf{hk}_{\mathbf{w}}, h_{w_{[i-1]}}, i_b, \pi_{i_b}^{(w)}) = w_{i_b}$.
   (d) Check σ_{i-1} is well formed:
      - If $i - 1 = j^* + 1$, check that:
        * If $h_{w_{[i-1]}} = h_{j^*+1}$, check that $\sigma_{i-1} = \sigma'_{j^*+1}$.
        * Otherwise, check that $z'_{j^*+1} = F(\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K'^{(h)}_{j^*+1}, h_{w_{[i-1]}}) \oplus \sigma_{i-1})$ and $\beta_{j^*+1} = \mathsf{hcb}(\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K'^{(h)}_{j^*+1}, h_{w_{[i-1]}}) \oplus \sigma_{i-1})$.
      - Otherwise, compute $(y_{i-1}, r'_{i-1}) \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda^{(j^*+1)}, i-1)$.
      - Compute $K'_{i-1} \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{|h|}, 1^\lambda; r'_{i-1})$.
      - Check that $y_{i-1} = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K'_{i-1}, h_{w_{[i-1]}}) \oplus \sigma_{i-1}$.
2. If $i < \max(S \cup \{j^*+1\})$ set $w_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i) \oplus g_i(w_{i_0} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i_0), w_{i_1} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i_1))$. Otherwise, set $w_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i)$.
3. Compute $h_{w_{[i]}} = \Pi_{\mathsf{PA}}.\mathsf{Append}(\mathsf{hk}_{\mathbf{w}}, h_{w'[i-1]}, w_i)$
   - If $i - 1 \in S \cup \{j^*+1\}$, check $h_{w_{[i-1]}} = h_{i-1}$. Abort with $\perp$ otherwise.
4. Compute σ_i as:
   - If $i = j^* + 1$, return σ'_{j^*+1}.
   - Otherwise, compute $(y_i, r'_i) \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda^{(j^*+1)}, i-1)$.
   - Compute $K'_i \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{|h|}, 1^\lambda; r'_i)$
   - Set $\sigma_i = y_{i-1} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K'_{i-1}, h_{w'[i-1]})$.
5. Output (w_i, σ_i)

Figure 13: Program $\mathsf{RefGen}'_6[\mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, K_\lambda^{(j^*+1)}, K'^{(h)}_{j^*+1}, K_{\mathsf{bit}}, \{h_j\}, \sigma'_{j^*+1}, z'_{j^*+1}, \beta_{j^*+1}]$.

- $\mathsf{sExp}_7^{(S,j^*)}$: In this experiment, we undo the change of Exp_4.
  1. The challenger samples the following keys:
     (d) $K_\lambda \leftarrow \mathsf{KeyGen}(1^\lambda, 1^{\log \ell}, 1^{2\lambda})$ is a puncturable PRF key with domain $[\ell]$ and 2λ bit outputs.
     ii. Sample $y'_{j^*+1}, r'_{j^*+1} \xleftarrow{R} \{0,1\}^\lambda$. Set $z'_{j^*+1} = F(y'_{j^*+1})$ and $\beta_{j^*+1} = \mathsf{hcb}(y'_{j^*+1})$.
- $\mathsf{sExp}_8^{(S,j^*)}$: In this experiment, we undo the change of Exp_3.
  1. The challenger samples the following keys:
     (d) $K_\lambda \leftarrow \mathsf{KeyGen}(1^\lambda, 1^{\log \ell}, 1^{2\lambda})$ is a puncturable PRF key with domain $[\ell]$ and 2λ bit outputs.
     ii. Sample $y'_{j^*+1}, r'_{j^*+1} \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda, j^*+1)$. Set $z'_{j^*+1} = F(y'_{j^*+1})$ and $\beta_{j^*+1} = \mathsf{hcb}(y'_{j^*+1})$.

- $\mathsf{sExp}_9^{(S,j^*)}$: In this experiment, we undo the change of Exp_2.

Constants: Hash keys $\mathsf{hk}_{\mathbf{w}}, \mathsf{hk}_C$, hash h_C, puncturable PRF keys $K_\lambda, K_{\mathsf{bit}}$, hashes h_j for $j \in S \cup \{j^*+1\}$.
Inputs: Hash $h_{w_{[i-1]}}$, string σ_{i-1}, wire index i, gate description $C_i = (g_i, i_0, i_1)$, local opening $\pi_i^{(C)}$, wire value w_{i_b} and $\pi_{i_b}^{(w)}$ for $b \in \{0,1\}$.

1. Verify inputs are well formed. Abort with $\perp$ if any checks do not pass.
   (a) Check $i \in [n+1, \ell]$.
   (b) Check $\Pi_{\mathsf{PA}}.\mathsf{Verify}(\mathsf{hk}_C, h_C, i, C_i, \pi_i^{(C)}) = 1$.
   (c) For $b \in \{0,1\}$, check $\Pi_{\mathsf{PA}}.\mathsf{Verify}(\mathsf{hk}_{\mathbf{w}}, h_{w_{[i-1]}}, i_b, \pi_{i_b}^{(w)}) = w_{i_b}$.
   (d) Check σ_{i-1} is well formed:
      - Compute $(y_{i-1}, r'_{i-1}) \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda^{(j^*+1)}, i-1)$.
      - Compute $K'_{i-1} \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{|h|}, 1^\lambda; r'_{i-1})$
      - Check that $y_{i-1} = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K'_{i-1}, h_{w_{[i-1]}}) \oplus \sigma_{i-1}$.
2. If $i < \max(S)$ set $w_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i) \oplus g_i(w_{i_0} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i_0), w_{i_1} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i_1))$. Otherwise, set $w_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, i)$.
3. Compute $h_{w_{[i]}} = \Pi_{\mathsf{PA}}.\mathsf{Append}(\mathsf{hk}_{\mathbf{w}}, h_{w'[i-1]}, w_i)$
   - If $i-1 \in S \cup \{j^*+1\}$, check $h_{w_{[i-1]}} = h_{i-1}$. Abort with $\perp$ otherwise.
4. Compute σ_i as:
   - Compute $(y_i, r'_i) \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda^{(j^*+1)}, i-1)$.
   - Compute $K'_i \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{|h|}, 1^\lambda; r'_i)$
   - Set $\sigma_i = y_{i-1} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K'_{i-1}, h_{w'[i-1]})$.
5. Output (w_i, σ_i)

Figure 14: Program $\mathsf{RefGen}'_9[\mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, K_\lambda, K_{\mathsf{bit}}, \{h_i\}]$.

- $\mathsf{sExp}_{10}^{(S,j^*)}$: In this experiment, we undo the change of Exp_1.

1. The challenger samples the following keys:
   (a) Sample $\mathsf{hk}_{\mathbf{w}} \leftarrow \mathsf{Setup}(1^\lambda, 1, 1^2)$.

Claim 5.18. *Suppose Π_{PA} is setup indistinguishable. Then for any efficient algorithm $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that for all sets $S \subseteq [n, \ell]$, $j^* \in S$:*

$$\left|\Pr[\mathsf{Exp}_S(\mathcal{A})] - \Pr[\mathsf{sExp}_0^{(S,j^*)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

Proof. Suppose that there exists an efficient adversary $\mathcal{A}$ that can distinguish these two experiments with non-negligible probability ε. We use $\mathcal{A}$ to construct an adversary $\mathcal{B}$ that breaks setup indistinguishability game of Π_{PA} with the same advantage:

0. Reduction $\mathcal{B}$ runs $\mathcal{A}$, playing the role of the challenger as per Exp_S, and receives circuit C and input x.

1. $\mathcal{B}$ samples keys as per $\mathsf{sExp}_0^{(S,j^*)}$, *except*:
   (b) $\mathcal{B}$ submits block length $1 + 2\log \ell$ and input sequence C, and index set $\{j\}$ to the setup indistinguishability challenger, receiving hash key hk_C.

2. 3. 4. $\mathcal{B}$ continues to sample inputs as per exp_S, eventually sending

$$s = (\mathsf{hk}_{\mathsf{sd}}, \mathsf{hk}_C, \mathsf{sd}, \sigma_n, i\mathcal{O}(1^\lambda, \mathsf{RefGen}'[\mathsf{hk}_C, \mathsf{hk}_{\mathsf{sd}}, K_\lambda, K_{\mathsf{bit}}, \{h_i\}_{i\in S}]))$$

to $\mathcal{A}$, who returns a bit b' which $\mathcal{B}$ simply forwards to the setup indistinguishability challenger.

Observe that the only difference

□

Claim 5.19. *Suppose Π_{PA} is setup indistinguishable. Then for any efficient algorithm $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that for all sets $S \subseteq [n, \ell]$, $j^* \in S$:*

$$\left|\Pr[\mathsf{sExp}_0^{(S,j^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_1^{(S,j^*)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

This claim follows by an analogous argument to Claim 5.18.

Claim 5.20. *Suppose Π_{PPRF} is a perfectly correct puncturable PRF, Π_{PA} is read and write enforcing, F is injective, and $i\mathcal{O}$ is a secure indistinguishability obfuscator. Then for any efficient algorithm $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that for all sets $S \subseteq [n, \ell]$, $j^* \in S$:*

$$\left|\Pr[\mathsf{sExp}_1^{(S,j^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_2^{(S,j^*)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

Proof.

Claim 5.21. *Program RefGen' (Fig. 10) in $\mathsf{sExp}_1^{(S,j^*)}$ program RefGen_2' (Fig. 11) and $\mathsf{sExp}_2^{(S,j^*)}$ are functionally equivalent.*

Proof. Observe we changed program RefGen_2' in two main locations

- When verifying σ_{i-1} is well formed, we add two new checks:
  - On hash $h_{w_{j^*+1}} = h_{j^*+1}$, RefGen' checks $\sigma_{i-1} = \sigma'_{j^*+1}$. However, recall $\sigma'_{j^*+1} = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K'_{j^*+1}, h_{j^*+1}) \oplus y'_{j^*+1}$, so this is equivalent to the check that $y'_{j^*+1} = \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(K'_{j^*+1}, h_{j^*+1}) \oplus \sigma_{i-1}$, which, by injectivity of F, is equivalent to checking $F(y'_{j^*+1}) = F(\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K'_{j^*+1}, h_{j^*+1}) \oplus \sigma_{i-1})$. Since $\beta_{j^*+1} = \mathsf{hcb}(y'_{j^*+1})$, this additional check never fails when the the former check passes.
  - On other hashes for index $i - 1 = j^* + 1$, we can see that by punctured correctness, $K'^{(h)}_{j^*+1}$ evaluates equivalently to $K'^{(h)}_{j^*+1} = \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{|h|}, 1^\lambda; r'_{j^*+1})$, and since $z'_{j^*+1} = F(y_{j^*+1})$ and $\beta_{j^*+1} = \mathsf{hcb}(y_{j^*+1})$, this is exactly how RefGen' verifies the same inputs.
- when computing σ_i on $i = j^* + 1$, rather than using the PPRF, we instead directly output hard coded σ'_{j^*+1}. Observe that since $j^* \in S$, we have directly checked that $h'_{[w-1]} = h_j$. By read-enforcement of hk_C and $\mathsf{hk}_{\mathbf{w}}$, we can see our hash statistically binds to the bits w_{j_0}, w_{j_1}, which fixes w_{j^*+1} (otherwise $\Pi_{\mathsf{PA}}.\mathsf{Verify}$ fails on our program outputs $\perp$). By write-enforcement of $\mathsf{hk}_{\mathbf{w}}$, this means for RefGen' to have not aborted with $\perp$, $h_{w_{[i]}}$ must equal h_{j^*+1}, and so σ'_{j^*+1} is the signature that would have been output by both programs.

Taking these two parts together, we can see our programs $\mathsf{RefGen}', \mathsf{RefGen}_2'$ are functionally equivalent. □

Once Claim 5.21 is established, this follows from a reduction to $i\mathcal{O}$ security analogous to Lemma 5.7.

□

Claim 5.22. *Suppose Π_{PPRF} is pseudorandom. Then for any efficient algorithm $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that for all sets $S \subseteq [n, \ell]$, $j^* \in S$:*

$$\left|\Pr[\mathsf{sExp}_2^{(S,j^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_3^{(S,j^*)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

This claim follows by an analogous argument to Claim 5.15.

Claim 5.23. *Suppose* hcb *is a hardcore bit of F. Then for any efficient algorithm $\mathcal{A}$, there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that for all sets $S \subseteq [n, \ell]$, $j^* \in S$:*

$$\left|\Pr[\mathsf{sExp}_3^{(S,j^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_4^{(S,j^*)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

Proof. Observe these experiments only differ on the distribution of F_{j^*+1} as either the hardcore bit of F on a uniformly random string y'_{j^*+1} or as $1 \oplus$ that bit. Since y'_{j^*+1} only appears in the final distribution through this bit and z'_{j^*+1}, these distributions are indistinguishable by hardcore bit security. □

Claim 5.24. *Suppose $i\mathcal{O}$ is a secure indistinguishability obfuscator. Then for any efficient algorithm $\mathcal{A}$, there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that for all sets $S \subseteq [n, \ell]$, $j^* \in S$:*

$$\left|\Pr[\mathsf{sExp}_4^{(S,j^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_5^{(S,j^*)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

Proof.

Claim 5.25. *Program* RefGen'_2 *(Fig. 11) in* $\mathsf{sExp}_4^{(S,j^*)}$ *program* RefGen'_5 *(Fig. 12) and* $\mathsf{sExp}_5^{(S,j^*)}$ *are functionally equivalent.*

Proof. Observe here that since β_{j^*+1} is set to $\mathsf{hcb}(y'_{j^*+1}) \oplus 1$. Since F is injective, y'_{j^*+1} is the only preimage of z'_{j^*+1}, so the check of σ_j being well formed in Step 1d passes when $h_{\mathbf{w}_{[j]}} = h_j$, which is equivalent to the added check. Thus, repeating this check in Step 3 is functionally equivalent. □

Once Claim 5.25 is established, this follows from a reduction to $i\mathcal{O}$ security analogous to Lemma 5.7. □

Lemma 5.26. *Suppose Π_{PPRF} is a secure and perfectly correct puncturable PRF, and $i\mathcal{O}$ is a secure indistinguishability obfuscator. Then for any efficient algorithm $\mathcal{A}$, there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that for all sets $S \subseteq [n, \ell]$, $j^* \in S$:*

$$\left|\Pr[\mathsf{sExp}_5^{(S,j^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_6^{(S,j^*)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

Observe when $j^* \neq \max(S)$, these experiments are identical. Thus, it remains to show this is true when $j^* = \max(S)$. We defer the proof of this lemma to Section 5.5.

Claim 5.27. *Suppose* hcb *is a hardcore bit of F. Then for any efficient algorithm $\mathcal{A}$, there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that for all sets $S \subseteq [n, \ell]$, $j^* \in S$:*

$$\left|\Pr[\mathsf{sExp}_6^{(S,j^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_7^{(S,j^*)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

This claim follows by an analogous argument to Claim 5.23.

Claim 5.28. *Suppose Π_{PPRF} is pseudorandom. Then for any efficient algorithm $\mathcal{A}$, there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that for all sets $S \subseteq [n, \ell]$, $j^* \in S$:*

$$\left|\Pr[\mathsf{sExp}_7^{(S,j^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_8^{(S,j^*)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

This claim follows by an analogous argument to Claim 5.22.

Claim 5.29. *Suppose Π_{PPRF} is a perfectly correct puncturable PRF, Π_{PA} is read enforcing, and $i\mathcal{O}$ is a secure indistinguishability obfuscator. Then for any efficient algorithm $\mathcal{A}$, there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that for all sets $S \subseteq [n, \ell]$, $j^* \in S$:*

$$\left|\Pr[\mathsf{sExp}_8^{(S,j^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_9^{(S,j^*)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

This claim follows by an analogous argument to Claim 5.20.

Claim 5.30. *Suppose Π_{PA} is setup indistinguishable. Then for any efficient algorithm $\mathcal{A}$, there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that for all sets $S \subseteq [n, \ell]$, $j^* \in S$:*

$$\left|\Pr[\mathsf{sExp}_9^{(S,j^*)}(\mathcal{A})] - \Pr[\mathsf{sExp}_{10}^{(S,j^*)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

This claim follows by an analogous argument to Claim 5.19.

Claim 5.31. *Suppose Π_{PA} is setup indistinguishable. Then for any efficient algorithm $\mathcal{A}$, there exists a negligible function* $\mathsf{negl}(\lambda)$ *such that for all sets $S \subseteq [n, \ell]$, $j^* \in S$:*

$$\left|\Pr[\mathsf{sExp}_{10}^{(S,j^*)}(\mathcal{A})] - \Pr[\mathsf{Exp}_{S\cup\{j^*+1\}}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

This claim follows by an analogous argument to Claim 5.18.

Taking together Claims 5.18 to 5.20, 5.22 and 5.23, we have the proof of Lemma 5.9. □

5.5 Extending Evaluation Encoding

In this section, we will show Lemma 5.26, the indistinguishability between $\mathsf{sExp}_5^{(S,j)}$ and $\mathsf{sExp}_6^{(S,j)}$ deferred from the prior section. As mentioned previously, these hybrids are identical except when $\max(S) = j$, which will be the focus of our analysis below.

Proof. To show this, we will go through another series of subexperiments:

- $\mathsf{iExp}_0^{(S,j)}$: In this experiment, we puncture K_{bit} on $j+1$ and hard code the corresponding outputs into the program (Fig. 15).

  1. The challenger samples the following keys:

     (c) $K_{\mathsf{bit}} \leftarrow \mathsf{KeyGen}(1^\lambda, 1^{\log \ell}, 1)$. Then sample $K_{\mathsf{bit}}^{(j+1)} \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Puncture}(K_{\mathsf{bit}}, j+1)$ and set $u_{j+1} \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, j+1)$.

  3. Output $(\mathsf{hk}_{\mathbf{w}}, \mathsf{hk}_C, C, x, w, \sigma_n, i\mathcal{O}(1^\lambda, \mathsf{RefGen}_0''[\mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, K_\lambda^{(j+1)}, K_{j+1}'^{(h)}, K_{\mathsf{bit}}^{(j+1)}, \{h_i\}, \sigma_{j+1}', z_{j+1}', \beta_{j+1}, u_{j+1}]))$.

Constants: Hash keys $\mathsf{hk}_{\mathbf{w}}, \mathsf{hk}_C$, hash h_C, puncturable PRF keys $K_\lambda^{(j+1)}, K_{\mathsf{bit}}^{(j+1)}, K_{j+1}'^{(h)}$, hashes h_i for $i \in S \cup \{j+1\}$, bitstrings σ'_{j+1}, z'_{j+1}, bit u_{j+1}.

Inputs: Hash $h_{w_{[i-1]}}$, string σ_{i-1}, wire index i, gate description $C_i = (g, i_0, i_1)$, local opening $\pi_i^{(C)}$, wire value w_{i_b} and $\pi_{i_b}^{(w)}$ for $b \in \{0,1\}$.

1. Verify inputs are well formed. Abort with $\perp$ if any checks do not pass.
   (a) Check $i \in [n+1, \ell]$.
   (b) Check $\Pi_{\mathsf{PA}}.\mathsf{Verify}(\mathsf{hk}_C, h_C, i, C_i, \pi_i^{(C)}) = 1$.
   (c) For $b \in \{0,1\}$, check $\Pi_{\mathsf{PA}}.\mathsf{Verify}(\mathsf{hk}_{\mathbf{w}}, h_{w_{[i-1]}}, i_b, \pi_{i_b}^{(w)}) = w_{i_b}$.
   (d) Check σ_{i-1} is well formed:
   - If $i - 1 = j + 1$, check that:
     * If $h_{w_{[i-1]}} = h_{j+1}$, check that $\sigma_{i-1} = \sigma'_{j+1}$.
     * Otherwise, check that $z'_{j+1} = F(\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{j+1}'^{(h)}, h_{w_{[i-1]}}) \oplus \sigma_{i-1})$ and $\beta_{j+1} = \mathsf{hcb}(\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{j+1}'^{(h)}, h_{w_{[i-1]}}) \oplus \sigma_{i-1})$.
   - Otherwise, compute $(y_{i-1}, r'_{i-1}) \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda^{(j+1)}, i-1)$.
   - Compute $K'_{i-1} \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{|h|}, 1^\lambda; r'_{i-1})$
   - Check that $y_{i-1} = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K'_{i-1}, h_{w_{[i-1]}}) \oplus \sigma_{i-1}$.
2. If $i < \max(S)$ set $w_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}^{(j+1)}, i) \oplus g(w_{i_0} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}^{(j+1)}, i_0), w_{i_1} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}^{(j+1)}, i_1))$. If $i = j+1$, set $w_i = u_{j+1}$. Otherwise, set $w_i = \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}^{(j+1)}, i)$.
3. Compute $h_{w_{[i]}} = \Pi_{\mathsf{PA}}.\mathsf{Append}(\mathsf{hk}_{\mathbf{w}}, h_{w'[i-1]}, w_i)$
   - If $i - 1 \in S \cup \{j+1\}$, check $h_{w_{[i-1]}} = h_{i-1}$. Abort with $\perp$ otherwise.
4. Compute σ_i as:
   - If $i = j + 1$, return σ'_{j+1}.
   - Otherwise, compute $(y_i, r'_i) \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_\lambda^{(j+1)}, i-1)$.
   - Compute $K'_i \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{KeyGen}(1^\lambda, 1^{|h|}, 1^\lambda; r'_i)$
   - Set $\sigma_i = y_{i-1} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K'_{i-1}, h_{w'[i-1]})$.
5. Output (w_i, σ_i)

Figure 15: Program $\mathsf{RefGen}_0''[\mathsf{hk}_C, \mathsf{hk}_{\mathbf{w}}, h_C, K_\lambda^{(j+1)}, K_{j+1}'^{(h)}, K_{\mathsf{bit}}^{(j+1)}, \{h_i\}, \sigma'_{j+1}, z'_{j+1}, \beta_{j+1}, u_{j+1}]$.

- $\mathsf{iExp}_1^{(S,j)}$: In this experiment, we switch the hard coded value of K_{bit} evaluated at $j+1$ to a uniformly random value.
  1. The challenger samples the following keys:
     (c) $K_{\mathsf{bit}} \leftarrow \mathsf{KeyGen}(1^\lambda, 1^{\log \ell}, 1)$. Then sample $K_{\mathsf{bit}}^{(j+1)} \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Puncture}(K_{\mathsf{bit}}, j+1)$ and set $u_{j+1} \xleftarrow{R} \{0,1\}$.
- $\mathsf{iExp}_2^{(S,j)}$: In this experiment, we extend computing w_i with gate evaluation to index $j+1$.
  1. The challenger samples the following keys:
     (c) $K_{\mathsf{bit}} \leftarrow \mathsf{KeyGen}(1^\lambda, 1^{\log \ell}, 1)$. Then sample $K_{\mathsf{bit}}^{(j+1)} \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Puncture}(K_{\mathsf{bit}}, j+1)$ and set $u_{j+1} \leftarrow \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, j+1) \oplus g(w_{j_0} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, j_0), w_{j_1} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, j_1))$.

Claim 5.32. *Suppose Π_{PPRF} is a perfectly correct puncturable PRF and $i\mathcal{O}$ is a secure indistinguishability obfuscator. Then for any efficient algorithm $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that for all sets $S \subseteq [n+1, \ell]$, $j \in S$:*

$$\left|\Pr[\mathsf{sExp}_5^{(S,j)}(\mathcal{A})] - \Pr[\mathsf{iExp}_0^{(S,j)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

Proof. Note here we simply puncture K_{bit} in program RefGen_0'' and hard code the corresponding outputs. By perfect correctness of PPRF, these programs are functionally equivalent, so we can rely on $i\mathcal{O}$ security analogous to Lemma 5.7. □

Claim 5.33. *Suppose Π_{PPRF} is pseudorandom. Then for any efficient algorithm $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that for all sets $S \subseteq [n+1, \ell]$, $j \in S$:*

$$\left|\Pr[\mathsf{iExp}_0^{(S,j)}(\mathcal{A})] - \Pr[\mathsf{iExp}_1^{(S,j)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

This claim follows by an analogous argument to Claim 5.15.

Claim 5.34. *Suppose Π_{PPRF} is pseudorandom. Then for any efficient algorithm $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that for all sets $S \subseteq [n+1, \ell]$, $j \in S$:*

$$\left|\Pr[\mathsf{iExp}_1^{(S,j)}(\mathcal{A})] - \Pr[\mathsf{iExp}_2^{(S,j)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

This claim follows by an analogous argument to Claim 5.15. Note here $g(w_{j_0} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, j_0), w_{j_1} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, j_1))$ is simply a constant.

Claim 5.35. *Suppose Π_{PPRF} is a perfectly correct puncturable PRF, Π_{PA} is read enforcing, and $i\mathcal{O}$ is a secure indistinguishability obfuscator. Then for any efficient algorithm $\mathcal{A}$, there exists a negligible function $\mathsf{negl}(\lambda)$ such that for all sets $S \subseteq [n+1, \ell]$, $j \in S$:*

$$\left|\Pr[\mathsf{iExp}_2^{(S,j)}(\mathcal{A})] - \Pr[\mathsf{sExp}_6^{(S,j)}(\mathcal{A})]\right| \leq \mathsf{negl}(\lambda).$$

Proof.

Claim 5.36. *Program RefGen_0'' (Fig. 15) in $\mathsf{iExp}_2^{(S,j)}$ program RefGen_6' (Fig. 13) and $\mathsf{sExp}_6^{(S,j)}$ are functionally equivalent.*

Proof. Since by $\mathsf{iExp}_2^{(S,j)}$, u_{j+1} is being set as $\Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, j{+}1) \oplus g(w_{j_0} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, j_0), w_{j_1} \oplus \Pi_{\mathsf{PPRF}}.\mathsf{Eval}(K_{\mathsf{bit}}, j_1))$, this is equivalent to how w_{j+1} is computed in $\mathsf{sExp}_6^{(S,j)}$. Since PA is read enforcing, we know openings for gate $j+1$ must use have input wires j_0, j_1, and similarly, $h_{w_{[j]}}$ is binding to w_{j_0}, w_{j_1} on these wires. Taken together, along with perfect correctness of PPRF while removing the punctured point on K_{bit} on point $j+1$, we get that these programs are functionally equivalent □

Once Claim 5.36 is established, this follows from a reduction to $i\mathcal{O}$ security analogous to Lemma 5.7. □

Taking together Claims 5.32 to 5.35, we get the proof of our Lemma 5.26.

□

Acknowledgments

Brent Waters is supported by NSF CNS-2318701 and a Simons Investigator award.

References

[ABF+19] Benny Applebaum, Amos Beimel, Oriol Farràs, Oded Nir, and Naty Peter. Secret-sharing schemes for general and uniform access structures. In *EUROCRYPT*, 2019.

[ABI+23] Benny Applebaum, Amos Beimel, Yuval Ishai, Eyal Kushilevitz, Tianren Liu, and Vinod Vaikuntanathan. Succinct computational secret sharing. In *STOC*. ACM, 2023.

[ABNP20] Benny Applebaum, Amos Beimel, Oded Nir, and Naty Peter. Better secret sharing via robust conditional disclosure of secrets. In *STOC*. ACM, 2020.

[ADM+25] Gennaro Avitabile, Nico Döttling, Bernardo Magri, Christos Sakkas, and Stella Wohnig. Signature-based witness encryption with compact ciphertext. In *International Conference on the Theory and Application of Cryptology and Information Security*, 2025.

[AN21] Benny Applebaum and Oded Nir. Upslices, downslices, and secret-sharing with complexity of 1.5^n. In *CRYPTO*, 2021.

[ASY22] Damiano Abram, Peter Scholl, and Sophia Yakoubov. Distributed (correlation) samplers: how to remove a trusted dealer in one round. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, 2022.

[BBK+23] Zvika Brakerski, Maya Farber Brodsky, Yael Tauman Kalai, Alex Lombardi, and Omer Paneth. SNARGs for monotone policy batch NP. In *CRYPTO*, 2023.

[BCG+19] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. Efficient pseudorandom correlation generators: Silent ot extension and more. In *CRYPTO*, 2019.

[Bei96] Amos Beimel. *Secure Schemes for Secret Sharing and Key Distribution*. PhD thesis, Technion, 1996.

[BFK+19] Saikrishna Badrinarayanan, Rex Fernando, Venkata Koppula, Amit Sahai, and Brent Waters. Output compression, mpc, and io for turing machines. In *International Conference on the Theory and Application of Cryptology and Information Security*, 2019.

[BGI+12] Boaz Barak, Oded Goldreich, Russell Impagliazzo, Steven Rudich, Amit Sahai, Salil P. Vadhan, and Ke Yang. On the (im)possibility of obfuscating programs. *J. ACM*, 2012.

[BGI14] Elette Boyle, Shafi Goldwasser, and Ioana Ivan. Functional signatures and pseudorandom functions. In *International workshop on public key cryptography*. Springer, 2014.

[BGL+15] Nir Bitansky, Sanjam Garg, Huijia Lin, Rafael Pass, and Sidharth Telang. Succinct randomized encodings and their applications. In *STOC*, 2015.

[BL88] Josh Cohen Benaloh and Jerry Leichter. Generalized secret sharing and monotone functions. In *CRYPTO*, 1988.

[Bla79] G. R. Blakley. Safeguarding cryptographic keys. In *1979 International Workshop on Managing Requirements Knowledge, MARK*. IEEE, 1979.

[BM82] Manuel Blum and Silvio Micali. How to generate cryptographically strong sequences of pseudo random bits. In *FOCS*. IEEE Computer Society, 1982.

[BOGW88] Michael Ben-Or, Shafi Goldwasser, and Avi Wigderson. Completeness theorems for non-cryptographic fault-tolerant distributed computation. In *Proceedings of the twentieth annual ACM symposium on Theory of computing*, 1988.

[BPW15] Nir Bitansky, Omer Paneth, and Daniel Wichs. Perfect structure on the edge of chaos: Trapdoor permutations from indistinguishability obfuscation. In *Theory of Cryptography Conference*. Springer, 2015.

[BW13] Dan Boneh and Brent Waters. Constrained pseudorandom functions and their applications. In *ASIACRYPT*. Springer, 2013.

[CCD88] David Chaum, Claude Crépeau, and Ivan Damgard. Multiparty unconditionally secure protocols. In *Proceedings of the twentieth annual ACM symposium on Theory of computing*, 1988.

[CHJV15] Ran Canetti, Justin Holmgren, Abhishek Jain, and Vinod Vaikuntanathan. Succinct garbling and indistinguishability obfuscation for RAM programs. In *STOC*, 2015.

[CJJ21a] Arka Rai Choudhuri, Abhishek Jain, and Zhengzhong Jin. Non-interactive batch arguments for NP from standard assumptions. In *CRYPTO*, 2021.

[CJJ21b] Arka Rai Choudhuri, Abhishek Jain, and Zhengzhong Jin. SNARGs for P from LWE. In *FOCS*, 2021.

[Csi96] László Csirmaz. The dealer's random bits in perfect secret sharing schemes. *Studia Scientiarum Mathematicarum Hungarica*, 1996.

[Csi97] László Csirmaz. The size of a share must be large. *Journal of cryptology*, 1997.

[GGH+13] Sanjam Garg, Craig Gentry, Shai Halevi, Mariana Raykova, Amit Sahai, and Brent Waters. Candidate indistinguishability obfuscation and functional encryption for all circuits. In *FOCS*, 2013.

[GGI+15] Craig Gentry, Jens Groth, Yuval Ishai, Chris Peikert, Amit Sahai, and Adam Smith. Using fully homomorphic hybrid encryption to minimize non-interative zero-knowledge proofs. *Journal of Cryptology*, 2015.

[GL89] Oded Goldreich and Leonid A. Levin. A hard-core predicate for all one-way functions. In *STOC*. ACM, 1989.

[GM82] Shafi Goldwasser and Silvio Micali. Probabilistic encryption & how to play mental poker keeping secret all partial information. In *Proceedings of the fourteenth annual ACM symposium on Theory of computing*, 1982.

[GPSZ17] Sanjam Garg, Omkant Pandey, Akshayaram Srinivasan, and Mark Zhandry. Breaking the sub-exponential barrier in obfustopia. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 2017.

[GS18] Sanjam Garg and Akshayaram Srinivasan. A simple construction of io for turing machines. In *TCC*, 2018.

[GSWW22] Rachit Garg, Kristin Sheridan, Brent Waters, and David J. Wu. Fully succinct batch arguments for sfnp from indistinguishability obfuscation. In *TCC*, 2022.

[HIJ+16] Shai Halevi, Yuval Ishai, Abhishek Jain, Eyal Kushilevitz, and Tal Rabin. Secure multiparty computation with general interaction patterns. In *Proceedings of the 2016 ACM Conference on Innovations in Theoretical Computer Science*, 2016.

[HW15] Pavel Hubácek and Daniel Wichs. On the communication complexity of secure function evaluation with long output. In *ITCS*, 2015.

[ISN89] Mitsuru Ito, Akira Saito, and Takao Nishizeki. Secret sharing scheme realizing general access structure. *Electronics and Communications in Japan Part Iii-fundamental Electronic Science*, 1989.

[JJ22] Abhishek Jain and Zhengzhong Jin. Indistinguishability obfuscation via mathematical proofs of equivalence. In *FOCS*, 2022.

[KLW15] Venkata Koppula, Allison Bishop Lewko, and Brent Waters. Indistinguishability obfuscation for turing machines with unbounded memory. In *STOC*. ACM, 2015.

[KPTZ13] Aggelos Kiayias, Stavros Papadopoulos, Nikos Triandopoulos, and Thomas Zacharias. Delegatable pseudorandom functions and applications. In *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*, 2013.

[LV18] Tianren Liu and Vinod Vaikuntanathan. Breaking the circuit-size barrier in secret sharing. In *Proceedings of the 50th Annual ACM SIGACT Symposium on Theory of Computing*, 2018.

[LVW18] Tianren Liu, Vinod Vaikuntanathan, and Hoeteck Wee. Towards breaking the exponential barrier for general secret sharing. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 2018.

[NWW24] Shafik Nassar, Brent Waters, and David J Wu. Monotone policy bargs from bargs and additively homomorphic encryption. In *Theory of Cryptography Conference*, 2024.

[OPWW15] Tatsuaki Okamoto, Krzysztof Pietrzak, Brent Waters, and Daniel Wichs. New realizations of somewhere statistically binding hashing and positional accumulators. In *ASIACRYPT*, 2015.

[RBO89] Tal Rabin and Michael Ben-Or. Verifiable secret sharing and multiparty protocols with honest majority. In *Proceedings of the twenty-first annual ACM symposium on Theory of computing*, 1989.

[Sha79] Adi Shamir. How to share a secret. *Commun. ACM*, 1979.

[VNS+03] Vinod Vaikuntanathan, Arvind Narayanan, K. Srinathan, C. Pandu Rangan, and Kwangjo Kim. On the power of computational secret sharing. In *INDOCRYPT*, 2003.

[WW24] Brent Waters and David J. Wu. A pure indistinguishability obfuscation approach to adaptively-sound snargs for NP. *IACR Cryptol. ePrint Arch.*, 2024.

[Yao82] Andrew C Yao. Theory and application of trapdoor functions. In *23rd Annual Symposium on Foundations of Computer Science (SFCS 1982)*. IEEE, 1982.

[Yao89] Andrew Chi-Chih Yao. Unpublished manuscript, 1989.